

Từ zero tới năng lực QA với Playwright + SWP391

Teaching Volume v4.0

JavaScript/TypeScript từ số 0, Playwright thực chiến và Spring reverse

Zero-ready teaching • runnable labs • work evidence • mastery gates • canonical Laundry transfer

Personalized self-study edition v4.0 • August 2026

Mục lục

00 - Hợp đồng học tập cá nhân hóa	5
Part I - Nền tảng JavaScript/TypeScript cho Automation Tester từ zero	10
QA00 - Máy của m đang chạy cái gì?	13
JS01 - Value, variable và biểu thức: chương trình đang giữ dữ liệu gì?	17
JS02 - Condition và loop: chương trình chọn nhánh nào?	22
JS03 - Function: đặt tên cho một việc và kiểm soát input/output	26
JS04 - Array và object: model data, không bám index	30
JS05 - Module, package, JSON và stack trace: code được ghép lại thế nào?	34
JS06 - Promise và async/await: công việc hoàn tất ở tương lai	38
JS07 - Test tự động cơ bản: arrangement, action và oracle	43
JS08 - Cầu nối vào Playwright: đọc một spec từ trái sang phải	47
TS01 - TypeScript strict: feedback trước khi chạy	51
TS02 - Domain types: union, literal, optional, readonly và discriminated data	55
TS03 - Unknown, narrowing và boundary validation: đừng tin dữ liệu ngoài	59
TS04 - TypeScript trong Playwright: fixture, helper và failure layers	63
Foundation Capstone - 90 phút không AI	67
Review queue cho ba tuần foundation	69
Foundation definition of learned	70
PHẦN II - Kỹ năng QA nền: test design, API, SQL, Git, CI và English	71
J01 - QA fundamentals: từ requirement đến test có lý do	73
J02 - HTTP và API testing: nhìn xuyên qua UI	77
J03 - SQL as oracle: kiểm tra dữ liệu mà không đoán	81
J04 - Git workflow: thay đổi có thể hiểu, review và phục hồi	85

J05 - CI/CD đủ dùng: test phải sống được trên máy sạch	88
J06 - Docker operational literacy: vận hành dependency, không học lan sang DevOps	92
J07 - English for QA: truyền evidence không làm người khác đoán	95
4. Runway integration gate - kết thúc ba tuần	98
5. SWP integrated technical gate - cuối hai tháng	99
PHẦN III - Playwright từ bài test đầu đến suite có thể tin	101
P01 - Anatomy của một test và oracle	102
P02 - Locator là query recipe, không phải tọa độ	104
P03 - Actionability, auto-wait và web-first assertion	106
P04 - Form, table, file, dialog và browser context	108
P05 - Isolation, hooks và data ownership	111
P06 - Authentication, authorization, session và CSRF	113
P07 - Config, project, fixture và suite architecture	115
P08 - APIRequestContext và UI-API hybrid	117
P09 - Debug bằng evidence, không bằng ritual	119
P10 - CI cho test automation ở mức đi làm	122
P11 - Company-style capstone ticket	124
PHẦN IV - Java/Spring reverse và tự làm toàn bộ SWP391 Laundry	126
J00 - Java reading bridge cho người zero	127
S01 - Từ browser tới Servlet container	130
S02 - DispatcherServlet, binding, validation và PRG	132
S03 - Service, transaction và business invariant	134
S04 - Repository, JPA, PostgreSQL và Flyway	136
S05 - Spring Security filter chain	138

S06 - Test pyramid của Laundry	140
S07 - Full reverse trace và random change	142
PHẦN V - Tám tuần tự làm full canonical Laundry	144
W1 - Scaffold và vertical core đầu tiên	146
W2 - Identity, Staff, customer và catalogue management	147
W3 - Order intake, search, detail và receipt	148
W4 - Revision, cancellation và processing lifecycle	150
W5 - Issues, immutable money và pickup	151
W6 - Public tracking, dashboard, reports, settings và schedule	152
W7 - Full regression, documentation và demo integrity	154
W8 - Defense và graduation	155

00 - Hợp đồng học tập cá nhân hóa

Bản này được thiết kế cho ai?

V4 không giả định m đã biết lập trình. Điểm xuất phát được xem là **zero có kinh nghiệm dùng AI**, nghĩa là m có thể đã nhìn thấy code nhiều lần nhưng chưa chắc tự tạo, giải thích, sửa biến thể hoặc debug được khi đóng cửa sổ chat.

Thông tin dùng để cá nhân hóa bản này:

- kỳ internship còn gần ba tháng;
- SWP391 bắt đầu sau khoảng ba tuần và kéo dài khoảng hai tháng;
- pass môn không phải nút thắt chính, nhưng m muốn tự hiểu và tự làm toàn bộ Laundry;
- m chưa tự tin ở JavaScript, Java, SQL, Git hay testing vì bốn năm phụ thuộc AI;
- đích nghề nghiệp sau internship nằm ở Cần Thơ hoặc Thành phố Hồ Chí Minh;
- tiếng Anh tự đánh giá khoảng IELTS 6.5, đủ để đọc tài liệu và phát triển kỹ năng giải thích kỹ thuật;
- m tự tìm việc; bộ này tập trung tạo năng lực, không biến ba tháng thành khóa viết CV.

Đích của v4 không phải gắn nhãn “Junior” vào tuần 12. Đích là tạo bằng chứng rằng m có thể làm bốn việc khi không có AI viết hộ:

1. biến requirement thành test condition, test case và bug report có lý do;
2. đọc, viết, chạy và debug Playwright bằng JavaScript/TypeScript;
3. lần một feature Laundry từ browser qua Spring tới PostgreSQL và tự sửa biến thể nhỏ;
4. đưa một suite có ý nghĩa lên CI, giải thích failure và bảo vệ quyết định của mình.

Ba lane, một năng lực chung

Lane	Việc thật cần đạt	Không biến thành
Internship	đọc repo, chạy một test, viết/sửa test nhỏ, thu evidence, hỏi đúng câu	copy convention mà không hiểu
SWP391	tự đi hết requirement -> UI -> Controller -> Service -> Repository/DB -> tests -> defense	chia phần xong chỉ biết file mình
Nghề QA Automation	testing foundation + JS/TS + Playwright + API + SQL + Git + CI	sưu tầm tool, học Docker/Kubernetes để trông “senior”

Ba lane dùng chung một mental model: **input** nào đi vào, **object/state** nào thay đổi, **outcome** nào người dùng **quan sát được**, **failure** xuất hiện ở **tầng** nào, **evidence** nào phân biệt các giả thuyết.

Phán quyết CI/CD và Docker

CI/CD nằm trong core nhưng chỉ học sau khi m đã có test ổn định. M phải biết đọc một workflow, chạy clean install, cung cấp environment an toàn, chạy suite, lấy report/trace artifact và phân biệt test fail với infrastructure fail.

Docker không phải nhánh học chính trong ba tháng này. M chỉ cần operational literacy khi project hoặc công ty dùng nó:

- image khác container ra sao;
- port host/container;
- environment variable;
- start, stop, inspect và logs;

- đọc một `compose.yaml` có sẵn để chạy app + PostgreSQL.

Chưa học sâu Dockerfile optimization, networking nâng cao, Kubernetes, cloud platform hay DevOps pipeline architecture. Thời gian đó có lợi hơn nếu đổ vào test design, JS/TS, Playwright, API, SQL và debug.

Nhịp thực tế: ba tuần runway, tám tuần SWP, một tuần consolidation

M không cần hoàn tất toàn bộ JavaScript trước khi nhìn Playwright. Tuy nhiên, do điểm xuất phát là zero, ba tuần đầu không được bỏ foundation gate.

Tuần 1-3	Zero runway QA thinking + runtime/Git + JS/TS + HTTP/DOM + Playwright nhỏ
Tuần 4-11	SWP term build toàn bộ Laundry theo vertical slices Playwright, API/HTTP evidence, SQL và Spring reverse đi cùng feature
Tuần 12	Consolidation random change + seeded bug + clean-run + defense + CI evidence

Đây là bản đồ tải, không phải lời hứa rằng mọi người zero đều mastery sau đúng 12 tuần. Foundation và Playwright core chứa khoảng hơn 100 giờ active practice. Nếu một tuần chỉ có sáu giờ, curriculum tự kéo dài; m không được “pass” gate bằng cách đọc đáp án để giữ lịch đẹp.

Một concept được dạy như thế nào

Mỗi lesson trong teaching volume dùng cùng một chuỗi. Nếu một lesson thiếu mất xích, đó là lỗi tài liệu chứ không phải lỗi kỷ luật của m.

Pha	M làm gì	Bằng chứng
Problem	gặp tình huống trước thuật ngữ	nói được tại sao concept cần tồn tại
Mental model	vẽ data, object, state hoặc timeline	giải thích bằng lời mình
Worked	đọc ví dụ hoàn chỉnh, dự đoán từng chặng	output/state prediction trước khi chạy
Completion	điền 30-50% phần còn thiếu	test đỏ đúng rồi xanh
Independent	chỉ nhận contract và examples	tự tạo code không copy cấu trúc
Failure injection	sửa một bug cài sẵn	hypothesis + decisive evidence
Transfer	đổi domain hoặc dùng trên Laundry/company-safe flow	làm được biến thể mới
Retrieval	quay lại D+1, D+3, D+7, D+14	trả lời/code từ trí nhớ

Không có “đọc hiểu rồi”. Một concept chỉ được đánh dấu learned khi m làm được independent, failure và transfer.

Syntax legend cho người zero

Trong code block, đọc ký hiệu theo thứ tự này:

```
const label = status === 'READY' ? 'Sẵn sàng' : 'Chưa sẵn sàng';
```

- `const label`: tạo một binding tên `label` và không gán lại binding này;
- `=`: lấy giá trị bên phải gán vào binding bên trái;
- `status === 'READY'`: so sánh cả giá trị và loại;
- `condition ? valueA : valueB`: nếu condition đúng lấy A, sai lấy B;
- `;`: kết thúc statement; trong JavaScript đôi khi được tự chèn, nhưng curriculum dùng rõ ràng.

Không cần thuộc tên mọi dấu trước khi học. Khi gặp cú pháp lạ, dùng ba câu:

1. Runtime phải làm hành động nào ở dòng này?
2. Dữ liệu đi vào từ đâu và giá trị đi ra đâu?
3. Nếu bỏ hoặc đổi phần này, lỗi thuộc syntax, type, runtime hay assertion?

Gate 10 điểm

Mỗi mission được chấm năm trục, mỗi trục 0-2.

Trục	0	1	2
Predict	chạy ngay hoặc đoán mò	đoán outcome nhưng lý do yếu	dự đoán state/output/failure và nêu why
Execute	chỉ chạy được solution	làm theo khung	tự hoàn tất contract từ starter
Explain	đọc lại code	kể happy path	kể object, state, timeline và decision
Diagnose	sửa mò/copy patch	tìm đúng vùng	hypothesis -> evidence -> root cause -> regression
Transfer	chỉ làm ví dụ cũ	đổi data nhỏ	làm flow/requirement khác không AI

Pass mặc định là **8/10 và không trục nào bằng 0**. Với gate cuối phase, Diagnose và Transfer đều phải đạt 2.

Test xanh chỉ chứng minh runtime chấp nhận outcome vừa kiểm tra. Nó không tự chứng minh m hiểu, test đủ risk, dữ liệu không leak hoặc code chịu được biến thể.

Luật chống brainrot do AI

AI vẫn được dùng, nhưng theo quyền tăng dần.

A0 - Closed book

Trong 10-20 phút đầu:

- viết prediction;
- đọc error;
- ghi expected/actual;
- nêu ít nhất hai hypothesis;
- chọn evidence tiếp theo.

Không hỏi AI “làm bài này”.

A1 - Socratic hint

Được hỏi một câu thu hẹp, ví dụ: “Evidence nào phân biệt locator sai với button bị overlay?” Không nhận code hoàn chỉnh.

A2 - Evidence review

Đưa prediction, error, trace/log và hypothesis của m. AI được phản biện reasoning hoặc gợi ý phép đo kế tiếp.

A3 - Patch review

Chỉ mở sau timebox và sau khi m đã ghi root-cause dự đoán. Nếu nhận patch, m phải:

1. giải thích từng thay đổi;
2. đóng patch;
3. tự làm một biến thể khác;
4. gây lại lỗi và chứng minh regression.

Hai buổi no-agent mỗi tuần

- một buổi 30-45 phút cho code/test;
- một buổi 30-45 phút cho SWP requirement/trace/debug.

Không cần hoàn thành feature lớn. Mục đích là đo kiến thức đang nằm trong đầu m hay trong cửa sổ chat.

Phiên 20, 45 và 90 phút

Phiên 20 phút - giữ mạch

1. hai phút đọc restart line;
2. năm phút retrieval không note;
3. mười phút làm một completion step hoặc đọc một failure;
4. ba phút ghi actual + next action.

Không mở concept mới.

Phiên 45 phút - mặc định

1. năm phút retrieval;
2. mười phút mental model/worked example;
3. hai mươi phút completion hoặc independent;
4. bảy phút failure/transfer;
5. ba phút restart line.

Phiên 90 phút - deep work

1. mười phút closed-book retrieval;
2. hai mươi phút worked + prediction;
3. ba mươi lăm phút independent task;
4. mười lăm phút seeded bug;
5. mười phút evidence, gate và review queue.

Không dùng phiên 90 phút để xem video liên tục.

Recovery không tạo “nợ học”

Nghỉ một hoặc hai ngày

Đọc restart line, làm prompt D+1 và tiếp tục đúng task đang dở.

Nghỉ ba đến sáu ngày

Làm recovery quiz 20 phút:

1. viết lại mental model từ trí nhớ;
2. chạy một example cũ;
3. sửa một biến thể nhỏ;
4. chọn đúng gate đang rớt.

Không reset phase.

Nghỉ từ bảy ngày

Lấy một evidence cũ và làm lại independent variant trong 45 phút. Nếu đạt, tiếp tục. Nếu không, quay lại đúng lesson prerequisite bị lộ, không học lại toàn bộ.

WIP bằng một

Mỗi thời điểm chỉ có một **NOW** mission. Công ty và SWP có thể phát sinh việc, nhưng control panel vẫn ghi một prerequisite trực tiếp đang mở khóa việc đó.

Ví dụ task công ty là sửa login test flaky:

```
NOW = P06 Auth state
Transfer = task login công ty
Parking lot = P08 suite architecture, Docker, Allure
```

M không vừa học auth, Docker, SQL window function và Spring Security trong cùng tối.

Definition of learned

Một mục được coi là học xong khi m có đủ:

- code/test hoặc artifact tự làm;
- prediction trước run;
- một failure được chẩn đoán bằng evidence;
- một transfer khác ví dụ mẫu;
- teach-back ba phút;
- review prompt được đặt lịch.

Đối với SWP, “done” còn yêu cầu trace:

```
actor outcome + business rule
-> screen/route/form
-> Controller
-> focused Service + transaction
-> Repository/JPA + table/Flyway
-> automated proof
-> demo/defense explanation
```

Setup tối thiểu

Làm theo `lab-kit/README.md`. Lần đầu m cần Node 22+, Git và Chromium của Playwright.

Java/PostgreSQL/Maven chỉ kích hoạt khi vào repo Laundry thật.

Sau cài đặt, ba command phải có ý nghĩa khác nhau:

```
npm run check:kit      kiểm tra reference infrastructure xanh
npm run typecheck      kiểm tra TypeScript compile contract
npm run status         nói rõ learner labs nào còn chưa pass
```

`check:kit` xanh không có nghĩa bài learner đã xong. `status` phải phân biệt reference infrastructure với work missions.

Bắt đầu đúng

Không làm baseline event-loop như v3. Người zero bắt đầu ở QA00 rồi JS01:

1. mở control panel, điền ngày bắt đầu và lịch SWP dự kiến;
2. làm QA00 requirement-to-test trước khi code;
3. chạy QA00 để thấy edit -> run -> error -> fix loop, rồi mới sang JS01;
4. chỉ mở hints sau timebox;
5. chốt một restart line trước khi nghỉ.

Hôm nay m không cần chứng minh mình “có tổ chất”. M chỉ cần tạo vòng feedback đầu tiên mà không giao quyền suy nghĩ cho AI.

Part I - Nền tảng JavaScript/TypeScript cho Automation Tester từ zero

Draft cho Self-Study Edition v4. File này là learner volume, chưa phải đáp án. Các bài completion và independent phải được làm trong lab kit; lời giải đầy đủ chỉ mở ở H5 sau timebox.

Part I giúp m làm được gì?

Part này không cố biến m thành frontend developer. Nó xây đúng lượng JavaScript và TypeScript cần để m:

- đọc một Playwright spec mà không bị từng ký hiệu làm ngẫn;
- tự viết helper, test data và assertion có chủ đích;
- hiểu Promise, `async`, `await` thay vì thêm `await` theo cảm giác;
- đọc TypeScript error và dùng type để chặn sai sót trước khi chạy;
- phân biệt lỗi môi trường, syntax, type, test và lỗi behavior;
- debug từ expected/actual/hypothesis/evidence thay vì sửa mò;
- chuyển kiến thức sang flow Laundry và suite ở công ty.

Ví dụ trong Part I dùng service, order, status, phone và transaction của miền Laundry để giảm số context m phải giữ. Đây vẫn là **foundation sandbox**. Production Laundry giữ nguyên Java 21, Spring Boot, Thymeleaf, PostgreSQL và các business rule canonical. TypeScript chỉ là ngôn ngữ của automation suite. Không copy phép tính JavaScript thành nguồn giá chính thức ở browser, không thêm REST endpoint chỉ để bài test dễ viết.

Cách học mỗi chapter

Mỗi chapter chạy theo cùng một vòng:

1. Đọc mental model và tự kể lại bằng lời của m.
2. Đọc worked example theo từng bước, không copy vội.
3. Ghi prediction trước khi chạy bất kỳ snippet nào.
4. Làm completion task với khung có sẵn.
5. Đóng example và làm independent task.
6. Kích hoạt failure injection, ghi expected/actual/hypothesis/evidence.
7. Làm transfer sang một ngữ cảnh khác.
8. Chấm gate và đặt review D+1, D+3, D+7.

Thời lượng ghi ở mỗi chapter là **active practice**, đã gồm prediction, bug và transfer. Nó không phải thời gian đọc lướt. Một chapter 4 giờ nên được chia thành bốn đến sáu phiên. Khi mệt, chỉ làm một prediction hoặc một failure case rồi ghi restart line.

Bản đồ artifact executable - authority cho Part I

Mọi path dưới đây tính từ root `lab-kit/`. Nếu text bài học và file tree không đồng ý, dừng lại và chạy validator; không tự đoán một tên file khác. `work/` là chỗ m sửa, `reference/` chỉ mở ở H5. Starter được phép đỏ vì TODO; `reference` bắt buộc xanh.

ID	Completion/independent artifact thật	Focused command
QA00	<code>foundations/work/qa00.mjs</code>	<code>npm run lab:qa00</code>
JS01	<code>foundations/work/js01.mjs</code> , <code>foundations/work/js01_service_snapshot.mjs</code>	<code>npm run lab:js01</code>

ID	Completion/independent artifact thật	Focused command
JS02	foundations/work/js02.mjs	npm run lab:js02
JS03	foundations/work/js03.mjs	npm run lab:js03
JS04	foundations/work/js04.mjs	npm run lab:js04
JS05	foundations/work/js05.mjs và ba module js05-*.mjs cạnh nó	npm run lab:js05
JS06	foundations/work/js06.mjs	npm run lab:js06
JS07	production helper foundations/work/js07.mjs; m viết foundations/work/js07.learner.test.mjs	npm run lab:js07
JS08	helper/plan foundations/work/js08.mjs và browser spec tests/work/foundation/js08_bridge.spec.ts	npm run lab:js08
TS01	foundations/work/ts01.ts và foundations/type-tests/ts01.learner-errors.ts	npm run lab:ts01
TS02	foundations/work/ts02.ts và foundations/type-tests/ts02.learner-errors.ts	npm run lab:ts02
TS03	foundations/work/ts03.ts	npm run lab:ts03
TS04	helper foundations/work/ts04.ts, type-test foundations/type-tests/ts04.learner-errors.ts và browser spec tests/work/foundation/ts04_bridge.spec.ts	npm run lab:ts04

Các file `foundations/tests/` là executable contract, không phải chỗ sửa để ép bài xanh. Riêng JS08/TS04 có hai tầng: giải helper bằng `npm run foundation:test:work -- JS08/-- TS04`, sau attempt browser mới đổi `test.fixme` thành `test` và chạy focused command. Focused command còn skip/fixme sẽ đỡ có chủ đích.

Evidence tối thiểu cho Part I

Mỗi chapter chỉ được đánh dấu pass khi có:

- file bài làm chạy được;
- prediction viết trước lần chạy đầu;
- ít nhất một failure note;
- independent task không dựa vào walkthrough;
- transfer task khác tên biến hoặc data shape của worked example;
- teach-back từ ba đến năm câu.

Không tính là đã học nếu chỉ đọc hiểu, xem đáp án, hoặc làm test xanh sau khi AI đưa full patch.

Calibration cá nhân: zero thật, đã lệ thuộc AI lâu, còn ba tuần trước SWP

Part I giả định m **chưa có nền lập trình đáng tin để dựa vào**. Không có placement skip. Tất cả QA00, JS01-JS08 và TS01-TS04 đều bắt buộc vì chúng tạo cùng một chain: chạy code → hiểu data/control flow → tách function/module → hiểu async → viết test → dùng type → đọc Playwright.

Ba tuần không đủ để “học xong JavaScript” theo nghĩa rộng. Nó đủ để tạo một runway có kiểm chứng trước khi SWP bắt đầu, nếu m dành khoảng **55-63 giờ active practice tổng**, tức một sprint khá nặng 18-21 giờ mỗi tuần:

Tuần	Chapters bắt buộc	Sản phẩm cuối tuần	Giờ mục tiêu
1	QA00, JS01-JS04	Chạy lab độc lập; viết helper và xử lý order/service data; một no-AI drill 30-45 phút	18-20

Tuần	Chapters bắt buộc	Sản phẩm cuối tuần	Giờ mục tiêu
2	JS05-JS08, retrieval JS01-04	Module + error journal; async repair; unit-test mini suite; no-AI drill 45 phút	19-22
3	TS01-TS04, Foundation capstone	Strict typecheck; typed Playwright spec; seeded-bug diagnosis; no-AI capstone 90 phút	18-21

Nếu quỹ giờ thực tế thấp hơn, m vẫn đi đúng thứ tự và mang phần chưa pass sang tuần đầu SWP; không tuyên bố runway hoàn tất giả. Nếu một gate chưa pass, m không giả vờ “xong chapter” để giữ lịch. Trong ba tuần, ưu tiên giữ đủ independent/failure/transfer và cắt bớt số biến thể phụ. Khi SWP bắt đầu, repair queue tiếp tục chạy song song; không reset và không bỏ lỗ hổng.

Anti-brainrot contract

Vì m đã dùng AI thay suy nghĩ trong thời gian dài, v4 dùng constraint mạnh hơn một khóa học bình thường:

1. **Prediction luôn no-AI.** Viết output/nhánh/type trước khi chạy và trước khi hỏi.
2. **Hai mươi phút đầu của completion là no-AI.** Được đọc error, docs trong pack và code m đã viết.
3. **Independent task và transfer là no-AI cho tới khi có một attempt chạy được hoặc một bug note đầy đủ.**
4. Không paste code do AI sinh vào lab. Nếu đã xem snippet H4/H5, đóng nó rồi tự gõ lại từ mental model.
5. Trước khi xin hint phải đưa: expected, actual, hypothesis, evidence và câu hỏi hẹp. Chỉ xin H1/H2 trước.
6. Một patch có AI hỗ trợ chỉ được giữ khi m giải thích từng decision, xóa một phần quan trọng và tự dựng lại, rồi làm variation khác không AI.
7. Không dùng câu “AI làm cho t”. Bug journal phải ghi root cause ở code/runtime/data nào.
8. Mỗi tuần có một random drill không AI, không note, không autocomplete chat. Thời gian hết thì dừng và ghi gap; không lén hoàn tất bằng AI rồi tự chấm pass.

AI vẫn hữu ích cho evidence review, đặt câu hỏi Socratic, kiểm tra reasoning và review diff sau attempt. Mục tiêu không phải cấm công cụ; mục tiêu là lấy lại khả năng tự tạo hypothesis và giữ một mental model trong đầu.

Definition of ready khi SWP bắt đầu

Sau ba tuần, m chưa cần thuộc mọi API. M cần chứng minh được:

- từ file trắng viết function xử lý data nhỏ và test nó;
- đọc code có object, array, callback, module, Promise và `await` mà không đoán từng ký hiệu;
- sửa một syntax/module/type/test failure bằng first meaningful error;
- dùng TypeScript strict để model một data shape và narrow `unknown`;
- đọc một Playwright spec và kể runner, callback, fixture, locator, action, assertion;
- làm một variation 90 phút không AI và giải thích chỗ chưa biết.

Đây là minimum entry gate để vừa học SWP theo full browser-to-database chain vừa tiếp tục Playwright, không phải giấy chứng nhận “đã giỏi JavaScript”.

QA00 - Máy của m đang chạy cái gì?

Thời lượng thường gặp: 2-3 giờ **Prerequisite:** Không có. **Exit outcome:** M có thể đứng đúng thư mục, chạy đúng lệnh, đọc error theo tầng và lưu evidence đầu tiên mà không cần hiểu JavaScript trước.

Mental model

Khi m gõ một lệnh, máy không hiểu ý định chung chung như “chạy bài học”. Nó nhận một command cụ thể trong một thư mục cụ thể.

```
Terminal
→ shell tìm command
→ Node/npm đọc file hoặc package.json
→ parser đọc syntax
→ module loader tìm import
→ chương trình/test runner chạy
→ assertion so actual với expected
```

Một lỗi ở tầng trên làm tầng dưới chưa bao giờ chạy. Nếu npm báo không tìm thấy `package.json`, việc sửa locator là vô nghĩa. Nếu parser báo thiếu dấu ngoặc, assertion chưa được đăng ký. Nếu assertion fail, điều đó lại cho biết runtime và test runner đã chạy đủ xa.

Năm loại feedback m phải tập phân biệt:

Tầng	Dấu hiệu thường gặp	Câu hỏi đầu tiên
Shell/thư mục	command not found, file not found	M đang đứng ở đâu và command có tồn tại không?
npm/package	missing script, package.json not found	Script tên gì và package hiện tại là package nào?
Syntax/module	SyntaxError, module not found, export mismatch	File có parse được và import có trở đúng không?
Type/test collection	TypeScript error, no tests found	Code có đạt contract compile và test có được đăng ký không?
Behavior/assertion	expected X, received Y	Actual khác expected ở đâu, evidence nào giải thích?

Terminal không phải nơi “dành cho hacker”. Nó là một cửa sổ đưa command cho máy và nhận feedback có thể copy lại. Ba command định vị cơ bản trên macOS/Linux:

```
pwd
ls
node --version
```

- `pwd` cho biết thư mục hiện tại.
- `ls` cho biết các entry ở đó.
- `node --version` xác nhận runtime có thể được gọi.

npm là công cụ đọc `package.json`, cài dependency và gọi các script đã được project đặt tên. `npm run lab:qa00` không phải câu thần chú của npm; nó chỉ tìm key `lab:qa00` trong phần `scripts` rồi chạy command tương ứng.

Worked example

Lab kit thật giữ worked example ở:

```
lab-kit/
├─ package.json
├─ foundations/
│  └─ worked/
│     └─ qa00.mjs
```

Một worked example tối thiểu có thể in các tầng feedback như `foundations/worked/qa00.mjs`. Đoạn rút gọn:

```
console.log('QA00 ready');
console.log('Current lesson: runtime and feedback');
```

Trong `package.json` có script:

```
{
  "scripts": {
    "lab:qa00": "node foundations/scripts/run-suite.mjs work QA00"
  }
}
```

Khi đứng trong `lab-kit` và chạy:

```
npm run lab:qa00
```

npm đọc script, gọi Node, Node đọc file và thực hiện hai lệnh `console.log` theo thứ tự từ trên xuống. Expected output phần chương trình là:

```
QA00 ready
Current lesson: runtime and feedback
```

Nếu m đứng ở thư mục cha không có `package.json`, chương trình chưa chạy. Nếu script ghi sai tên file, npm đã chạy nhưng Node không load được module. Hai lỗi có symptom khác nhau vì dừng ở hai tầng khác nhau.

Prediction - ghi trước khi chạy

Không chạy lệnh trong ba phút đầu. Viết câu trả lời:

1. Nếu đổi thứ tự hai dòng `console.log`, output thay đổi thế nào?
2. Nếu đổi tên file target thành `qa00-old.mjs` nhưng không sửa script/contract, tầng nào fail?
3. Nếu đứng ở `foundations/work` rồi chạy `npm run lab:qa00`, npm sẽ tìm `package.json` nào?
4. Nếu Node in đủ hai dòng rồi terminal vẫn báo exit code 1, “thấy output” có đủ để kết luận bài pass không?

Sau khi chạy, ghi thêm một dòng: prediction nào đúng, prediction nào khác runtime.

Completion task

Mở starter thật `foundations/work/qa00.mjs` (đây là bài receipt, không có file `starter/receipt.mjs` ẩn ở nơi khác):

```
export const receiptLines = [
  'Order code: TODO',
  'Customer: TODO',
  'Status: TODO'
];

for (const line of receiptLines) console.log(line);
```

Hoàn thành sao cho exact output là:

```
Order code: LD-001
Customer: Nguyễn An
Status: Đã tiếp nhận
```

Constraints:

- Không đổi tên file.
- Không sửa executable contract `foundations/tests/qa00.test.mjs`.
- Chạy bằng npm script, không double-click file.
- Lưu command và output vào evidence note.

Mục tiêu không phải học biến ở đây; m chỉ tập edit → run → compare → record.

Independent task

Không nhìn lại worked example trong mười phút:

1. Dùng `pwd` và `ls` để tìm lab kit.
2. Tìm script kiểm tra reference suite trong `package.json`.
3. Chạy script đó.
4. Ghi bốn thứ: command, exit status, first meaningful output, artifact hoặc folder được tạo.
5. Viết restart line đủ cụ thể để ngày mai m biết mở file nào và làm gì.

Một restart line yếu: “Mai học tiếp JS.” Một restart line tốt: “Mở `foundations/work/js01.mjs`, ghi prediction cho ba biểu thức rồi chạy `npm run lab:js01`.”

Failure injection

Lần lượt gây bốn lỗi, mỗi lần chỉ đổi một thứ:

1. Chạy script từ thư mục không chứa package hiện tại.
2. Gõ sai một ký tự trong tên npm script.
3. Xóa dấu nháy đóng của một chuỗi.
4. Đổi tên file nhưng giữ script cũ.

Với mỗi lỗi, ghi:

```
Expected:
Actual:
Failure layer:
First meaningful line:
Smallest correction:
Proof after correction:
```

Không chụp mỗi dòng cuối màu đỏ. Dòng cuối thường chỉ nói process thất bại; first meaningful line mới nói vì sao.

Transfer

Trong một repo thật m đang dùng ở công ty hoặc SWP391, chỉ làm read-only:

- xác định root dựa trên file `build/package`;
- tìm command khởi động hoặc command test đã được project định nghĩa;
- kể project dùng runtime/build tool nào;
- không tự cài hoặc nâng version nếu chưa biết contract của repo.

Nếu repo là Spring Maven, m có thể thấy `pom.xml` thay vì `package.json`. Mental model vẫn giống: đứng đúng root, gọi đúng tool, đọc lỗi theo tầng.

Gate QA00

Pass khi m làm được tất cả mà không có walkthrough:

- đứng đúng lab root và chứng minh bằng output;
- giải thích npm script đến từ đâu;
- gây và phân loại được ít nhất ba lỗi khác tầng;
- sửa đúng một biến mỗi lần và rerun;
- lưu một evidence note và restart line rõ;
- sau D+1, chạy lại completion task không nhìn hướng dẫn.

Nếu fail, repair đúng tầng bị yếu. Không cài lại toàn bộ Node chỉ vì một script bị gõ sai.

No-AI drill QA00 - 20 phút

Từ terminal mới, không mở chat: tìm lab root, chạy QA00, gây một module-not-found, sửa, rồi viết restart line. Chỉ dùng output của terminal làm evidence.

JS01 - Value, variable và biểu thức: chương trình đang giữ dữ liệu gì?

Thời lượng thường gặp: 3-4 giờ **Prerequisite:** QA00 **Exit outcome:** M phân biệt được value với variable, chọn `const/let`, dự đoán type và kết quả biểu thức cơ bản trước khi chạy.

Mental model

Value là dữ liệu runtime đang xử lý. Variable là một cái tên trỏ tới value để code có thể dùng lại.

Các primitive m gặp nhiều trong automation:

Loại	Ví dụ	Dùng cho
string	'LD-001'	text, code, URL, label
number	25000	count, timeout, số tiền lab
boolean	true	điều kiện có/không
undefined	chưa có value	property/return chưa được cung cấp
null	chủ động biểu diễn không có	API/domain contract cho phép rỗng

`const` nghĩa là binding không được trỏ sang value khác. Nó không có nghĩa mọi object phía sau đều bất biến; phần đó học ở JS04. Dùng `const` mặc định. Chỉ dùng `let` khi chính binding cần đổi trong flow.

```
const orderCode = 'LD-001';
let attempt = 1;
attempt = attempt + 1;
```

Operator tạo expression mới:

- số học: +, -, *, /;
- so sánh: ===, !==, >, <=;
- logic: &&, ||, !;
- nối chuỗi: 'Order ' + orderCode.

Ưu tiên strict equality ===. JavaScript có coercion: nó đôi khi tự đổi type để thực hiện phép toán. Coercion có thể tiện nhưng nguy hiểm khi test data từ HTML luôn bắt đầu là text.

Phép so sánh **tự tạo ra một boolean**; chưa cần `if/else` để có `true` hoặc `false`:

```
const orderTotal = 120000;
const collectedAmount = 50000;
const remainingAmount = orderTotal - collectedAmount;
const fullyPaid = remainingAmount === 0;

console.log(remainingAmount); // 70000
console.log(fullyPaid); // false
```

Đọc dòng cuối như một câu hỏi: “`remainingAmount` có bằng đúng 0 không?”. JavaScript trả lời câu hỏi đó bằng boolean. JS02 mới dùng boolean để chọn hành động bằng `if/else`.

```
console.log('2' + 3); // '23'
console.log('2' * 3); // 6
console.log('2' === 2); // false
```

Một test đáng tin không dựa vào việc nhớ ngẫu nhiên operator nào coercion theo cách nào. Khi boundary trả string, parse hoặc validate có chủ đích.

Worked example

Ta tạo summary cho một service item trong sandbox:

```
const serviceCode = 'UI_QUAN_AO';
const serviceName = 'Ủi quần áo';
const unitPrice = 12000;
const quantity = 3;
const active = true;

const lineAmount = unitPrice * quantity;
const summary = serviceCode + ' | ' + serviceName + ' | ' + lineAmount + ' VND';

console.log(summary);
console.log(typeof serviceCode);
console.log(typeof lineAmount);
console.log(active === true);
```

Đọc theo data flow:

1. Năm binding giữ input.
2. `lineAmount` phụ thuộc hai number.
3. Khi nối với string, number được biểu diễn thành text trong summary.
4. `typeof` giúp quan sát type runtime, không thay validation.
5. Biểu thức cuối tạo boolean.

Expected output:

```
UI_QUAN_AO | Ủi quần áo | 36000 VND
string
number
true
```

Trong production Laundry, server Java/`BigDecimal` mới là pricing authority. Phép tính trên chỉ là bài data-flow và test fixture, không cấp quyền đưa giá authoritative ra browser.

Prediction

Ghi exact value và type, rồi mới chạy:

```
const priceText = '25000';
const quantity = 2;

console.log(priceText + quantity);
console.log(Number(priceText) + quantity);
console.log(priceText * quantity);
console.log(priceText === 25000);
console.log(Boolean('false'));
```

Giải thích từng dòng bằng rule, không dùng câu “JavaScript kỳ vậy”. Đặc biệt: string `'false'` không phải boolean `false`.

Completion task

Hoàn thành starter, chỉ thay phần `TODO`:

Artifact: `foundations/work/js01.mjs`. Independent task tiếp tục ở

`foundations/work/js01_service_snapshot.mjs`; chạy cả hai bằng `npm run lab:js01`.

```
const orderCode = 'LD-20260813-000123';
const customerName = 'Trần Thu Hà';
const collectedAmount = 50000;
const orderTotal = 120000;

const remainingAmount = TODO;
```

```
const fullyPaid = TODO;
const label = TODO;

console.log(remainingAmount);
console.log(fullyPaid);
console.log(label);
```

Expected:

```
70000
false
LD-20260813-000123 | Trần Thu Hà | remaining 70000 VND
```

Constraints:

- Không ghi thẳng 70000 vào `remainingAmount`.
- Không ghi thẳng `false` vào `fullyPaid`.
- Label phải dùng các variable đã có.
- Không dùng `if/else`; task này chỉ tạo value mới bằng expression.
- Trong starter thật, từ khóa `export` chỉ là test glue để lab đọc value. Không xóa,

không sửa và chưa cần hiểu nó ở JS01; module được dạy ở JS05.

Independent task

Mở file **đã có sẵn** `foundations/work/js01_service_snapshot.mjs`; không tạo `service-snapshot.mjs` khác. “Snapshot” ở đây chỉ là một nhóm value ghi lại tên, đơn vị và giá tại một thời điểm. Trong JS01, m chỉ luyện data flow; chưa học object.

File gồm bảy binding riêng:

- `serviceCode`: mã dạng `string`;
- `serviceName`: tên tiếng Việt dạng `string`;
- `pricingUnit`: chọn `string` 'KG' hoặc 'ITEM';
- `unitPrice`: `number`;
- `quantity`: `number`;
- `lineAmount`: expression lấy `unitPrice * quantity`, không ghi thẳng kết quả;
- `active`: `boolean`.

Các từ khóa `export` là test glue; không sửa và chưa cần hiểu. Bài này không yêu cầu m tự viết function, array hay module.

Làm đúng thứ tự sau:

1. Chọn value cho sáu input binding và hoàn thành expression `lineAmount`.
2. Trước khi chạy, dự đoán chính xác bốn dòng: identity, unit, line amount và type của `quantity`.
3. Chạy `npm run lab:js01`. Trạng thái `number` đúng phải xanh.
4. Ghi prediction mới, rồi **tạm thời** đổi `quantity` từ `number 3` thành `string '3'`; không sửa phép nhân.
5. Chạy `node foundations/work/js01_service_snapshot.mjs`. Quan sát line amount có thể vẫn trông đúng do coercion, nhưng dòng type đã thành `string`.
6. Ghi failure note: expected, actual và vì sao output số đúng chưa chứng minh input đúng type.
7. **Đổi** `quantity` **lại thành number**, rồi chạy lại `npm run lab:js01`.

Đây mới là trạng thái cuối để chấm gate.

Failure injection

Dùng playground chính thức `foundations/work/js01_failure_playground.mjs` để không làm bản hai artifact đang được chấm. Đứng tại `lab-kit/`, ghi prediction rồi chạy:

```
node foundations/work/js01_failure_playground.mjs
```

Mỗi lần chỉ gây **một** lỗi. Sau khi ghi expected, actual và first meaningful error/output, khôi phục block đó trước khi chuyển sang block kế tiếp. JS01 chỉ quan sát value, type, binding và expression; không thêm `if`, `function`, `return` hoặc `throw`.

Failure A - nói chuỗi ngoài ý muốn

```
const quantityFromInput = '2';
const nextQuantity = quantityFromInput + 1;
console.log(nextQuantity);
```

Ghi expected trước. Sau đó conversion có chủ đích và quan sát riêng hai boolean, chưa cần `if/else` hoặc `throw`:

```
const convertedQuantity = Number(quantityFromInput);
const isFiniteNumber = Number.isFinite(convertedQuantity);
const isPositive = convertedQuantity > 0;

console.log(convertedQuantity);
console.log(isFiniteNumber);
console.log(isPositive);
```

Lặp lại với `'abc'.Number.isFinite(...)` trả `false`, nên m đã có evidence rằng conversion không tạo một number hữu hạn. Cách **chọn hành động** khi flag `false` thuộc JS02; cách ném/bắt error thuộc JS05.

Failure B - reassign const

```
const status = 'RECEIVED';
status = 'PROCESSING';
```

Đây là lỗi binding, chưa phải business rule về transition.

Failure C - typo tên variable

```
const serviceName = 'Giặt sấy quần áo';
console.log(servceName);
```

Đây là `ReferenceError`: code hỏi một binding chưa từng được khai báo. Ghi đúng tên identifier mà runtime báo và phân biệt nó với value của `serviceName`. Property typo trên object được để sang JS04; TypeScript sẽ bắt nhiều typo trước runtime từ TS01.

Transfer

Đọc một Playwright spec có các value sau và phân loại chúng mà chưa giải thích API:

```
const orderCode = 'LD-001';
const expectedStatus = 'Đã tiếp nhận';
const timeout = 5000;
const shouldBeVisible = true;
```

Trả lời:

- value nào là test input;
- value nào là expected outcome;
- value nào là execution configuration;
- vì sao không nên đổi `expectedStatus` dựa trên text thực tế vừa đọc từ trang.

Nếu expected được tính từ actual của chính UI, test có thể tự đồng ý với bug.

Gate JS01

Pass khi m:

- giải thích value khác variable thế nào;
- chọn `const` mặc định và đưa ví dụ thật cần `let`;
- dự đoán đúng ít nhất 4/5 coercion cases;
- chuyển string input sang number và quan sát validity flag thay vì tin coercion ngầm;
- hoàn thành independent task không copy worked example;
- kể một cách type sai có thể tạo false confidence trong automation.

Review D+1: `'2' + 1` và `'2' * 1`. Review D+3: `const` khóa cái gì? Review D+7: Viết lại completion task từ file trống.

No-AI drill JS01 - 25 phút

Từ file trống, tạo sáu binding cho một service line, tính một derived value và in type của ba expression. Sau đó tự cài string-number bug, conversion có chủ đích và in validity flag. Chưa dùng `if/else` hoặc `throw`.

JS02 - Condition và loop: chương trình chọn nhánh nào?

Thời lượng thường gặp: 4 giờ **Prerequisite:** JS01 **Exit outcome:** M dự đoán được nhánh `if`, dùng boolean rõ nghĩa, lặp qua một danh sách nhỏ và không viết assertion thay đổi theo actual behavior.

Mental model

Chương trình chạy tuần tự cho tới khi control flow yêu cầu chọn nhánh hoặc lặp.

```
input values
→ boolean condition
→ đúng thì chạy branch A
→ sai thì chạy branch B
```

Một condition tốt đọc như câu hỏi có đáp án yes/no:

```
const isReceived = status === 'RECEIVED';
const canStartProcessing = isReceived && accountActive;
```

`if` không “biết” business rule. Nó chỉ đánh giá expression. Chính code của m phải biểu diễn rule đúng.

```
if (canStartProcessing) {
  console.log('Allowed');
} else {
  console.log('Rejected');
}
```

JavaScript có khái niệm truthy/falsy. Các value như `false`, `0`, chuỗi rỗng, `null`, `undefined` là falsy. Nhiều value khác là truthy, kể cả chuỗi `'false'` và `'0'`. Với business decision, ưu tiên comparison rõ thay vì dựa vào truthiness mơ hồ.

Loop dùng khi cùng một operation áp dụng cho nhiều item. Ở chapter này chỉ dùng `for...of`; array methods sâu hơn nằm ở JS04.

```
const statuses = ['RECEIVED', 'PROCESSING', 'READY_FOR_PICKUP'];

for (const currentStatus of statuses) {
  console.log(currentStatus);
}
```

Mỗi vòng, `currentStatus` giữ một value từ danh sách. Loop không tự tạo assertion hoặc oracle; m vẫn phải quyết định outcome nào đúng.

Worked example

Ta kiểm tra một command “start processing” ở mức logic sandbox:

```
const role = 'STAFF';
const accountActive = true;
const orderStatus = 'RECEIVED';

const roleAllowed = role === 'STAFF' || role === 'MANAGER';
const statusAllowed = orderStatus === 'RECEIVED';
const commandAllowed = accountActive && roleAllowed && statusAllowed;

if (commandAllowed) {
  console.log('Start processing is allowed');
} else {
  console.log('Start processing is rejected');
}
```

Điểm quan trọng:

- OR dùng cho hai role đều hợp lệ.
- AND dùng vì cả ba guard phải pass.
- Ta đặt tên cho từng boolean để failure dễ giải thích.
- Đây là model học tập; production authority vẫn ở Spring service sau authentication/authorization.

Nếu đổi status thành `PROCESSING`, `statusAllowed` false và command bị reject. Không cần đợi UI ẩn button mới biết rule; UI hiding không thay server guard.

Prediction

Không chạy, điền bảng output:

```
const cases = [
  true,
  false,
  0,
  1,
  '',
  'false',
  null,
  undefined
];

for (const value of cases) {
  if (value) {
    console.log('T');
  } else {
    console.log('F');
  }
}
```

Sau đó trả lời: tại sao form value 'false' không nên được dùng trực tiếp như boolean?

Prediction thứ hai:

```
const status = 'READY_FOR_PICKUP';
const remainingAmount = 0;
const hasOpenIssue = false;

const ready = status === 'READY_FOR_PICKUP';
const paid = remainingAmount === 0;
const clear = hasOpenIssue === false;

console.log(ready && paid && clear);
console.log(ready && paid || clear);
```

Hai expression khác nhau về grouping và business meaning. Viết thêm ngoặc để ý định rõ.

Completion task

Hoàn thành role matrix:

Artifact: `foundations/work/js02.mjs`; command: `npm run lab:js02`.

File này cố ý là một script chạy từ trên xuống. M chưa cần `function`, `export`, `return` hay `throw`; các khái niệm đó lần lượt đến ở JS03 và JS05. Lab chấm các dòng được in ra khi chạy file.

```
const roles = ['ANONYMOUS', 'STAFF', 'MANAGER'];

for (const role of roles) {
  const authenticated = TODO;
  const managerRouteAllowed = TODO;

  if (managerRouteAllowed) {
    console.log(role + ': 200');
  } else if (authenticated) {
```

```

    console.log(role + ': 403');
  } else {
    console.log(role + ': 302 login');
  }
}

```

Expected:

```

ANONYMOUS: 302 login
STAFF: 403
MANAGER: 200

```

Không hard-code output theo từng index. Condition phải dựa trên `role`.

Independent task

Ngay dưới role matrix trong cùng starter có danh sách status:

```

const statuses = [
  'RECEIVED',
  'PROCESSING',
  'READY_FOR_PICKUP',
  'COMPLETED',
  'CANCELLED',
  'UNKNOWN'
];

```

Viết script in đúng một action khả dụng cho mỗi status theo canonical forward lifecycle:

- RECEIVED: start hoặc cancel;
- PROCESSING: mark ready;
- READY_FOR_PICKUP: complete nếu các guard khác pass;
- COMPLETED/CANCELLED: terminal.

Viết một chuỗi `if / else if / else`, gán action rồi in đúng một dòng cho mỗi status. Case `UNKNOWN` đã được đặt sẵn để ép nhánh cuối in `ERROR unknown status`, thay vì coi unknown là terminal hợp lệ. Không cần tự nghĩ cách chia function ở bài này.

Failure injection

Failure A - assignment thay comparison

```

let status = 'PROCESSING';

if (status = 'RECEIVED') {
  console.log('Start allowed');
}

console.log(status);

```

Dự đoán cả hai output. Root cause không phải “if bị sai”; nó là expression đã thay đổi binding và trả một string truthy.

Failure B - assertion thích nghi với bug

```

const actualStatus = 'Đã hủy';
let expectedStatus;

if (actualStatus === 'Đã hủy') {
  expectedStatus = 'Đã hủy';
} else {
  expectedStatus = 'Đã tiếp nhận';
}

console.log(actualStatus === expectedStatus);

```


Đoạn này luôn dễ pass theo actual. Hãy viết lại để expected xuất phát từ scenario/business rule, không xuất phát từ UI vừa quan sát.

Failure C - loop che case thiếu

Xóa `CANCELLED` khỏi test data rồi thấy toàn bộ case còn lại pass. Giải thích vì sao loop chạy xanh không chứng minh danh sách case đầy đủ. Coverage là quyết định của tester, không phải property tự nhiên của loop.

Transfer

Đọc requirement: “MANAGER thừa hưởng thao tác STAFF, nhưng chỉ MANAGER được refund.” Tạo decision table bằng text trước khi code:

Role	Collect	Refund
Anonymous	?	?
Staff	?	?
Manager	?	?

Sau đó viết condition cho từng cột và một loop in matrix. Transfer tiếp theo: chuyển matrix đó thành tên ba Playwright tests, chưa cần viết locator.

Gate JS02

Pass khi m:

- dự đoán đúng truthy/falsy cases quan trọng;
- dùng comparison rõ nghĩa cho role/status;
- giải thích AND khác OR trong một business guard;
- viết `for...of` qua toàn bộ list;
- phát hiện assignment-inside-condition;
- chỉ ra vì sao expected không được suy ra từ actual;
- tạo decision table rồi mới code independent task.

Review D+1: chuỗi `'false'` truthy hay falsy? Review D+3: Viết role matrix từ file trắng. Review D+7: Nhận một status mới và chỉ ra nơi cần thay đổi/test.

No-AI drill JS02 - 30 phút

Nhận ngẫu nhiên một role và một order status. Tạo decision table nhỏ, code outcome, thêm một unknown case và giải thích vì sao expected không được lấy từ actual UI.

JS03 - Function: đặt tên cho một việc và kiểm soát input/output

Thời lượng thường gặp: 4 giờ **Prerequisite:** JS01-JS02 **Exit outcome:** M tự viết function có parameter, return và guard; phân biệt định nghĩa với gọi; đọc callback ở mức cơ bản.

Mental model

Function là một value chứa một quy trình có thể gọi lại. Một function hữu ích có contract:

```
input qua parameters
→ validate/transform/decide
→ output qua return hoặc throw
```

```
function add(a, b) {
  return a + b;
}
```

```
const total = add(2, 3);
```

- `function add...` định nghĩa function.
- `a, b` là parameter, tên dùng trong definition.
- `2, 3` là argument ở call site.
- `return` đưa value về caller và dừng function.
- `total` nhận output.

`console.log` không thay `return`. In một value cho con người thấy khác với đưa value cho code gọi tiếp.

Scope là vùng một name tồn tại. Parameter và variable khai báo trong function không tự tồn tại bên ngoài. Function nên lấy dependency qua parameter thay vì âm thầm đọc và sửa global state; điều này làm test dễ dự đoán hơn.

Function cũng có thể được truyền như value. Khi một function khác nhận nó và gọi sau, nó là callback:

```
function announce(value) {
  console.log(value);
}
```

```
function runJob(job) {
  job('done');
}
```

```
runJob(announce);
```

Ai gọi callback? Trong ví dụ, `runJob`. Trong Playwright, test runner gọi callback m truyền vào `test(...)` khi fixture sẵn sàng.

Arrow function là cú pháp ngắn thường thấy:

```
const double = value => value * 2;
```

Đừng học arrow như một loại phép thuật khác. Trước hết đọc nó là “function nhận value, trả value nhân hai”.

Trong file lab, một số definition bắt đầu bằng `export`. Ở JS03, chỉ cần đọc `export function name(...)` như `function name(...)`; từ khóa đó là móc để test runner gọi function của m. JS05 mới giải thích module/import/export đầy đủ. Đừng xóa `export` khỏi starter.

Worked example

Phone normalization là ví dụ tốt vì có input, transform và invalid boundary rõ:

```
function normalizeStoredPhone(rawPhone) {
```

```
const trimmed = rawPhone.trim();
const withoutSeparators = trimmed.replaceAll(/[\s-]/g, '');
const local = withoutSeparators.startsWith('+84')
  ? '0' + withoutSeparators.slice(3)
  : withoutSeparators;

const valid = /^0\d{9}$/.test(local);

if (!valid) {
  throw new Error('Phone must contain 10 digits and start with 0');
}

return local;
}

const normalized = normalizeStoredPhone(' +84 912-345-678 ');
console.log(normalized);
```

Data flow:

1. Caller đưa raw string.
2. Function tạo intermediate values, mỗi name giải thích một transform.
3. Ternary chỉ chọn conversion cho prefix +84.
4. Guard throw nếu contract không đạt.
5. Chỉ valid value mới return.

Regex có thể chưa quen; ở chapter này m không cần tự thiết kế regex. Hãy đọc nó như một dependency đang kiểm tra “0 rồi chín digit”. Điều quan trọng là contract và flow.

Prediction

Không chạy:

```
function describe(status) {
  if (status === 'RECEIVED') {
    return 'Đã tiếp nhận';
  }

  console.log('unknown status');
}

const first = describe('RECEIVED');
const second = describe('PROCESSING');
console.log(first);
console.log(second);
```

Ghi exact output và value của `second`. Tại sao log “unknown” không phải return?

Prediction callback:

```
function execute(task) {
  console.log('before');
  const result = task(4);
  console.log(result);
  console.log('after');
}

execute(value => value * 3);
```

Ai gọi arrow callback? Argument 4 đến từ đâu? Callback return gì?

Completion task

Hoàn thành ba function:

Artifact: foundations/work/js03.mjs; cùng file chứa khung independent canCompletePickup; command: npm run lab:js03.

```
function lineAmount(unitPrice, quantity) {
  // TODO: reject non-positive inputs, then return multiplication
}

function remainingAmount(orderTotal, collectedAmount, refundedAmount) {
  // TODO: net paid = collected - refunded
  // TODO: remaining cannot be below zero
}

function statusLabel(status) {
  // TODO: support all five canonical order statuses
  // TODO: throw for unknown value
}
```

Starter tests sẽ gọi function bằng nhiều argument. Không đọc expected file rồi hard-code đúng data mẫu. Với mỗi function, viết contract một câu trước code.

Independent task

Viết từ file trống:

```
canCompletePickup(status, remainingAmount, hasOpenIssue) → boolean
```

Canonical outcome: chỉ true khi status là `READY_FOR_PICKUP`, remaining bằng 0 và không có open issue.

Bắt buộc:

- không đọc global variable;
- không mutate argument;
- return boolean ở mọi path;
- tạo tối thiểu năm calls, mỗi call đối đúng một guard;
- giải thích case nào là happy path và case nào chứng minh từng guard.

Sau đó đổi tên các local variable nhưng giữ contract. Nếu logic vỡ chỉ vì đổi tên, m đang dựa vào copy/pattern chứ chưa hiểu flow.

Failure injection

Failure A - missing return

```
function add(a, b) {
  const result = a + b;
}

console.log(add(2, 3));
```

Failure B - return quá sớm

```
function requirePositive(value) {
  return value;

  if (value <= 0) {
    throw new Error('positive required');
  }
}
```

Failure C - shadowing

```
const status = 'RECEIVED';

function printStatus(status) {
  const statusLabel = status === 'RECEIVED' ? 'Đã tiếp nhận' : 'Khác';
  console.log(statusLabel);
}

printStatus('PROCESSING');
console.log(status);
```

Ghi value nào thuộc scope nào. Shadowing không luôn là bug, nhưng name trùng làm beginner dễ kể sai state.

Failure D - gọi thay vì truyền callback

```
function greet() {
  console.log('hello');
}

function register(callback) {
  console.log('registered');
  callback();
}

register(greet());
```

So sánh `greet` và `greet()`. Một cái là function value, một cái là kết quả của call.

Transfer

Đọc anatomy:

```
test('visitor sees services', async ({ page }) => {
  await page.goto('/services');
});
```

Trả lời không cần biết Playwright API sâu:

- `test` là function hay keyword?
- Có bao nhiêu argument được truyền vào `test`?
- Argument thứ hai là function definition hay function call?
- Ai sẽ gọi callback?
- `{ page }` là input runner cung cấp, không phải page m tự tạo trong callback; tại sao?

Gate JS03

Pass khi m:

- viết function từ contract thay vì từ sample output;
- phân biệt parameter/argument và definition/call;
- dùng return ở mọi path phù hợp;
- giải thích scope cho hai name trùng;
- phát hiện callback được gọi sớm;
- hoàn thành `canCompletePickup` và variation không AI.

No-AI drill JS03 - 30 phút

Từ file trống, viết `canCollect(status, amount, remaining)`, năm examples và một callback runner. Gây missing-return rồi dùng output/test để tìm, không hỏi AI.

JS04 - Array và object: model data, không bám index

Thời lượng thường gặp: 5 giờ **Prerequisite:** JS01-JS03 **Exit outcome:** M đọc/ghi object property, lập và transform array, hiểu reference/shallow copy, chọn item bằng business identity thay vì vị trí.

Mental model

Object gom các field có tên thành một data shape:

```
const order = {
  code: 'LD-001',
  status: 'RECEIVED',
  total: 100000
};
```

Đọc property bằng dot hoặc bracket:

```
console.log(order.code);
console.log(order['status']);
```

Array là sequence có thứ tự:

```
const services = ['Giặt sấy', 'Ủi quần áo'];
console.log(services[0]);
```

Index bắt đầu từ 0 nhưng index hiếm khi là business identity. “Row thứ hai” có thể đổi khi sort/filter; “order có code LD-002” giữ ý nghĩa.

Các operation cần cho automation:

- `find`: lấy item đầu tiên thỏa condition;
- `filter`: lấy danh sách item thỏa condition;
- `map`: đổi mỗi item thành representation khác;
- `some`: có ít nhất một item thỏa condition;
- `every`: mọi item đều thỏa;
- `reduce`: tích lũy thành một result.

Chúng nhận callback. Callback được array method gọi cho từng item.

Object và array là reference values. Hai variable có thể trỏ cùng object:

```
const original = { status: 'RECEIVED' };
const alias = original;
alias.status = 'PROCESSING';
console.log(original.status); // PROCESSING
```

Spread tạo shallow copy:

```
const copy = { ...original };
```

Nhưng nested object/array bên trong vẫn có thể dùng chung reference. “Có spread” không đồng nghĩa deep immutable.

Destructuring lấy field ra bằng name:

```
const { code, status } = order;
```

Worked example

```
const transactions = [
  { code: 'TX-001', type: 'COLLECTION', amount: 70000 },
  { code: 'TX-002', type: 'COLLECTION', amount: 50000 },
  { code: 'TX-003', type: 'REFUND', amount: 20000 }
];

const collections = transactions.filter(tx => tx.type === 'COLLECTION');
const hasRefund = transactions.some(tx => tx.type === 'REFUND');
const transactionCodes = transactions.map(tx => tx.code);

const collectedAmount = collections.reduce(
  (sum, tx) => sum + tx.amount,
  0
);

const refundedAmount = transactions
  .filter(tx => tx.type === 'REFUND')
  .reduce((sum, tx) => sum + tx.amount, 0);

const netPaid = collectedAmount - refundedAmount;

console.log(transactionCodes);
console.log(hasRefund);
console.log(netPaid);
```

Đọc callback `tx => tx.type === 'COLLECTION'` là: array method đưa từng transaction vào parameter `tx`; callback trả boolean để quyết định giữ item.

Với `reduce`, accumulator `sum` bắt đầu từ 0. Mỗi vòng trả accumulator mới. Nếu quên `return` trong block callback, accumulator trở thành `undefined`.

Production Laundry derive money bằng ledger ở server/DB theo canonical rule. Lab này chỉ rèn data manipulation và oracle reasoning.

Prediction

Reference

```
const order = {
  code: 'LD-001',
  customer: { name: 'Nguyễn An' },
  items: [{ service: 'Giặt sấy', quantity: 2 }]
};

const alias = order;
const shallow = { ...order };

alias.code = 'LD-CHANGED';
shallow.customer.name = 'Trần Bình';
shallow.items.push({ service: 'Ủi', quantity: 1 });

console.log(order.code);
console.log(order.customer.name);
console.log(order.items.length);
```

Ghi exact output và vẽ ba arrow reference. Vì sao `shallow.code = ...` sẽ khác `shallow.customer.name = ...`?

Array methods

```
const values = [1, 2, 3, 4];
console.log(values.filter(value => value % 2 === 0));
console.log(values.map(value => value * 10));
console.log(values.find(value => value > 2));
console.log(values.some(value => value === 5));
```

Completion task

Cho data:

Artifact: foundations/work/js04.mjs; command: npm run lab:js04.

```
const orders = [
  { code: 'LD-001', status: 'RECEIVED', total: 75000 },
  { code: 'LD-002', status: 'PROCESSING', total: 120000 },
  { code: 'LD-003', status: 'READY_FOR_PICKUP', total: 80000 },
  { code: 'LD-004', status: 'RECEIVED', total: 50000 }
];
```

Hoàn thành:

```
const target = orders.find(TODO);
const receivedOrders = orders.filter(TODO);
const codes = orders.map(TODO);
const totalValue = orders.reduce(TODO, 0);
const allHaveCodes = orders.every(TODO);
```

Expected contracts:

- target có code LD-003;
- receivedOrders có hai item;
- codes giữ đúng order hiện tại;
- totalValue là tổng bốn total;
- allHaveCodes true khi mọi code không rỗng.

Không dùng `orders[2]` để tìm LD-003.

Independent task

Tạo model:

```
order
  code
  status
  customer snapshot: name, phone
  items[]: serviceCode, serviceName, unit, unitPrice, quantity
  transactions[]: code, type, amount
```

Viết các function:

- `findOrderByCode(orders, code);`
- `activeOrderCodes(orders)` loại terminal statuses;
- `hasOpenIssue(order)` dựa trên issues m tự thêm;
- `deriveLedger(transactions)` trả object gồm collected/refunded/netPaid;
- `snapshotSummary(order)` trả một object mới, không mutate input.

Starter tests sẽ reorder array. Code dùng business identity phải vẫn pass.

Failure injection

Failure A - index coupling

```
const detailOrder = orders[1];
```

Reverse array rồi chứng minh test chọn nhầm. Sửa bằng `find` và handle explicit khi không tìm thấy.

Failure B - mutate fixture dùng chung

```
const baseOrder = { code: 'LD-001', items: [] };

function addItem(order, item) {
  order.items.push(item);
  return order;
}
```

Gọi hai scenario liên tiếp trên `baseOrder`. Scenario sau thấy state của scenario trước. Sửa bằng factory hoặc copy đúng depth; giải thích ai sở hữu data.

Failure C - missing return trong reduce

```
const total = [10, 20, 30].reduce((sum, value) => {
  sum + value;
}, 0);
```

Failure D - typo property

`order.costumer.name` trả gì hoặc throw ở đâu tùy shape? Ghi first meaningful line. TS01 sẽ chuyển nhiều typo như vậy thành compile-time feedback.

Transfer

Từ data sang locator reasoning:

```
const row = page.getByRole('row').filter({ hasText: orderCode });
```

Giải thích sự tương đồng:

- `array.find(item => item.code === code)` chọn bằng business identity;
- locator scope row bằng order code cũng chọn business object;
- `nth(1)` giống index coupling và có thể vỡ khi DOM reorder.

Sau đó thiết kế test data cho hai orders cùng action text “Chi tiết” nhưng khác code. Chưa cần biết toàn bộ locator API; chỉ cần nêu selection contract.

Gate JS04

Pass khi m:

- model được nested object/array từ requirement;
- dùng `find/filter/map/some/every/reduce` đúng purpose;
- giải thích callback do ai gọi;
- dự đoán alias và shallow-copy mutation;
- sửa test bám index thành business identity;
- viết data factory hoặc copy strategy tránh shared mutation;
- làm independent task và reorder variation không AI.

No-AI drill JS04 - 45 phút

Nhận một file JSON-like gồm orders và transactions. Viết ba query và một ledger derivation, sau đó cài reorder + shared-mutation bug và ghi root cause.

JS05 - Module, package, JSON và stack trace: code được ghép lại thế nào?

Thời lượng thường gặp: 4 giờ **Prerequisite:** QA00, JS01-JS04 **Exit outcome:** M tách/nhập module có chủ đích, đọc JSON, throw/catch ở boundary phù hợp và tìm frame đầu tiên thuộc code của mình.

Mental model

Khi mọi helper nằm trong một file, dependency bị che. Module tạo boundary rõ:

```
module A export contract
→ module B import đúng name/path
→ module loader nối dependency
→ caller dùng value/function đã import
```

ES module thường dùng named export:

```
// phone.mjs
export function normalizePhone(raw) {
  return raw.trim();
}

// app.mjs
import { normalizePhone } from './phone.mjs';
```

Ba thứ phải đồng ý: file path, export name và module system. Một import error xảy ra trước khi test behavior chạy.

`package.json` mô tả package, scripts và dependency. `package-lock.json` khóa dependency tree để install lặp lại hơn. Không sửa lockfile thủ công để “cho hết đồ”.

JSON là data format, không phải JavaScript đầy đủ:

```
{
  "code": "LD-001",
  "active": true
}
```

JSON yêu cầu double quotes cho key/string, không có comment, function hay `undefined`. `JSON.parse` đổi text thành runtime value; parsed value vẫn cần validation nếu đến từ bên ngoài.

Error là một value mô tả failure. `throw` dừng normal flow và đưa error lên caller. `catch` chỉ nên xử lý khi layer đó có thể thêm meaning, recover hoặc translate. Catch rồi im lặng làm mất evidence.

Stack trace là dấu vết các call frames. Khi dependency có nhiều frame, bắt đầu ở message và frame đầu tiên thuộc file của m, không sửa frame sâu trong thư viện theo cảm giác.

Worked example

`src/status-labels.mjs`:

```
const labels = {
  RECEIVED: 'Đã tiếp nhận',
  PROCESSING: 'Đang xử lý',
  READY_FOR_PICKUP: 'Sẵn sàng trả',
  COMPLETED: 'Đã hoàn tất',
  CANCELLED: 'Đã hủy'
};

export function statusLabel(status) {
  const label = labels[status];

  if (label === undefined) {
    throw new Error('Unknown order status: ' + status);
  }
}
```

```
    return label;
  }

src/order-summary.mjs:

import { statusLabel } from './status-labels.mjs';

export function orderSummary(order) {
  return order.code + ' | ' + statusLabel(order.status);
}

run.mjs:

import { readFileSync } from 'node:fs';
import { orderSummary } from './src/order-summary.mjs';

const raw = readFileSync(new URL('./data/order.json', import.meta.url), 'utf8');
const order = JSON.parse(raw);
console.log(orderSummary(order));
```

Dependency chain là `run` → `JSON/file API` + `order-summary` → `status-labels`. Nếu `status unknown`, stack cho biết call site qua chain. Nếu `JSON syntax sai`, `orderSummary` chưa được gọi. JS05 cố ý dùng API đọc file đồng bộ để dependency flow vẫn tuần tự; `Promise` và `await` chỉ bắt đầu ở JS06.

Prediction

Cho module:

```
// math.mjs
export function add(a, b) {
  return a + b;
}
```

Với từng import, dự đoán parse/load/run:

```
import { add } from './math.mjs';
import { sum } from './math.mjs';
import add from './math.mjs';
import { add } from './math';
```

Không cần nhớ wording error tuyệt đối, nhưng phải phân loại được `name mismatch`, `default/named mismatch` và `path mismatch`.

Prediction JSON:

```
const raw = '{"amount":"25000","active":true}';
const value = JSON.parse(raw);
console.log(typeof value.amount);
console.log(typeof value.active);
```

Completion task

Starter có một file lớn chứa:

Monolith thật ở `foundations/work/js05-monolith.mjs`; chạy baseline bằng `npm run lab:js05:monolith`. Target main là `foundations/work/js05.mjs`. Ba module starter `js05-phone.mjs`, `js05-status.mjs`, `js05-orders.mjs` nằm cạnh nó; fixture ở `foundations/fixtures/js05-orders.json`. Chạy `npm run lab:js05`.

- `normalizePhone`;
- `statusLabel`;
- `findOrderByCode`;
- main code đọc `orders.json`.

Tách thành ba modules theo contract, rồi main import chúng. Bắt buộc:

- named exports;
- relative imports có extension;
- không duplicate helper để tránh import;
- khi không tìm thấy order, throw error có code được tìm;
- npm script chạy file main.
- dùng `readFileSync` trong bài này; chưa tự đổi sang API Promise trước JS06.

Independent task

Tạo package mini `laundry-data-tools`:

Starter thật nằm ở `labs/js05/laundry-data-tools/starter/`; chạy bằng `npm run lab:js05:independent`. H5 nằm ở folder `reference/` cạnh starter, không mở trước timebox.

```
src/  
  phone.mjs  
  ledger.mjs  
  orders.mjs  
data/  
  scenario.json  
run.mjs
```

`run.mjs` phải đọc scenario, normalize phone, tìm order, derive ledger và in summary. Tự thiết kế exports. Sau khi chạy xanh, vẽ dependency graph và trả lời module nào không nên biết filesystem. Starter đã dùng `readFileSync`; giữ flow đồng bộ để bài đọc lập với JS06.

Failure injection

Gây lộn lạo:

1. Named export/import mismatch.
2. Relative path sai một cấp.
3. JSON trailing comma hoặc single quote.
4. Property bắt buộc bị thiếu nhưng parse vẫn thành công.
5. Catch rồi chỉ log `'failed'` và tiếp tục exit 0.

Với lỗi 4, phân biệt parse success với domain validation success. Với lỗi 5, sửa sao cho automation/CI có thể nhận biết failure.

Stack-trace drill: tạo chain `run` → `loadScenario` → `validateOrder` → `requireCode`, cài missing code và khoanh:

- error message;
- first frame thuộc code m;
- call path;
- nơi hợp lý nhất để sửa;
- regression test cần thêm.

Transfer

Đọc một Playwright import:

```
import { test, expect } from '@playwright/test';  
import { loginAs } from './support/auth';
```

Phân biệt package import và relative import. Tìm module boundary hợp lý cho:

- static test data;

- reusable pure calculation;
- browser action có assertion;
- environment secret.

Không tách helper chỉ vì “file dài hai dòng”. Boundary phải làm dependency/contract rõ hơn hoặc loại duplication có thật.

Gate JS05

Pass khi m:

- tách và import ba modules không walkthrough;
- giải thích named/default mismatch;
- phân biệt JSON parse với validation;
- không nuốt error làm process pass giả;
- đọc stack từ message tới first own frame;
- hoàn thành dependency graph và một import-error variation.

No-AI drill JS05 - 35 phút

Từ starter sai, sửa module-not-found, export mismatch và invalid JSON. Mỗi lần ghi hypothesis trước sửa. Kết thúc bằng một npm script exit 0 thật sự.

JS06 - Promise và async/await: công việc hoàn tất ở tương lai

Thời lượng thường gặp: 6 giờ **Prerequisite:** JS01-JS05 **Exit outcome:** M đọc được Promise lifecycle, dùng `await` theo dependency, propagate/catch error có chủ đích và phân biệt sequential với concurrent work.

Mental model

Một synchronous call trả kết quả trước khi caller đi tiếp. Một asynchronous operation thường trả ngay một Promise - object đại diện cho kết quả tương lai.

```
Promise
├── pending
├── fulfilled(value)
└── rejected(error)
```

Function khai báo `async` luôn trả Promise. `return value` trong `async function` trở thành fulfilled Promise; `throw error` trở thành rejected Promise.

```
async function loadOrder() {
  return { code: 'LD-001' };
}

const promise = loadOrder();
const order = await promise;
```

`await` tạm dừng continuation của `async function` hiện tại cho tới khi Promise settle. Nó không block toàn bộ JavaScript runtime.

Dependency quyết định thứ tự:

```
Nếu B cần output A: await A rồi B.
Nếu A và B độc lập: có thể start cùng lúc rồi await Promise.all.
```

`Promise.all` không phải “tối ưu mặc định”. Hai browser actions trên cùng page thường có dependency/state và không nên song song chỉ vì cú pháp cho phép.

Event loop tối thiểu cần nhớ:

- current synchronous stack chạy hết;
- Promise continuations/microtasks đã queue chạy;
- timer/task tiếp theo mới chạy.

Mục tiêu là dự đoán timeline, không thuộc khẩu hiệu “Promise nhanh hơn timer”.

Worked example

```
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

async function loadCatalog() {
  await delay(30);
  return ['Giặt sấy', 'Ủi quần áo'];
}

async function loadStoreSettings() {
  await delay(20);
  return { shopName: 'Laundry Demo' };
}

async function buildPageModel() {
  const catalogPromise = loadCatalog();
  const settingsPromise = loadStoreSettings();

  const [catalog, settings] = await Promise.all([
    catalogPromise,
    settingsPromise
  ]);

  return { catalog, settings };
}

const model = await buildPageModel();
console.log(model.settings.shopName);
console.log(model.catalog.length);
```

Hai loaders độc lập, nên start cả hai trước rồi await chung là hợp lệ. Nếu settings cần một ID từ catalog, chúng không còn độc lập.

Error propagation:

```
async function requireOrder(code) {
  await delay(5);
  throw new Error('Order not found: ' + code);
}

try {
  await requireOrder('LD-X');
} catch (error) {
  console.error(error.message);
}
```

Catch chỉ vì entry point muốn chuyển error thành user-facing/evidence output. Helper sâu hơn có thể để rejection propagate.

Prediction

Timeline A

```
console.log('A');
setTimeout(() => console.log('T'), 0);
Promise.resolve().then(() => console.log('P'));
console.log('B');
```

Ghi exact order và vẽ stack → microtask → timer.

Timeline B

```
async function f() {
  console.log('F1');
  await 0;
  console.log('F2');
}

console.log('A');
f();
console.log('B');
```

`f()` chạy sync tới đâu trước khi caller tiếp tục?

Missing await

```
async function valueLater() {
  return 42;
}

const value = valueLater();
console.log(value === 42);
```

`value` là 42 hay Promise? Dùng evidence gì để chứng minh?

Sau khi đã khóa cả ba prediction trên giấy, chạy sandbox `npm run lab:js06:predict` (artifact `labs/js06/predict.mjs`) để đối chiếu. Command này thuộc JS06; không chạy nó khi đang học JS02.

Completion task

Starter có ba functions:

Artifact: `foundations/work/js06.mjs`; command: `npm run lab:js06`.

```
async function fetchOrder(code) { /* provided */ }
async function fetchTransactions(code) { /* provided */ }
async function fetchCustomer(customerId) { /* provided */ }
```

Hoàn thành `buildOrderDetail`:

1. Fetch order trước vì cần `customerId`.
2. Sau khi có order, transactions theo code và customer theo ID là độc lập; start cùng lúc.
3. Return object tổng hợp.
4. Nếu order không tồn tại, error phải propagate với code.

Trước khi code, vẽ dependency graph. Starter test đo call order và result, không chỉ thời gian tổng.

Independent task

Viết một command simulation:

```
recordCollection(orderCode, amount)
→ load order
→ derive remaining from transactions
→ reject invalid amount/status
→ save transaction
→ return receipt
```

Các provided async dependency cố tình delay/reject. M phải:

- await theo dependency;
- không catch rồi giả success;
- tạo happy path và ba rejection cases;
- ghi timeline cho một case;
- giải thích operation nào tuyệt đối không chạy nếu guard fail.

Đây là sandbox reasoning, không phải implementation transaction của Spring.

Failure injection

Failure A - quên await assertion-like Promise

```
async function assertEventually() {
  await delay(10);
  throw new Error('Expected ready, received processing');
}

async function testCase() {
  assertEventually();
  console.log('test ended');
}

await testCase();
```

Quan sát false completion/unhandled rejection. Sửa ở caller và chứng minh process exit khác.

Failure B - helper không return Promise

```
function waitForSave() {
  delay(20).then(() => console.log('saved'));
}

await waitForSave();
console.log('after wait');
```

Failure C - Promise.all sai dependency

Start `fetchCustomer(order.customerId)` trước khi có `order`. Root cause là dependency graph sai, không phải `Promise.all` “bị flaky”.

Failure D - swallowed error

```
async function login() {
  try {
    await Promise.reject(new Error('401'));
  } catch (error) {
    console.log('login issue');
  }
}
```

Caller nhận fulfilled Promise `undefined`. Quyết định: recover thật, translate rồi throw, hay để propagate.

Transfer

Phân loại từng Playwright call theo dependency:

```
await page.goto('/orders');
await page.getByRole('link', { name: 'Chi tiết' }).click();
await expect(page.getByRole('heading')).toContainText('LD-001');
```

- Click phụ thuộc navigation/DOM nào?
- Assertion phụ thuộc outcome nào?
- Vì sao chạy ba Promise trong `Promise.all` là sai mental model?
- Bỏ await ở action và assertion có thể cho symptom khác nhau thế nào?

Gate JS06

Pass khi m:

- vẽ Promise pending/fulfilled/rejected;
- dự đoán đúng hai event-loop snippets;

- nhận ra async function luôn trả Promise;
- sắp xếp task dựa trên dependency graph;
- dùng Promise.all chỉ cho work độc lập;
- sửa missing-return/missing-await/swallowed-error bằng evidence;
- làm independent command và one variation không AI.

No-AI drill JS06 - 45 phút

Nhận ba async dependencies chưa biết delay. Vẽ graph, implement aggregator, cài một missing await và một rejection, rồi ghi actual timeline/root cause.

JS07 - Test tự động cơ bản: arrangement, action và oracle

Thời lượng thường gặp: 4-5 giờ **Prerequisite:** JS01-JS06 **Exit outcome:** M viết test nhỏ bằng Node test runner, chọn oracle độc lập với implementation, đọc diff và tránh test chỉ chứng minh code của test.

Mental model

Automated test là một chương trình chạy code khác và quyết định pass/fail. Cấu trúc tối thiểu:

```
Arrange: tạo input/precondition
Act: gọi behavior
Assert: so actual với expected oracle
```

Test runner đăng ký test callbacks, gọi từng callback, thu Promise/error và báo kết quả. Assertion throw khi contract không đạt. Test xanh chỉ có ý nghĩa bằng contract nó kiểm tra; một assertion yếu cho confidence yếu.

```
import test from 'node:test';
import assert from 'node:assert/strict';

test('adds two values', () => {
  const actual = 2 + 3;
  assert.equal(actual, 5);
});
```

Tên test nên mô tả observable behavior, không mô tả method được gọi. “rejects collection above remaining amount” hữu ích hơn “test recordCollection”.

Oracle là nguồn xác định expected. Nó có thể đến từ requirement, calculation độc lập, known seed hoặc protocol contract. Nếu expected gọi lại chính function đang test, bug có thể xuất hiện ở cả actual và expected.

Một case mạnh thường đối đúng một dimension để biết guard nào được chứng minh. Hai mươi cases copy/paste mà không có reason không tự động tốt hơn năm cases có boundary rõ.

Worked example

Production function:

```
export function canCompletePickup(status, remaining, hasOpenIssue) {
  return status === 'READY_FOR_PICKUP'
    && remaining === 0
    && hasOpenIssue === false;
}
```

Tests:

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { canCompletePickup } from '../src/pickup.mjs';

test('allows pickup when ready, fully paid and issue-free', () => {
  const actual = canCompletePickup('READY_FOR_PICKUP', 0, false);
  assert.equal(actual, true);
});

test('rejects pickup with remaining balance', () => {
  const actual = canCompletePickup('READY_FOR_PICKUP', 1, false);
  assert.equal(actual, false);
});

test('rejects pickup with an open issue', () => {
  const actual = canCompletePickup('READY_FOR_PICKUP', 0, true);
  assert.equal(actual, false);
});
```

```
test('rejects pickup from processing status', () => {
  const actual = canCompletePickup('PROCESSING', 0, false);
  assert.equal(actual, false);
});
```

Mỗi negative case giữ hai guard valid và đổi một guard. Khi fail, ta biết dimension nào đang thiếu.

Với error contract:

```
assert.throws(
  () => statusLabel('UNKNOWN'),
  /Unknown order status/
);
```

Với async rejection:

```
await assert.rejects(
  () => recordCollection('LD-001', -1),
  /positive/
);
```

Prediction

Test nào hữu ích, test nào tautology?

```
const actual = lineAmount(25000, 2);

assert.equal(actual, 50000);
assert.equal(actual, lineAmount(25000, 2));
assert.equal(typeof actual, 'number');
assert.ok(actual);
```

Không phải assertion sai cú pháp là vô ích; hãy nói chính xác contract mỗi assertion chứng minh và cái gì nó bỏ sót.

Prediction runner:

```
test('async failure', async () => {
  assert.rejects(
    () => Promise.reject(new Error('boom')),
    /boom/
  );
});
```

Thiếu await trước assert.rejects tạo risk gì?

Completion task

Viết tests cho normalizeStoredPhone có sẵn:

Production helper được cấp ở foundations/work/js07.mjs. M chỉ viết assertion trong foundations/work/js07.learner.test.mjs; chạy npm run lab:js07.

- plain local phone;
- spaces/dots/hyphens;
- +84 conversion;
- shorter than 10 digits;
- wrong first digit;
- alphabetic input;
- input không mutate.

Mỗi case phải có tên behavior. Không loop toàn bộ cases ngay lần đầu; viết ba cases rõ, sau đó mới cân nhắc table-driven tests nếu readability tốt hơn.

Independent task

Cho canonical-like ledger function `deriveBalance(orderTotal, transactions)`. Từ requirement, tự lập decision table rồi viết suite chứng minh:

- no transactions;
- one collection;
- multiple collections;
- collection + refund;
- overpaid mathematical remaining clamped at zero;
- unknown transaction type bị reject;
- input transaction array không mutate.

Sau đó cố ý đổi production code từ subtract refund thành add refund. Suite phải đồ đúng case và message giúp xác định contract.

Failure injection

Failure A - expected từ actual

```
const actual = statusLabel('RECEIVED');
const expected = actual;
assert.equal(actual, expected);
```

Failure B - assertion quá rộng

```
assert.ok(order);
```

Nếu requirement là đúng code/status/total, truthy object không đủ.

Failure C - test phụ thuộc thứ tự

Test A mutate shared array; Test B mong array sạch. Chạy B riêng, cả suite, đảo declaration order. Ghi evidence khác nhau.

Failure D - catch làm test pass

```
test('rejects invalid phone', () => {
  try {
    normalizeStoredPhone('abc');
  } catch (error) {
    console.log(error.message);
  }
});
```

Test pass cả khi không throw. Sửa bằng assertion đúng contract.

Failure E - chỉ test happy path

Cài production code luôn return true. Nếu suite chỉ có happy path, nó xanh. Thêm smallest negative case khiến mutation bị bắt.

Transfer

Map Node test sang Playwright:

Node test callback	↔ Playwright test callback
function input	↔ page/data/precondition
function call	↔ browser action
assert.equal	↔ web-first expect
returned/error value	↔ user-visible outcome/network/state

Cho flow “Staff mở order LD-002”, viết test contract trước locator:

- Arrange nào cần seed/login?
- Action duy nhất là gì?
- Observable outcome nào chứng minh đúng order, không chỉ “page có mở”?
- Negative case nào chứng minh wrong role hoặc wrong code?

Gate JS07

Pass khi m:

- viết Arrange-Act-Assert rõ;
- giải thích test runner gọi callback;
- tạo expected từ requirement độc lập;
- dùng assert cho value, throw và rejection;
- làm mutation khiến suite phải đổ;
- chứng minh shared state bằng order variation;
- hoàn thành ledger suite no-AI.

No-AI drill JS07 - 45 phút

Nhận một pure function chưa có tests. Viết decision table, tối thiểu một happy + ba negative/boundary cases, cài mutation và chứng minh suite bắt được.

JS08 - Cầu nối vào Playwright: đọc một spec từ trái sang phải

Thời lượng thường gặp: 4-5 giờ **Prerequisite:** JS01-JS07 **Exit outcome:** M đọc anatomy một Playwright spec, biết runner/callback/fixture/locator/action/assertion, tự hoàn thành spec nhỏ và phân loại failure.

Mental model

Playwright Test là test runner + browser automation library. Một spec TypeScript vẫn là chương trình dùng function, callback, object destructuring, Promise và assertions đã học.

```
import { test, expect } from '@playwright/test';

test('visitor sees active services', async ({ page }) => {
  await page.goto('/services');

  const row = page
    .getByRole('row')
    .filter({ hasText: 'Giặt sấy quần áo' });

  await expect(row).toContainText('KG');
  await expect(row).toContainText('25.000 ₫');
});
```

Đọc từng layer:

1. Import lấy named exports từ package.
2. `test` nhận title và async callback.
3. Runner đăng ký rồi gọi callback sau, cung cấp fixtures.
4. `{ page }` destructure property `page` từ fixture object.
5. `page.goto` trả Promise; callback await navigation.
6. Locator là query recipe, chưa phải cached DOM node.
7. `filter` scope row theo business text.
8. `expect(locator)` tạo web-first assertion retry condition.
9. Test outcome là đúng service row hiển thị unit/price, không chỉ URL mở được.

Test code có ba loại intent:

Setup/precondition → action → observable assertion

Đừng giấu assertion vào generic action helper quá sớm. Khi beginner không nhìn thấy outcome, suite dễ thành script click qua UI.

Worked example

```
test('staff opens order LD-002 from the order list', async ({ page }) => {
  await page.goto('/orders?order=desc');

  const targetOrder = page
    .getByRole('row')
    .filter({ hasText: 'LD-002' });

  await targetOrder
    .getByRole('link', { name: 'Chi tiết' })
    .click();

  await expect(
    page.getByRole('heading', { name: 'Đơn LD-002' })
  ).toBeVisible();
});
```

Tại sao không dùng `getByRole('link', { name: 'Chi tiết' }).nth(1)`? Vì position không phải identity; DOM đã đảo order. Scope row giống `array.find` bằng code ở JS04.

Tại sao vẫn assert heading sau click? Click không chứng minh navigation tới đúng order hoặc render đúng detail.

Prediction

Đọc, không chạy:

```
test('save phone', async ({ page }) => {
  page.goto('/slow-form');
  page.getByLabel('Số điện thoại').fill('0912345678');
  page.getByRole('button', { name: 'Lưu' }).click();
  expect(page.getByRole('status')).toHaveText('Đã lưu 0912345678');
});
```

Cho từng dòng:

- return value là gì?
- dependency sau cần outcome nào?
- missing await có thể tạo race, teardown hay false green nào?
- tại sao symptom không nhất thiết giống nhau mỗi lần?

Prediction locator:

Trang có ba link tên “Chi tiết”. Locator link toàn trang resolve bao nhiêu element? Assertion/click có thể báo loại lỗi nào? Scope business object nên dùng text nào?

Completion task

Completion có hai artifact, làm đúng thứ tự:

1. Hoàn thành `foundations/work/js08.mjs:buildOrderTestPlan(code)` phải mô tả

fixture, row theo exact business-code cell, action semantic, visible-heading oracle và ba awaited steps;

`openOrderByCode(page, code)` phải thực thi đúng dependency đó. Chạy `npm run`

`foundation:test:work -- JS08` cho tới khi Node contract xanh.

2. Sau khi tự viết browser attempt, đổi hai marker trong

`tests/work/foundation/js08_bridge.spec.ts` từ `test.fixme` thành `test`, rồi chạy `npm run lab:js08`. Command này chấm lại cả helper và browser.

Artifact browser thật: `tests/work/foundation/js08_bridge.spec.ts`; `command: npm run lab:js08`.

Không dùng fake Page contract thay bài này.

```
test('visitor sees the ironing service', async ({ page }) => {
  // TODO 1: navigate to service page
  // TODO 2: locate row by service name, not index
  // TODO 3: assert ITEM and 12.000 đ inside that row
});
```

Bắt buộc:

- semantic locator;
- mọi Promise được await;
- ít nhất hai assertions cùng scoped row;
- gây một assertion fail rồi ghi call log/first own line;
- reorder table vẫn pass.

Independent task

Không nhìn worked example trong 25 phút:

1. Mở order list ở descending DOM order.
2. Tìm LD-001 bằng business code.
3. Click đúng link “Chi tiết” trong row.
4. Assert heading chứa đúng code.
5. Assert page không hiển thị detail của LD-002 bằng contract hợp lý.
6. Chạy riêng test và lặp lại ít nhất ba lần.

Viết teach-back: test runner làm gì, browser object nào được dùng, locator giữ gì, assertion chờ gì.

Failure injection

Failure A - syntax/import

Xóa closing brace hoặc đổi `expect` thành named import không tồn tại. Test callback có được chạy không?

Failure B - no tests collected

Đổi file/name/path ngoài `testDir`. Phân biệt “0 tests” với “tests pass”.

Failure C - locator zero/multiple

Đổi accessible name sai, rồi bỏ row scope. So sánh call log.

Failure D - missing await

Bỏ await lần lượt khỏi goto, fill/click và expect. Mỗi lần chạy isolated, ghi symptom thực tế; không khẳng định cả ba luôn fail giống nhau.

Failure E - weak oracle

Chỉ assert URL chứa `/orders/detail`. Server trả detail nhầm code vẫn pass. Thêm user-visible business identity.

Transfer

Cho một màn Laundry thật (chỉ read requirement/screenshot nếu repo chưa chạy), thiết kế spec skeleton:

```
Title:
Actor/precondition:
Business object identity:
Action:
Observable outcome:
Negative oracle:
Data owner:
```

Không thêm REST/test-support endpoint vào canonical app. Nếu chưa có deterministic data contract, ghi đó là blocker/design question chứ không hard-code row position.

Gate JS08

Pass khi m:

- annotate đúng registration/callback/fixture/action/assertion;
- giải thích destructuring `{ page }`;
- viết hai specs không copy;
- chọn row bằng business identity;

- phân biệt collection/import, locator, action và assertion failures;
- sửa missing await từ evidence;
- làm transfer skeleton cho Laundry.

No-AI drill JS08 - 45 phút

Từ prompt business một đoạn, viết spec, chạy, tự cài strictness và missing-await bugs, sửa bằng call log. Không mở reference spec hoặc chat.

TS01 - TypeScript strict: feedback trước khi chạy

Thời lượng thường gặp: 4-5 giờ **Prerequisite:** JS01-JS08 **Exit outcome:** M giải thích TypeScript nằm ở đâu trong toolchain, dùng inference/annotation cho variable, function, array/object và sửa type error thay vì né bằng any.

Mental model

TypeScript là JavaScript có static type checker và thêm syntax cho types. Browser/Node cuối cùng chạy JavaScript; toolchain kiểm tra/transpile TypeScript trước hoặc trong quá trình test.

```
.ts source
→ TypeScript parser + type checker
→ nếu type contract sai: feedback trước runtime
→ nếu đạt: JavaScript/runtime behavior
→ test/assertion vẫn có thể fail
```

TypeScript không chứng minh business behavior đúng. Nó có thể chặn `order.costumer` nếu type chỉ có `customer`, nhưng không biết giá 25.000 đ có đúng requirement hay không.

`tsconfig.json` định nghĩa project type-check như thế nào. V4 dùng strict baseline. Không tắt `strict` hoặc thêm `// @ts-ignore` chỉ để hết đồ.

Inference nghĩa là TypeScript suy type từ value:

```
const orderCode = 'LD-001'; // inferred string
const amount = 25000;       // inferred number
const active = true;        // inferred boolean
```

Annotation ghi contract khi inference không đủ rõ hoặc tại boundary:

```
let currentStatus: string = 'RECEIVED';

function lineAmount(unitPrice: number, quantity: number): number {
  return unitPrice * quantity;
}
```

Object type mô tả shape:

```
const service: {
  code: string;
  name: string;
  price: number;
  active: boolean;
} = {
  code: 'GIAT_SAY_KG',
  name: 'Giặt sấy quần áo',
  price: 25000,
  active: true
};
```

Array type:

```
const orderCodes: string[] = ['LD-001', 'LD-002'];
```

Function type kiểm tra input/output ở call site và implementation. Nếu function hứa return number nhưng một path trả undefined, strict checking giúp lộ lỗi hổng.

Worked example

Từ JavaScript:

```
function findOrderByCode(orders, code) {
```

```
    return orders.find(order => order.code === code);
  }
}
```

Sang TypeScript bước đầu:

```
type Order = {
  code: string;
  status: string;
  total: number;
};

function findOrderByCode(orders: Order[], code: string): Order | undefined {
  return orders.find(order => order.code === code);
}

const orders: Order[] = [
  { code: 'LD-001', status: 'RECEIVED', total: 75000 },
  { code: 'LD-002', status: 'PROCESSING', total: 120000 }
];

const target = findOrderByCode(orders, 'LD-002');

if (target !== undefined) {
  console.log(target.status);
}
```

Tại sao return là `Order | undefined`? Vì `find` có thể không thấy. Type checker buộc caller xử lý khả năng đó trước khi đọc property. Đây là honest contract, không phải TypeScript gây khó.

Lưu ý: `type alias` được giới thiệu nhẹ ở đây để đặt tên shape; TS02 sẽ học sâu hơn về domain types.

Prediction

Không chạy typecheck. Với từng dòng, dự đoán pass/type error/runtime-only risk:

```
const code: string = 'LD-001';
const amount: number = '25000';
const active = true;
active = false;

function double(value: number): number {
  return String(value * 2);
}

const order = { code: 'LD-001', total: 50000 };
console.log(order.status);
```

Phân biệt lỗi `const` reassignment do JavaScript rule và mismatch do TypeScript contract.

Prediction inference:

```
let value = 1;
value = 2;
value = 'two';
```

Type được suy ở declaration là gì? `let` không có nghĩa variable chấp nhận mọi type.

Completion task

Starter JavaScript ở `foundations/fixtures/ts01-starter.mjs` có object services và ba helpers. Đọc/chạy input đó, rồi chuyển contract sang target `foundations/work/ts01.ts` và hoàn thành types:

Input command: `npm run lab:ts01:input`. Target command: `npm run lab:ts01`.

```
type Service = {
  code: TODO;
  name: TODO;
  unitPrice: TODO;
```

```

    active: TODO;
  };

function activeServices(services: TODO): TODO {
  return services.filter(service => service.active);
}

function findService(services: TODO, code: TODO): TODO {
  return services.find(service => service.code === code);
}

function displayPrice(service: TODO): TODO {
  return service.name + ': ' + service.unitPrice + ' VND';
}

```

Bắt buộc chạy:

```

npm run typecheck
npm run lab:ts01

```

Hai command trả lời hai câu hỏi khác nhau. Ghi cả hai outputs.

Independent task

Tiếp tục trong phần khung independent đã có sẵn ở cuối `foundations/work/ts01.ts`; “từ file trắng” ở đây nghĩa là tự viết logic từ contract, không phải tạo thêm file hay đoán đường dẫn. Model:

```

CustomerSnapshot: name, phone
OrderItem: serviceCode, serviceName, unitPrice, quantity
Order: code, customer, items

```

Viết typed functions:

- `orderTotal(items)`;
- `findItem(items, serviceCode)` với honest return type;
- `orderSummary(order)`;
- `requireItem(items, serviceCode)` throw khi không có và return `OrderItem` khi có.

Mở scaffold `foundations/type-tests/ts01.learner-errors.ts`, cài ba invalid calls và dùng `npm run foundation:typecheck` để chứng minh chúng bị chặn. Đặt `// @ts-expect-error --` lý do của m ngay trên từng call: command xanh chỉ khi compiler thật sự phát hiện lỗi dự kiến; nếu call vô tình trở thành hợp lệ, directive thừa sẽ làm command đỏ. Không dùng `any`, `cast` hay `@ts-ignore` để né bài.

Failure injection

Failure A - dùng any để bịt lỗi

```

function printOrder(order: any) {
  console.log(order.costumer.nmae);
}

```

Typecheck xanh nhưng runtime có thể crash. Thay `any` bằng shape thật và quan sát hai typo bị bắt.

Failure B - non-null assertion vô căn cứ

```

const target = orders.find(order => order.code === 'LD-X')!;
console.log(target.status);

```

Dấu `!` bảo checker tin m; nó không tạo object ở runtime. Sửa bằng guard hoặc `requireOrder` có explicit error.

Failure C - compile xanh, business sai

```

function lineAmount(price: number, quantity: number): number {
  return price + quantity;
}

```

Types đúng nhưng operator sai. Cần test/oracle từ JS07.

Failure D - return path thiếu

Khai báo function return `string` nhưng unknown status không return/throw. Dùng checker feedback để sửa honest contract.

Transfer

Mở một Playwright spec và hover/read types nếu editor hỗ trợ:

```
test('service', async ({ page }) => {
  const row = page.getByRole('row');
  await expect(row).toBeVisible();
});
```

Không cần nhớ definition library. Ghi:

- TypeScript suy `page` từ fixture callback như thế nào ở mức high-level;
- `row` có phải DOM element không;
- action/assertion trả Promise nên `await` vẫn là runtime dependency, không phải chỉ type syntax;
- typo method có thể bị checker bắt trước browser launch.

Gate TS01

Pass khi m:

- kể được source → checker → runtime → assertion;
- dùng inference khi rõ, annotation ở function/boundary;
- type object, array, parameter và return;
- xử lý `undefined` từ `find` không dùng `!` vô căn cứ;
- giải thích type-safe không đồng nghĩa behavior-correct;
- hoàn thành independent task với strict typecheck và runtime tests.

No-AI drill TS01 - 40 phút

Chuyển một JS data helper sang strict TS, sửa tối thiểu năm type errors mà không dùng `any/@ts-ignore`, rồi cài một business mutation để chứng minh typecheck không thay tests.

TS02 - Domain types: union, literal, optional, readonly và discriminated data

Thời lượng thường gặp: 5 giờ **Prerequisite:** TS01 **Exit outcome:** M dùng union/literal để loại illegal values, model optional đúng nghĩa, đọc `type/interface`, và dùng discriminated union cho transaction/status behavior.

Mental model

`string` quá rộng cho một field chỉ nhận vài value. Literal union mô tả set hợp lệ:

```
type OrderStatus =
  | 'RECEIVED'
  | 'PROCESSING'
  | 'READY_FOR_PICKUP'
  | 'COMPLETED'
  | 'CANCELLED';
```

Giờ typo `'READY'` bị checker chặn. Union không thay runtime validation cho text từ HTTP/JSON; external value chưa tự trở thành `OrderStatus` chỉ vì m annotate.

`type` và `interface` đều có thể đặt tên object shape. Trong foundation:

- dùng `type` thuận tiện cho union và composition;
- đọc được `interface` vì library/framework dùng nhiều;
- không tranh luận style khi chưa có project convention.

```
interface CustomerSnapshot {
  name: string;
  phone: string;
}
```

Optional property `note?: string` nghĩa property có thể vắng/undefined. Nó không tự đồng nghĩa empty string hoặc null. Requirement phải quyết định.

`readonly` ngăn assignment qua type contract:

```
type Transaction = {
  readonly code: string;
  readonly amount: number;
};
```

Đây là compile-time protection, không deep-freeze runtime object.

Discriminated union dùng một field literal để nối variant với fields tương ứng:

```
type PaymentTransaction =
  | { type: 'COLLECTION'; amount: number; method: PaymentMethod }
  | { type: 'REFUND'; amount: number; method: PaymentMethod; reason: string };
```

Khi `type === 'REFUND'`, checker biết `reason` tồn tại.

Worked example

```
type PricingUnit = 'KG' | 'ITEM';

type ServiceSnapshot = {
  readonly serviceCode: string;
  readonly serviceName: string;
  readonly unit: PricingUnit;
  readonly unitPrice: number;
  quantity: number;
  note?: string;
};

function quantityLabel(item: ServiceSnapshot): string {
  if (item.unit === 'KG') {
    return item.quantity + ' kg';
  }

  return item.quantity + ' món';
}

function noteLabel(item: ServiceSnapshot): string {
  return item.note === undefined ? 'Không có ghi chú' : item.note;
}
```

Literal union giúp condition exhaustive ở set hiện tại. `readonly` trên snapshot fields diễn đạt “test data không nên reprice/rename history”. Nó gọi nhớ canonical concept nhưng không thay immutable database history.

Prediction

Với mỗi assignment, dự đoán checker:

```
let status: OrderStatus = 'RECEIVED';
status = 'PROCESSING';
status = 'READY';

const item: ServiceSnapshot = {
  serviceCode: 'GIAT_SAY_KG',
  serviceName: 'Giặt sấy quần áo',
  unit: 'KG',
  unitPrice: 25000,
  quantity: 2
};

item.quantity = 3;
item.unitPrice = 30000;
console.log(item.note.length);
```

Phân biệt optional handling với non-null assertion.

Prediction discriminated union:

```
type Result =
  | { ok: true; value: string }
  | { ok: false; error: string };

function print(result: Result) {
  if (result.ok) {
    console.log(result.value);
  } else {
    console.log(result.error);
  }
}
```

Tại mỗi branch, property nào hợp lệ?

Completion task

Hoàn thành domain types:

Artifact: foundations/work/ts02.ts; **command:** npm run lab:ts02.

```
type Role = TODO;
type OrderStatus = TODO;
type IssueStatus = TODO;
type PaymentMethod = TODO;

type PaymentTransaction =
  | {
    readonly code: string;
    type: TODO;
    amount: number;
    method: PaymentMethod;
  }
  | {
    readonly code: string;
    type: TODO;
    amount: number;
    method: PaymentMethod;
    reason: string;
  };
```

Sau đó viết `transactionLabel` và `signedAmount`. Refund phải có reason ở compile time; collection không có reason bắt buộc. Sau runtime contract, mở `foundations/type-tests/ts02.learner-errors.ts` và làm ba intentional type errors theo scaffold; `npm run foundation:typecheck` phải xác nhận bằng `@ts-expect-error` có lý do, không làm project đổ vĩnh viễn.

Independent task

Model một typed scenario cho pickup:

```
actor role
order status
remaining amount
issues[] với OPEN/RESOLVED
optional note
expected outcome: allowed hoặc rejected với reason code
```

Dùng discriminated union cho result:

```
type PickupDecision =
  | { allowed: true }
  | { allowed: false; reason: ... };
```

Viết `decidePickup` và tests. Thêm một reason mới rồi quan sát nơi checker bắt code chưa xử lý hết.

Failure injection

Failure A - union bị làm rộng

Đổi `OrderStatus` thành `string`; typo status typecheck xanh. Dùng type-negative case trong `foundations/type-tests/ts02.learner-errors.ts` chứng minh regression, rồi restore union trước khi qua case khác.

Failure B - optional dùng như chắc chắn có

`item.note.toUpperCase()`. Sửa bằng branch/default có business meaning, không dùng `!`.

Failure C - readonly không phải runtime freeze

Ép/cast qua một mutable shape hoặc mutate nested array và quan sát. Giải thích vì sao readonly vẫn có giá trị nhưng không phải security boundary.

Failure D - optional che missing required data

Biến mọi field thành ? để object dễ tạo. Checker không còn bảo vệ contract. Chỉ property thật sự có thể vắng mới optional.

Transfer

Playwright test data thường có role/status variants. Thiết kế:

```
type Account =  
  | { role: 'STAFF'; username: string }  
  | { role: 'MANAGER'; username: string };
```

Sau đó trả lời:

- Vì sao không thêm `ANONYMOUS` như một account có username giả?
- Một anonymous scenario nên là variant khác thế nào?
- Type nào giúp project config tránh gõ sai role project name?
- Runtime storage state vẫn có thể hết hạn dù types đúng như thế nào?

Gate TS02

Pass khi m:

- thu hẹp status/role/unit bằng literal unions;
- giải thích optional khác null/empty;
- dùng readonly đúng kỳ vọng;
- tạo và narrow discriminated union;
- không biến mọi field thành optional để hết lỗi;
- hoàn thành pickup decision + new-reason variation no-AI.

No-AI drill TS02 - 45 phút

Nhận object types quá rộng. Thu hẹp thành literal/discriminated unions, typecheck negative cases, thêm một variant mới và sửa mọi non-exhaustive branch.

TS03 - Unknown, narrowing và boundary validation: đừng tin dữ liệu ngoài

Thời lượng thường gặp: 5-6 giờ **Prerequisite:** TS01-TS02 **Exit outcome:** M dùng `unknown` cho external data/error, narrow bằng runtime evidence, viết type guard nhỏ và không dùng cast như validation.

Mental model

TypeScript types bị xóa trước runtime. Data từ JSON, environment, HTML, HTTP hay local storage không tự tuân type annotation.

```
external bytes/text/value
→ unknown
→ runtime checks/parse
→ trusted typed value
→ business/test logic
```

any **tất kiểm tra**: m có thể đọc/call gần như mọi thứ. `unknown` buộc m chứng minh trước khi dùng.

```
function printValue(value: unknown) {
  if (typeof value === 'string') {
    console.log(value.toUpperCase());
  }
}
```

`typeof`, `Array.isArray`, equality trên discriminant và property checks tạo narrowing. Type guard gom runtime checks thành function có predicate:

```
function isOrderStatus(value: unknown): value is OrderStatus {
  return value === 'RECEIVED'
    || value === 'PROCESSING'
    || value === 'READY_FOR_PICKUP'
    || value === 'COMPLETED'
    || value === 'CANCELLED';
}
```

Cast `value as OrderStatus` chỉ bảo checker tin. Nó không kiểm tra value ở runtime. Cast không phải parse/validation.

Trong `catch`, error nên được coi là `unknown`: JavaScript có thể throw bất kỳ value nào, không chỉ `Error`.

```
try {
  // work
} catch (error: unknown) {
  if (error instanceof Error) {
    console.error(error.message);
  } else {
    console.error('Non-Error thrown', error);
  }
}
```

Exhaustive check giúp compiler báo khi union thêm variant:

```
function assertNever(value: never): never {
  throw new Error('Unhandled value: ' + String(value));
}
```

Worked example

Ta parse một fixture JSON thay vì annotate mù:

```
type OrderStatus =
  | 'RECEIVED'
  | 'PROCESSING'
  | 'READY_FOR_PICKUP'
```

```

    | 'COMPLETED'
    | 'CANCELLED';

type OrderFixture = {
  code: string;
  status: OrderStatus;
  total: number;
};

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null;
}

function isOrderStatus(value: unknown): value is OrderStatus {
  return value === 'RECEIVED'
    || value === 'PROCESSING'
    || value === 'READY_FOR_PICKUP'
    || value === 'COMPLETED'
    || value === 'CANCELLED';
}

function parseOrderFixture(value: unknown): OrderFixture {
  if (!isRecord(value)) {
    throw new Error('Order fixture must be an object');
  }

  if (typeof value.code !== 'string' || value.code.length === 0) {
    throw new Error('Order fixture requires a non-empty code');
  }

  if (!isOrderStatus(value.status)) {
    throw new Error('Order fixture has an invalid status');
  }

  if (typeof value.total !== 'number' || !Number.isFinite(value.total)) {
    throw new Error('Order fixture requires a finite total');
  }

  return {
    code: value.code,
    status: value.status,
    total: value.total
  };
}

const external: unknown = JSON.parse('{"code":"LD-001","status":"RECEIVED","total":75000}');
const order = parseOrderFixture(external);
console.log(order.status);

```

Checker chỉ cho return typed object vì từng field đã được narrowed. Đây là validation nhỏ cho bài học; project thật có thể dùng schema library theo convention, không tự xây framework validation lớn.

Prediction

Cho code:

```

const external: unknown = JSON.parse('{"status":"READY"}');

console.log(external.status);
console.log((external as { status: OrderStatus }).status);

```

Dòng nào checker chặn? Dòng cast có chạy không? Runtime value trở thành canonical status thật chưa?

Prediction narrowing:

```

function describe(value: unknown): string {
  if (typeof value === 'string') {
    return 'text ' + value.length;
  }
}

```

```

    if (typeof value === 'number') {
      return 'number ' + value.toFixed(0);
    }

    return 'other';
  }

```

Tại mỗi branch, checker biết gì? `null` đi branch nào?

Completion task

Hoàn thành parser cho external transaction:

Artifact: `foundations/work/ts03.ts`; command: `npm run lab:ts03`.

```

type PaymentMethod = 'CASH' | 'BANK_TRANSFER_MANUAL';

type PaymentTransaction =
  | { type: 'COLLECTION'; amount: number; method: PaymentMethod }
  | { type: 'REFUND'; amount: number; method: PaymentMethod; reason: string };

function parseTransaction(value: unknown): PaymentTransaction {
  // TODO object guard
  // TODO positive finite amount
  // TODO payment method guard
  // TODO discriminate COLLECTION/REFUND
  // TODO require non-empty reason only for REFUND
  // TODO reject unknown type
}

```

Tests phải có valid variants và ít nhất sáu invalid shapes. Không dùng `as PaymentTransaction` trong parser.

Independent task

Tạo loader cho một scenario file:

```

scenario
  role: STAFF | MANAGER
  orderCode: non-empty string
  expectedStatus: canonical status
  expectedVisibleActions: string[]

```

Đọc JSON thành `unknown`, `validate/narrow`, rồi dùng typed scenario tạo test title strings. Yêu cầu:

- error chỉ rõ field/path;
- empty actions hợp lệ nếu có chủ đích;
- array phải chứa toàn string;
- unknown extra fields có policy rõ: ignore hoặc reject, giải thích lựa chọn;
- invalid fixture làm command exit non-zero.

Nếu chọn reject extra field, `Object.keys(value)` cho danh sách key runtime;

`allowedFields.includes(key)` kiểm tra key có nằm trong danh sách cho phép. Đây là tool nhỏ của boundary policy, không phải cast. Viết `allowedFields: string[]` để checker hiểu mọi key string có thể được kiểm tra mà không cần `as`.

Failure injection

Failure A - cast-only parser

```

function parseOrder(value: unknown): OrderFixture {
  return value as OrderFixture;
}

```

Đưa missing code/wrong total; typecheck xanh, runtime logic sai. Ghi vì sao cast chỉ đổi checker view.

Failure B - `typeof null === 'object'`

Object guard chỉ kiểm tra `typeof value === 'object'`. Test với null và sửa.

Failure C - array cũng là object

Đưa `[]` vào `isRecord`. Quyết định parser của m chấp nhận hay reject array; implement evidence đúng policy.

Failure D - error assumed Error

Provided function throw `'boom'`. Catch rồi đọc `error.message` khi error unknown. Narrow đúng và giữ evidence.

Failure E - incomplete union switch

```
function statusLabel(status: OrderStatus): string {
  switch (status) {
    // intentionally omit CANCELLED
  }
}
```

Dùng `assertNever` ở default. Sau đó thêm status giả trong practice union và quan sát compile location.

Transfer

Trong automation, các boundary thường gặp:

- `response.json()`;
- environment variables;
- JSON fixtures;
- values từ page evaluation;
- error trong catch;
- config đọc từ file.

Cho API-like response sandbox, viết chain:

```
HTTP status assertion
→ parse JSON as unknown
→ validate minimal response shape
→ assert business fields
```

Không thêm REST route vào Laundry production chỉ để thực hiện bài này. Dùng sandbox API. Khi chuyển sang app thật, browser-visible HTML và Spring tests vẫn là contract chính của server-rendered architecture.

Gate TS03

Pass khi m:

- giải thích `unknown` khác `any`;
- narrow `string/number/object/array` và `error`;
- viết type guard cho literal union;
- parse external object không cast-only;
- dùng exhaustive check để lộ variant thiếu;
- hoàn thành scenario loader + malformed variation no-AI.

No-AI drill TS03 - 50 phút

Nhận năm JSON strings gồm hai valid, ba invalid. Parse unknown, tạo typed values hoặc explicit errors, thêm một variant mới và sửa exhaustiveness mà không dùng `any/as DomainType`.

TS04 - TypeScript trong Playwright: fixture, helper và failure layers

Thời lượng thường gặp: 5-6 giờ **Prerequisite:** TS01-TS03 và JS08 **Exit outcome:** M viết typed Playwright spec/helper/data, đọc library types ở mức cần thiết, phân biệt typecheck với browser failure và hoàn thành capstone foundation không AI.

Mental model

Trong Playwright Test, TypeScript tạo feedback cho test code trước hoặc khi runner collect spec. Library cung cấp types cho `test`, `fixtures`, `Page`, `Locator`, `config` và `assertions`.

```
test source + Playwright declarations + tsconfig
→ parser/type checker
→ runner collects tests
→ fixtures created
→ async callback runs
→ browser actions + web assertions
→ report/trace
```

Failure layer quyết định evidence:

Failure	Browser có launch không?	Evidence đầu tiên
syntax/type/import	thường chưa	checker/collection message + own line
no tests collected	không có test callback	config, testDir, filename
fixture setup	có thể chưa vào body	setup/fixture stack
locator/action	có page	call log + trace/DOM
assertion	có behavior actual	expected/actual + trace/network

Type annotations nên nằm ở boundary/helper, không phủ kín mọi local variable vốn inference tốt.

```
import type { Locator, Page } from '@playwright/test';

async function openOrder(page: Page, orderCode: string): Promise<void> {
  const row: Locator = page
    .getByRole('row')
    .filter({ hasText: orderCode });

  await row.getByRole('link', { name: 'Chi tiết' }).click();
}
```

Trong ví dụ, annotation `Locator` không bắt buộc vì inference đủ; giữ hay bỏ tùy clarity. `Promise<void>` làm async helper contract rõ cho learner.

Generic xuất hiện trong fixture/config/library declarations. Foundation chỉ cần mental model “type parameter cho phép cùng một structure giữ type cụ thể”. Không cần tự thiết kế generic framework trước khi làm được suite bình thường.

Worked example

Typed scenario và helper:

```
import { test, expect, type Page } from '@playwright/test';

type ServiceExpectation = {
  readonly name: string;
  readonly unit: 'KG' | 'ITEM';
  readonly priceText: string;
```

```

};

async function expectServiceRow(
  page: Page,
  expected: ServiceExpectation
): Promise<void> {
  const row = page
    .getByRole('row')
    .filter({ hasText: expected.name });

  await expect(row).toContainText(expected.unit);
  await expect(row).toContainText(expected.priceText);
}

test('visitor sees the active service price snapshot', async ({ page }) => {
  const expected: ServiceExpectation = {
    name: 'Giặt sấy quần áo',
    unit: 'KG',
    priceText: '25.000 ₫'
  };

  await page.goto('/services');
  await expectServiceRow(page, expected);
});

```

Helper nhận explicit dependency `page` và `expected` data. Nó vừa action/assertion helper; name `expect...` làm hidden assertion ít bất ngờ hơn. Không tạo class/POM chỉ để bọc một locator.

Type checker bắt unit `'KILOGRAM'`, missing name, number thay `priceText` và typo property. Runtime assertion vẫn cần vì DOM có thể sai.

Prediction

Phân loại type/collection/runtime:

```

type Scenario = {
  code: string;
  expectedStatus: 'RECEIVED' | 'PROCESSING';
};

const scenario: Scenario = {
  code: 'LD-001',
  expectedStatus: 'READY_FOR_PICKUP'
};

test('order status', async ({ page }) => {
  await page.goto('/orders');
  const row = page.getByRole('row').filter({ hasText: scenario.orderCode });
  expect(row).toContainText(scenario.expectedStatus);
});

```

Có ít nhất ba lỗi/risk khác layer. Ghi thứ tự toolchain có thể dừng; nếu type error đầu tiên chưa sửa, m không có evidence runtime cho locator.

Prediction helper:

```

async function savePhone(page: Page, phone: string): Promise<void> {
  page.getByLabel('Số điện thoại').fill(phone);
  page.getByRole('button', { name: 'Lưu' }).click();
}

await savePhone(page, '0912345678');

```

Function return Promise và caller await, nhưng bên trong vẫn thiếu await. Tại sao outer await không tự chờ inner promises bị bỏ rơi?

Completion task

Starter cung cấp:

Helper artifact: `foundations/work/ts04.ts`; spec thật:

`tests/work/foundation/ts04_bridge.spec.ts`; command: `npm run lab:ts04`.

```
type OrderScenario = {
  code: string;
  expectedCustomer: string;
  expectedStatus: 'RECEIVED' | 'PROCESSING' | 'READY_FOR_PICKUP';
};

async function openOrderByCode(/* TODO */): /* TODO */ {
  // TODO scoped row and click
}
```

Hoàn thành:

- typed Page parameter;
- `Promise<void>` return;
- semantic row selection by code;
- all inner Promises awaited;
- spec dùng scenario và assertions trên đúng detail;
- `npm run typecheck` và focused Playwright command đều pass.

Sau đó mở `foundations/type-tests/ts04.learner-errors.ts`, tạo intentional invalid scenario: wrong status và missing customer. Checker phải phát hiện. Sau browser attempt, đổi marker trong

`tests/work/foundation/ts04_bridge.spec.ts` từ `test.fixme` thành `test`; `npm run lab:ts04` sẽ chấm helper, strict typecheck và spec thật, đồng thời fail nếu test vẫn bị skip.

Independent task

Viết ba specs từ requirement, không nhìn worked example:

1. Visitor sees a named active service with correct unit and price.
2. Staff opens an order by business code after table reorder.
3. Slow form saves phone and displays user-visible success.

Tạo tối đa hai helpers và một typed data module. Constraints:

- Không CSS/XPath/nth()/waitForTimeout.
- Không `any`, `@ts-ignore` hay non-null assertion để lách checker.
- Không `expected-from-actual`.
- Không shared mutable scenario object.
- Mỗi spec có business outcome assertion.
- Chạy từng spec, toàn file, repeat và typecheck.

Failure injection

Failure A - type-only confidence

Đổi expected price thành một string hợp type nhưng sai requirement. Typecheck xanh, assertion đỏ. Ghi vai trò khác nhau của checker/oracle.

Failure B - helper mất Promise

Xóa `async/return` phù hợp hoặc bỏ inner `await`. Caller có thể `await undefined/fulfilled` helper sớm. Dùng `trace/call log` và `fix contract`.

Failure C - stale/wide scenario type

Dùng `expectedStatus: string`; typo lỗi. Thu hẹp literal union.

Failure D - fixture/data mutation

Một test sửa object exported dùng chung. Chạy riêng rồi cả suite/reorder để chứng minh pollution. Sửa bằng `factory/fresh object` và `owner contract`.

Failure E - wrong failure layer

Tạo import error rồi thử tăng Playwright timeout. Giải thích vì sao browser chưa chạy nên timeout không liên quan.

Transfer

Chọn một flow canonical Laundry, ví dụ UC-09 create order, và chỉ thiết kế typed E2E contract - không invent API:

```
type CreateOrderScenario = {
  actor: 'STAFF' | 'MANAGER';
  customerPhone: string;
  items: ReadonlyArray<{
    serviceCode: string;
    quantity: number;
  }>;
  expected: {
    initialStatus: 'RECEIVED';
    paymentRows: 0;
  };
};
```

Viết notes:

- type nào chỉ bảo vệ test data;
- business oracle nào cần server-visible/HTML evidence;
- canonical authority nào không được tính lại ở browser;
- data setup/cleanup blocker nào cần giải quyết sau;
- Spring flow nào sẽ được học reverse khi SWP bắt đầu.

Gate TS04

Pass khi m:

- trace được typecheck → collection → fixture → callback → browser → assertion;
- viết typed helper không mất Promise;
- dùng literal/object types cho scenario;
- phân biệt type, collection, action và assertion failure;
- ba independent specs pass typecheck, isolated, repeat và reorder;
- transfer UC-09 không thêm architecture ngoài canonical scope.

No-AI drill TS04 - 60 phút

Nhận một typed spec có bốn lỗi cài sẵn thuộc bốn tầng. Không chat/reference solution: phân loại, sửa theo tầng, chạy focused regression và viết root-cause journal.

Foundation Capstone - 90 phút không AI

Capstone này là gate cuối Part I và là checkpoint cuối tuần 3. Không cần hoàn hảo; nó đo khả năng tự bắt đầu, giữ mental model và debug khi không có AI làm hộ.

Input

M nhận:

- một requirement ngắn về order/service sandbox;
- một JSON scenario chưa được tin;
- một pure JS/TS helper có một bug;
- một Playwright starter spec có một bug khác tầng;
- commands đã được ghi trong lab README.

Nhiệm vụ

1. Orient - 10 phút

- xác định root/scripts;
- chạy baseline;
- ghi first meaningful failures;
- không sửa trong ba phút đầu;
- tạo tối đa ba hypotheses.

2. Data/type - 20 phút

- đọc JSON thành unknown;
- validate shape tối thiểu;
- tạo typed scenario không `any/cast-only`;
- typecheck.

3. Pure behavior - 15 phút

- viết hoặc sửa function;
- thêm oracle độc lập;
- cài một negative/boundary case;
- chứng minh test đổ trước fix hoặc mutation đổ sau fix.

4. Browser behavior - 25 phút

- viết/sửa semantic locator;
- await đúng dependency;
- assertion user-visible bằng business identity;
- không sleep/index coupling.

5. Failure/evidence - 10 phút

- kích hoạt seeded bug còn lại;
- ghi expected/actual/hypothesis/evidence/root cause;
- chạy focused regression.

6. Teach-back - 10 phút

Không nhìn note, kể:

```

command
→ module/typecheck
→ test registration
→ fixture callback
→ Promise/action
→ DOM locator
→ assertion/oracle
→ report

```

Rubric 10 điểm

Tiêu chí	0	1	2
Orient/predict	sửa mờ	có prediction nhưng thiếu layer	khoanh layer và evidence trước sửa
JavaScript	không tự hoàn thành	happy path chạy nhờ hint cũ	function/data/control flow + negative case độc lập
TypeScript	dùng any/cast để lách	typecheck nhưng contract rộng	unknown boundary + strict useful types
Playwright	script click/sleep/index	flow chạy nhưng oracle yếu	semantic, awaited, business outcome
Diagnose/teach	kể thao tác	có symptom và fix	root cause, evidence, regression và transfer

Pass khi từ 8/10 và không có 0 ở JavaScript, TypeScript, Playwright hoặc Diagnose. Nếu chưa pass, tạo đúng một repair card cho tiêu chí thấp nhất và retest bằng variation mới trong 48 giờ. Không xem full solution rồi tự chấm lại cùng đề.

Handoff sang Part II

Pass Foundation không có nghĩa m đã job-ready hoặc xây được toàn SWP. Nó nghĩa m có đủ ngôn ngữ và debugging hygiene để bắt đầu:

- Playwright core: HTTP/DOM/accessibility/auto-wait/isolation/auth/trace;
- Spring reverse: Browser → HTTP → Security → MVC → Service → Repository/DB → HTML;
- canonical Laundry vertical slices;
- CI baseline sau khi local suite đáng tin.

Docker chưa phải prerequisite của Part I. Trong ba tuần đầu, biết chạy một command/container stack có sẵn nếu repo yêu cầu là đủ. Không dùng Docker/CI course như cách trốn lỗi hỏng function, async, type hoặc test oracle.

Review queue cho ba tuần foundation

Mỗi prompt làm không nhìn note trước, rồi mới kiểm tra:

Khi nào	Retrieval prompt
D+1 QA00	Năm failure layers và evidence đầu tiên?
D+1 JS01	<code>'2' + 1, '2' * 1, Boolean('false')?</code>
D+1 JS02	Viết role matrix và một unknown branch.
D+1 JS03	Definition/call, parameter/argument, log/return?
D+1 JS04	<code>find</code> khác <code>index</code> ; shallow copy dùng chung gì?
D+1 JS05	Name/path/module mismatch dừng ở tầng nào?
D+1 JS06	Async function trả gì; <code>await</code> dùng cái gì?
D+1 JS07	Oracle đến từ đâu; mutation nào suite phải bắt?
D+1 JS08	Runner, callback, fixture, locator, action, assertion?
D+1 TS01	Inference/annotation; type-safe vs correct behavior?
D+1 TS02	Union/optional/readonly/discriminant?
D+1 TS03	Unknown → narrow → trusted value; cast thiếu gì?
D+1 TS04	Type → collection → fixture → runtime failure chain?

D+3: làm lại một completion task nhưng đổi data shape/name. D+7: làm independent task cũ từ file trắng hoặc một equivalent variation. D+14: seeded bug khác tầng và teach-back 5 phút.

Foundation definition of learned

Một chapter chỉ chuyển sang `LEARNED` khi m:

1. dự đoán behavior trước run;
2. hoàn thành independent task không walkthrough;
3. giải thích data/control/timeline bằng lời của m;
4. tìm một seeded failure bằng hypothesis và evidence;
5. làm transfer khác example;
6. recall lại sau khoảng cách;
7. làm no-AI drill tương ứng.

Nếu m chỉ nhận ra code khi nhìn thấy, đó là recognition. Nếu m tự dựng được từ contract, chẩn đoán variation và giải thích được, đó mới là usable skill.

PHẦN II - Kỹ năng QA nền: test design, API, SQL, Git, CI và English

Phần này là một **learning volume**, không phải checklist công nghệ và không phải tài liệu tìm việc. Mục tiêu là biến kiến thức Playwright thành năng lực kỹ thuật dùng được trong dự án thật, đồng thời tạo bằng chứng phục vụ SWP391 Laundry.

1. Vai trò của volume này trong lộ trình v4

Thị trường fresher/junior không chỉ cần người “biết viết Playwright”. Một Automation QA có thể làm việc phải nổi được cả chuỗi:

```
Requirement/risk
→ test design
→ browser/HTTP evidence
→ data oracle
→ defect diagnosis
→ version-controlled change
→ clean-machine execution
→ communication
```

Vì vậy, volume này dạy bảy năng lực:

1. QA fundamentals và test design.
2. HTTP và API testing.
3. SQL như một test oracle.
4. Git workflow trong team.
5. CI/CD ở mức đủ dùng cho Automation QA.
6. Docker operational literacy.
7. English for QA.

CI/CD và Docker không phải hai nhánh nghề mới trong ba tháng này. CI/CD chỉ cần đủ để test chạy được trên máy sạch, giữ report và hỗ trợ chẩn đoán. Docker chỉ cần đủ để khởi động dependency, hiểu port/environment/log và sửa lỗi vận hành cơ bản. Trọng tâm vẫn là testing, JS/TS, Playwright, HTTP/API, SQL, Git và khả năng giải thích evidence.

2. Hai đường chạy: ba tuần runway và hai tháng SWP

Các module dưới đây không thay thế volume JS/TS hay Playwright. Chúng chạy song song và được xoắn ốc lại trong dự án SWP.

Ba tuần runway

Tuần	Trọng tâm	Bằng chứng tối thiểu
1	QA fundamentals, Git căn bản, English QA mỗi ngày	Một risk map, một decision table, ba test case, một bug report; một branch có commit sạch
2	HTTP/API testing, SQL oracle	Một HTTP contract map, một API suite nhỏ, ba truy vấn có giải thích cardinality
3	CI đủ dùng, Docker vận hành, tích hợp các kỹ năng	Một pipeline xanh và một lần deploy có artifact; tự khởi động dependency bằng container; hoàn thành runway gate

Thứ tự trên là thứ tự phụ thuộc, không phải lịch cưỡng ép. Nếu mất một ngày, không học bù gấp đôi. Tiếp tục từ evidence gần nhất và chỉ qua module khi đạt gate.

Hai tháng SWP391 Laundry

Giai đoạn	Kỹ năng được dùng lại
Vertical core: login, catalogue, customer, create/view order	Requirement analysis, state/decision test, Git theo vertical slice, HTTP form/session/CSRF, SQL đọc snapshot
Operations và money: transitions, issue, collection/refund, pickup	Boundary/negative/concurrency testing, ledger oracle, failure diagnosis, regression selection
Management và submission: reports, settings, release	SQL aggregation/time boundary, CI evidence, Docker local dependency, English technical walkthrough

Mọi bài Laundry phải giữ canonical baseline: UI server-rendered bằng Spring MVC/Thymeleaf, session/CSRF, PostgreSQL/Flyway, năm order status, hai issue status, chín bảng và các BR/UC đã phê duyệt. Không thêm REST API, customer account, React/JWT, payment provider, delivery hay feature ngoài scope chỉ để luyện công nghệ.

3. Learning loop chung

Mỗi module dùng một vòng học cố định:

1. **Mental model:** hiểu hệ thống đang làm gì và đâu là nguồn sự thật.
2. **Worked example:** đọc một ví dụ hoàn chỉnh đã được giải thích.
3. **Prediction:** dự đoán trước khi chạy hoặc xem kết quả.
4. **Completion:** hoàn thiện một bài có khung.
5. **Independent:** tự làm một biến thể không có khung.
6. **Failure injection:** chủ động làm hỏng, thu evidence và chẩn đoán.
7. **Transfer:** áp dụng vào tình huống giống công ty và canonical Laundry.
8. **Gate:** chứng minh hiểu bằng output và lời giải thích, không chỉ bằng dấu xanh.

Mỗi artifact phải dùng dữ liệu giả hoặc dữ liệu được phép. Không đưa URL nội bộ, token, cookie, thông tin khách hàng, source code độc quyền hay ảnh màn hình nhạy cảm vào tài liệu học/public repository.

4. Bản đồ artifact executable - authority cho J01-J07

Path tính từ lab-kit/:

Module	Starter/evidence thật	Command hoặc gate
J01	labs/j01/UC20_TEST_DESIGN.md, UC25_TRACKING_PACK.md, mutation toy cùng folder	npm run lab:j01
J02	labs/j02/HTTP_TRACE_MATRIX.md và tests/work/p08_api.spec.ts	npm run lab:p08
J03	labs/sql/work/queries.sql; schema/seed và H5 reference nằm trong labs/sql/	Chạy đúng lệnh trong labs/sql/README.md trên PostgreSQL disposable từ J06 Compose
J04	labs/git-toy/	npm run lab:j04 tạo repo mới trong OS temp
J05	labs/ci/broken/playwright.yml, labs/ci/fixed/playwright.yml, tests/work/pl0_ci.spec.ts	npm run lab:p10; active baseline ở .github/workflows/playwright.yml
J06	labs/docker/compose.yaml và README.md	npm run lab:j06:check trước khi chạy Docker
J07	labs/j07/FAILURE_REPORT.md, REQUIREMENT_QUESTION.md	review evidence + oral gate

J01 - QA fundamentals: từ requirement đến test có lý do

Mục tiêu: Không bắt đầu bằng việc “automate tất cả”. Người học xác định được rủi ro, oracle, loại test, dữ liệu và mức coverage trước khi chọn UI/API/DB.

Mental model

Testing không phải chứng minh phần mềm không còn bug. Testing là thu thập thông tin để giảm bất định về một rủi ro có ý nghĩa.

Một test tốt trả lời được năm câu:

```
Ta đang lo điều gì?
→ kích hoạt hành vi bằng input/action nào?
→ quan sát ở đâu?
→ dựa vào oracle nào để kết luận?
→ nếu fail thì tác động là gì?
```

Phân biệt các khái niệm nền:

- **Test basis:** requirement, rule, acceptance criterion, bug history hoặc code contract mà test dựa vào.
- **Risk:** xác suất lỗi × tác động nếu lỗi xảy ra.
- **Oracle:** nguồn cho biết kết quả đúng là gì: business rule, response contract, DB invariant, calculation độc lập hoặc quan sát người dùng.
- **Test level:** unit/service, integration/repository, web/controller, API, E2E.
- **Test type:** functional, regression, usability, security-oriented, performance-oriented... Không đồng nhất test type với tool.
- **Positive path:** dữ liệu hợp lệ đi đến kết quả hợp lệ.
- **Negative path:** dữ liệu/quyền/state không hợp lệ bị chặn đúng cách và không để lại partial write.
- **Regression:** bằng chứng rằng thay đổi mới không làm hỏng hành vi đã có.

Một số cặp khái niệm dễ bị trộn ở mức fresher:

- **Verification:** artifact có đúng với specification đã chấp nhận không? **Validation:** sản phẩm có giải quyết đúng nhu cầu/ngữ cảnh sử dụng không?
- **Severity:** tác động kỹ thuật/business của defect. **Priority:** thứ tự/thời điểm team chọn sửa. QA cung cấp evidence về impact; product/team quyết định priority theo bối cảnh.
- **Smoke:** tập nhỏ chứng minh build/môi trường và critical path đủ khỏe để test tiếp. **Regression:** tập kiểm tra hành vi cũ sau thay đổi. **Retest/confirmation:** chạy lại đúng defect đã sửa.
- **Test scenario:** mục tiêu kiểm tra ở mức cao. **Test case:** precondition, data, actions và expected result cụ thể. **Exploratory charter:** mission/timebox/risk để vừa học vừa thiết kế vừa thực thi.

Một mini test plan đủ dùng phải nói được: scope/out-of-scope, risks, test levels/types, environment/data, responsibilities, entry/exit criteria, evidence/reporting và known constraints. Trong Agile, QA tham gia từ lúc refinement để làm rõ rule/testability; không chờ code xong mới bắt đầu “phase testing”.

Automation phù hợp khi oracle rõ, hành vi lặp lại, setup kiểm soát được và giá trị hồi quy cao. Exploratory testing phù hợp khi cần học sản phẩm, tìm rủi ro mới hoặc oracle chưa rõ.

Bộ kỹ thuật test design tối thiểu

- **Equivalence partitioning:** chia dữ liệu thành các nhóm được hệ thống xử lý giống nhau.
- **Boundary value analysis:** kiểm tra ngay dưới, đúng tại và ngay trên biên.
- **Decision table:** phối hợp nhiều điều kiện độc lập.

- **State transition:** kiểm tra trạng thái, hành động hợp lệ/không hợp lệ và trạng thái sau hành động.
- **Error guessing:** dùng lịch sử lỗi và hiểu biết hệ thống để đặt biến thể nguy hiểm.

Worked example - quyết định hoàn tất trả đồ

Canonical Laundry quy định pickup chỉ thành công khi:

- Order đang ở `READY_FOR_PICKUP`.
- `remainingAmount = 0`.
- Không có issue `OPEN`.

Thay vì viết ngay một Playwright test “click nút Hoàn tất”, ta tạo decision table:

Case	Ready?	Remaining = 0?	Có issue mở?	Kết quả business
P1	Có	Có	Không	Pickup thành công; order chuyển <code>COMPLETED</code> ; history được ghi
N1	Không	Có	Không	Bị từ chối vì sai trạng thái; không có history mới
N2	Có	Không	Không	Bị từ chối vì còn tiền; không có history mới
N3	Có	Có	Có	Bị từ chối vì issue mở; không có history mới
N4	Có	Không	Có	Bị từ chối; không partial write

N4 có thể không cần một E2E riêng nếu service-level decision-table test đã chứng minh tổ hợp đó và các E2E trọng yếu đã đủ. Coverage là quyết định theo rủi ro, không phải đếm số test tối đa.

Một test case có thể viết như sau:

```
ID: PICKUP-N2
Basis: UC-24, BR-30, BR-31
Risk: Order bị hoàn tất dù khách còn nợ
Precondition: READY_FOR_PICKUP; total 150.000 đ; net paid 100.000 đ; no OPEN issue
Action: Staff gửi lệnh hoàn tất pickup
Expected UI/HTTP: lệnh bị từ chối bằng thông báo business phù hợp
Expected persistence: status vẫn READY_FOR_PICKUP; không có COMPLETED history; completed_at vẫn null
Evidence: screenshot/trace + service/repository assertion tương ứng
```

Điểm quan trọng: thông báo UI chỉ là một observation. Oracle mạnh hơn còn gồm trạng thái và việc **không có partial write**.

Prediction - trả lời trước khi xem hệ thống

Không chạy ứng dụng. Với từng tình huống sau, ghi: rủi ro, kỹ thuật test design, test level rẻ nhất có thể bắt lỗi và một E2E có thật sự cần hay không.

1. Collection bằng 0, 1, đúng remaining và remaining + 1.
2. Chuyển `PROCESSING` → `READY_FOR_PICKUP` khi có issue mở.
3. Hai người cùng gửi toàn bộ remaining balance gần như đồng thời.
4. Wrong order code và wrong phone trên public tracking.
5. Giá catalogue thay đổi sau khi order đã được tạo.

Không mở hint trước khi hoàn thành bảng dự đoán.

Completion - hoàn thiện test design có khung

Điền trực tiếp `labs/j01/UC20_TEST_DESIGN.md`. Baseline mutation chạy bằng `npm run lab:j01`; chỉ mutate sau khi đã ghi test nào dự đoán sẽ đổ.

Điền bảng cho UC-20 Record customer payment:

ID	Status	Amount partition/boundary	Net paid trước	Expected result	Persistence oracle
C1	RECEIVED	hợp lệ	0	...	...
C2	...	0	...	...	...
C3	READY_FOR_PICKUP	đúng remaining	...	...	...
C4	...	remaining + 1	...	...	...
C5	COMPLETED	hợp lệ về số	...	...	...

Sau đó đánh dấu test nào nên ở service level, web/controller level, repository level và E2E. Không được chọn E2E cho tất cả.

Independent - tự thiết kế

Dùng `labs/j01/UC25_TRACKING_PACK.md`; không tạo một checklist rồi không trace được về UC/BR.

Tự tạo một test pack cho public tracking theo UC-25/BR-07/BR-09:

- Một risk map tối thiểu năm rủi ro.
- Equivalence partitions cho code và phone.
- Tối thiểu sáu test case, trong đó có privacy/no-leak.
- Chỉ rõ test nào có thể data-driven.
- Viết một exploratory charter 20 phút, không trùng hoàn toàn với scripted cases.

Failure injection - làm hỏng để học chẩn đoán

Trong lab/toy implementation, thay điều kiện collection từ `amount > 0` thành `amount >= 0`, hoặc bỏ guard “open issue blocks ready”. Không sửa ngay.

Thực hiện:

1. Dự đoán test nào phải đổ trước khi chạy.
2. Chạy lớp test nhỏ nhất liên quan.
3. Thu expected/actual và vị trí oracle bị vi phạm.
4. Chạy một test xa hơn để xem lỗi có bị bắt muộn ở UI hay không.
5. Khôi phục code và chứng minh suite xanh.

Nếu mutation không bị test nào bắt, đó là coverage gap. Bổ sung test ở level rẻ nhất phù hợp.

Transfer công ty

Với một user story được phép sử dụng:

1. Viết lại acceptance criteria thành các rule quan sát được.
2. Liệt kê ba rủi ro cao nhất trước khi mở tool automation.
3. Chọn test level cho từng rủi ro.
4. Tách smoke, critical regression và exploratory scope.
5. Khi có bug, ghi evidence để developer tái hiện được, không chỉ gửi ảnh “bị lỗi”.

Không sao chép requirement/data nội bộ sang learning repository. Dùng bản mô phỏng đã khử nhạy cảm.

Transfer Laundry

Chọn một vertical slice và trace đầy đủ:

```
UC/BR
→ screen/route/form
→ Controller
→ service transaction/guard
→ repository/table
→ test level
→ observable result
```

Ứng viên tốt cho module này: UC-16/17 lifecycle, UC-20 collection, UC-24 pickup hoặc UC-25 tracking. Không gọi một UC là done chỉ vì trang render hoặc happy path chạy được.

Gate J01

Bảng chứng	Điều kiện pass
Risk map + test design	Mỗi test truy ngược được về risk/rule; có positive, negative và boundary/state coverage
Test-level selection	Giải thích được vì sao không đẩy mọi thứ lên E2E
Bug report mô phỏng	Có build/environment, precondition, steps, expected, actual, evidence và impact
Oral check 5 phút	Phân biệt risk, oracle, test case, exploratory charter, regression và automation
Mutation proof	Dự đoán đúng test đỏ hoặc nhận diện được coverage gap, rồi đưa suite về xanh

Chưa pass nếu: chỉ liệt kê thao tác UI; expected ghi chung chung “hoạt động đúng”; không kiểm tra no-partial-write; hoặc không giải thích được test đang bảo vệ rule nào.

J02 - HTTP và API testing: nhìn xuyên qua UI

Mục tiêu: Đọc được request/response, phân biệt transport result với business result, test API có contract và dùng HTTP evidence để debug Playwright/Spring flows.

Mental model

Browser không “gọi button”. Browser tạo HTTP traffic.

```
Client
→ method + URL/path/query + headers + cookies + body
→ server/router/security/controller/service
→ status + headers + body
→ client render/redirect/state update
```

Một HTTP exchange cần đọc theo sáu phần:

1. **Method:** GET đọc; POST tạo/gửi command; PUT/PATCH cập nhật theo contract; DELETE yêu cầu semantics rõ.
2. **Address:** scheme, host, port, path, query string.
3. **Headers:** content type, accept, authorization, cookie, correlation ID, caching.
4. **Body:** JSON, form-urlencoded, multipart hoặc không có body.
5. **Status:** 2xx, 3xx, 4xx, 5xx; status chỉ nói một phần sự thật.
6. **Side effect:** dữ liệu/state có thực sự thay đổi đúng, thay đổi đúng một lần và không lộ thông tin hay không.

Session, cookie và CSRF

- Session authentication thường dùng cookie để nối request hiện tại với server-side session.
- Cookie không đồng nghĩa với quyền hợp lệ; server vẫn phải kiểm tra role/state.
- CSRF token chứng minh protected form request xuất phát từ session/page hợp lệ. Thiếu hoặc sai token phải bị chặn trước business command.
- Redirect 302 sau POST là mẫu Post/Redirect/Get. Nó ngăn browser refresh lập form ở mức UX, nhưng không thay thế server-side revalidation/concurrency guard.

API test không chỉ là status code

Một API test có giá trị kiểm tra các lớp phù hợp:

```
transport status
+ response headers/content type
+ body shape/types
+ business values/invariants
+ authorization/privacy
+ persisted side effect hoặc downstream observation
+ retry/duplicate behavior nếu có risk
```

Worked example A - API lab bằng Playwright request

Ví dụ dưới dùng **practice API của course lab**, không phải endpoint của Laundry. `LAB_API_BASE_URL` trỏ đến môi trường disposable.

```
import { test, expect } from '@playwright/test';

test('create task then read it back', async ({ request }) => {
  const title = `qa-lab-${Date.now()}`;

  const createResponse = await request.post(
    `${process.env.LAB_API_BASE_URL}/tasks`,
    { data: { title, priority: 'HIGH' } },
  );
});
```

```

expect(createResponse.status()).toBe(201);
expect(createResponse.headers()['content-type']).toContain('application/json');

const created: unknown = await createResponse.json();
expect(created).toMatchObject({ title, priority: 'HIGH', status: 'OPEN' });

const taskId = (created as { id: string }).id;
expect(taskId).toBeTruthy();

const readResponse = await request.get(
  `${process.env.LAB_API_BASE_URL}/tasks/${taskId}`,
);

expect(readResponse.status()).toBe(200);
await expect(readResponse).toBeOK();
expect(await readResponse.json()).toMatchObject({ id: taskId, title });
});

```

Ví dụ cố ý quan sát lại bằng GET thay vì tin rằng 201 tự động chứng minh dữ liệu đã lưu đúng. Ở dự án thật, cần parse `unknown` bằng `schema/type guard` thay vì cast mù; phần TypeScript foundation sẽ dạy kỹ bước này.

Worked example B - đọc một Laundry form flow

UC-20 không phải REST API. Đây là server-rendered form command:

```

POST /payments/collect?orderCode=LD-...
Cookie: authenticated session
Content-Type: application/x-www-form-urlencoded
Body: amount=50000&method=CASH&_csrf=...

Success:
302 Location: /orders/detail?code=LD-...
→ browser GET trang detail
→ flash message xuất hiện một lần
→ ledger có đúng một COLLECTION mới

```

Ba lớp failure có thể trông giống nhau trên UI nhưng evidence khác nhau:

- Thiếu CSRF: security trả 403; command service không chạy.
- Amount vượt remaining: request qua security nhưng business rule từ chối; không có transaction mới.
- SQL/server exception bất ngờ: generic 500 có correlation code; không lộ stack trace.

Không thêm `/api/payments` vào Laundry chỉ để luyện API. Dùng request/response của chính server-rendered flow để hiểu HTTP; dùng practice API riêng cho JSON/API exercises.

Prediction

Ghi status/redirect/side effect dự kiến trước khi gửi request:

1. Guest GET `/services`.
2. Guest GET `/orders/detail?code=...`
3. Authenticated Staff POST collection thiếu CSRF.
4. Authenticated Staff POST collection hợp lệ.
5. Authenticated Staff POST refund.
6. Manager POST refund với cumulative refund lớn hơn collection.

Với từng dòng, tách **HTTP expectation** và **business/persistence expectation**. Không suy luận rằng mọi business conflict bắt buộc phải cùng status nếu contract chưa quy định.

Completion

Điền `labs/j02/HTTP_TRACE_MATRIX.md`, rồi hiện thực hai test còn thiếu trong `tests/work/p08_api.spec.ts`.

Đây là gate hai chặng: matrix HTTP có thể làm ngay sau J01/HTTP foundation; phần executable P08 chỉ mở sau TS04 và P01-P07. Nếu chưa tới P08, giữ test file `fixme`, ghi restart line và không nhảy prerequisite để “hoàn thành J02” trên lịch.

Mở một trace/network log của practice app và hoàn thiện bảng:

Field	Request create	Response create	Request read	Response read
Method/path	...	-	...	-
Auth/session	...	-	...	-
Content-Type	...	...	...	...
Status	-	...	-	...
Business invariant	-	...	-	...
Correlation/ID	...	...	...	...

Sau đó bổ sung hai tests:

- Invalid body trả lỗi contract và không tạo record.
- Request không có quyền không đọc/ghi được resource.

Sau attempt thật, enable các P08 tests liên quan và chạy `npm run lab:p08`. Command còn bị block vì `test.fixme` hoặc chỉ report skipped thì J02 chưa completion.

Independent

Tự thiết kế một suite 6-8 tests cho một practice API resource:

- Create hợp lệ.
- Validation boundary.
- Read existing và missing.
- Update/state transition nếu contract có.
- Authorization/ownership.
- Duplicate/retry hoặc concurrency risk nếu có ý nghĩa.
- Cleanup theo API hoặc fixture disposable; không phụ thuộc test order.

Mỗi test phải ghi test basis và oracle. Không dùng public production API để tạo rác.

Failure injection

Lần lượt tạo ba lỗi trong lab:

1. Bỏ `await` trước khi đọc body hoặc trước một API action.
2. Chỉ assert status `200/201`, trong khi server cố ý trả body sai.
3. Dùng một title/ID cố định để hai test workers đụng dữ liệu.

Với mỗi lỗi:

- Dự đoán symptom.
- Chạy lặp hoặc parallel để tái hiện.

- Phân loại failure: test bug, product bug, environment hay data collision.
- Sửa nguyên nhân, không thêm sleep/retry mù.

Transfer công ty

Với một flow được phép kiểm thử:

1. Dùng browser network để map UI action → HTTP exchange.
2. Xác định contract owner và auth mechanism trước khi automate.
3. Chọn API test cho logic/contract và UI test cho user journey/rendering.
4. Khi UI test fail, lưu status, request ID/correlation ID, response excerpt đã khử nhạy cảm và trace.
5. Không log token, full cookie, password hoặc PII.

Transfer Laundry

Lập HTTP contract map cho một flow, ví dụ UC-25 hoặc UC-20:

Step	Route	Public/protected	Binding	Expected navigation	Rule
Open tracking	GET /track	Public	-	Render form	UC-25
Submit tracking	POST /track	Public + CSRF theo form contract	PublicTrackingForm	Limited result hoặc generic mismatch	BR-07/09
Collect	POST /payments/collect?orderCode=...	Staff/Manager	CollectionForm	PRG về detail	BR-24-30

Chứng minh một request bị security chặn, một request bị business rule chặn và một request thành công. Với success, xác minh flash chỉ xuất hiện một request sau redirect.

Gate J02

Bảng chứng	Điều kiện pass
HTTP contract map	Đủ method/path/query/header/body/status/side effect; không trộn REST giả vào Laundry
API lab suite	Isolated, không phụ thuộc thứ tự, assert cả contract và business value
Failure triage	Phân biệt được product/test/environment/data failure bằng evidence
Session/CSRF oral check	Giải thích được cookie, session, CSRF, 302 PRG và vì sao 302 chưa đủ chứng minh success
Laundry transfer	Trace đúng route → security/binding → command → redirect/observation

Chưa pass nếu: chỉ test status code; hard-code shared data; log secret; dùng sleep để chữa race; hoặc thêm API ngoài canonical scope.

J03 - SQL as oracle: kiểm tra dữ liệu mà không đoán

Mục tiêu: Đọc dữ liệu quan hệ bằng truy vấn an toàn, dùng SQL để xác minh invariant/diagnose nhưng không biến mọi E2E thành test phụ thuộc implementation.

Mental model

Database là một nguồn evidence, không mặc định là oracle duy nhất.

```
Business rule
→ expected relational state/invariant
→ read-only query
→ compare with UI/API observation
```

Các nền tảng bắt buộc:

- Table, row, column, primary key và foreign key.
- SELECT, WHERE, ORDER BY, LIMIT.
- INNER JOIN và LEFT JOIN.
- GROUP BY, COUNT, SUM, CASE.
- NULL và ba-valued logic; dùng IS NULL, không dùng = NULL.
- Cardinality: one-to-one, one-to-many, many-to-many; một join có thể nhân số dòng.
- Transaction visibility: query phải chạy đúng database/environment và đúng thời điểm sau commit.
- Constraint/migration: schema thật đến từ Flyway; entity đổi mà migration không đổi chưa phải schema change.

SQL oracle hữu ích khi cần xác minh ledger, history, snapshot, report aggregation hoặc điều tra failure. Không cần query DB trực tiếp để assert mọi label UI; điều đó tạo coupling và có thể bỏ qua contract từ góc nhìn người dùng.

Worked example - tính balance từ immutable ledger

Canonical Laundry không lưu `paid_amount` hay `remaining_amount` trên order. Balance được suy ra từ `payment_transaction`.

```
SELECT
  o.order_code,
  o.total_amount,
  COALESCE(SUM(CASE
    WHEN p.transaction_type = 'COLLECTION' THEN p.amount
    ELSE 0
  END), 0) AS collected_amount,
  COALESCE(SUM(CASE
    WHEN p.transaction_type = 'REFUND' THEN p.amount
    ELSE 0
  END), 0) AS refunded_amount,
  COALESCE(SUM(CASE
    WHEN p.transaction_type = 'COLLECTION' THEN p.amount
    WHEN p.transaction_type = 'REFUND' THEN -p.amount
    ELSE 0
  END), 0) AS net_paid_amount,
  GREATEST(
    o.total_amount - COALESCE(SUM(CASE
      WHEN p.transaction_type = 'COLLECTION' THEN p.amount
      WHEN p.transaction_type = 'REFUND' THEN -p.amount
      ELSE 0
    END), 0),
    0
  ) AS remaining_amount
FROM laundry_order o
LEFT JOIN payment_transaction p ON p.order_id = o.order_id
```

```
WHERE o.order_code = :order_code
GROUP BY o.order_id, o.order_code, o.total_amount;
```

Tại sao dùng `LEFT JOIN`? Order chưa có payment vẫn phải xuất hiện với các tổng bằng 0. Nếu dùng `INNER JOIN`, order chưa thanh toán biến mất và test có thể hiểu nhầm “không có order”.

Ví dụ tính tay độc lập với seed của lab: total 150000, collections 50000 + 100000, refund 20000:

```
collected = 150000
refunded   = 20000
net paid   = 130000
remaining  = 20000
```

Tên cột trong query phải được đối chiếu với Flyway migration thật trước khi chạy; không sửa schema chỉ để khớp ví dụ học.

Worked example - immutable state history

```
SELECT
  o.order_code,
  h.from_status,
  h.to_status,
  h.changed_at,
  h.history_id
FROM laundry_order o
JOIN order_status_history h ON h.order_id = o.order_id
WHERE o.order_code = :order_code
ORDER BY h.changed_at ASC, h.history_id ASC;
```

Oracle của một transition thành công không chỉ là current status. History phải chứa đúng transition. Oracle của lệnh thất bại là **không có history mới**.

Prediction

Không chạy query. Dự đoán row count và aggregate:

1. Một order không có payment với `INNER JOIN`.
2. Cùng order đó với `LEFT JOIN`.
3. Một order có hai items và ba payments nếu join cả items lẫn payments trước khi `SUM(p.amount)`.
4. `WHERE h.from_status = NULL` so với `WHERE h.from_status IS NULL`.
5. `COUNT(*)` trên `LEFT JOIN` so với `COUNT(p.transaction_id)`.

Viết lý do bằng cardinality, không chỉ ghi con số.

Completion

Schema/seed disposable ở `labs/sql/schema.sql` và `seed.sql`. Chỉ sửa `labs/sql/work/queries.sql`; H5 là `labs/sql/reference/queries.sql`.

Từ `lab-kit/labs/sql/`, khởi động và chứng minh database ready:

```
docker compose -f ../docker/compose.yaml up -d --wait
docker compose -f ../docker/compose.yaml exec postgres pg_isready -U lab_user -d laundry_lab
```

Sau khi đã ghi prediction, chạy work file ngay trong container; máy host không cần cài `psql`:

```
docker compose -f ../docker/compose.yaml exec -T postgres \
  psql -U lab_user -d laundry_lab \
  -v order_code=LD-001 \
  < work/queries.sql
```

`-v order_code=LD-001` cấp giá trị cho cú pháp `psql : 'order_code'`. Nếu shell báo không tìm thấy `work/queries.sql`, bạn đang đứng sai folder; nếu `pg_isready` chưa báo accepting connections, chưa được kết luận query sai. Oracle cụ thể và lệnh reset an toàn nằm trong `labs/sql/README.md`.

Hoàn thiện ba query từ skeleton:

1. Liệt kê payment history của một order theo `recorded_at DESC, transaction_code DESC`.
2. Tìm các order đang active và quá promised time tại một `now` được cung cấp.
3. Chứng minh một failed pickup không đổi status và không thêm history.

Mỗi query kèm:

- Mục đích/oracle.
- Expected cardinality.
- Vì sao chọn join type.
- Một case `NULL` hoặc no-child-row.
- Cách tránh chạy nhầm database.

Independent

Tự xây query kiểm tra revenue theo BR-32 cho một half-open time range:

```
[fromStart, dayAfterToStart)
grossCollected = sum(COLLECTION)
refunded = sum(REFUND)
netRevenue = grossCollected - refunded
```

Yêu cầu:

- Filter bằng `payment_transaction.recorded_at`, không bằng ngày tạo order/current status.
- Có case transaction đúng tại lower bound, ngay trước upper bound và đúng tại upper bound.
- Giải thích timezone `Asia/Ho_Chi_Minh` và instant range.
- So sánh kết quả query với report UI nhưng không hard-code theo machine timezone.

Failure injection

Tạo một query sai bằng cách join:

```
laundry_order
→ laundry_order_item (2 rows)
→ payment_transaction (3 rows)
```

Rồi `SUM(payment_transaction.amount)` trực tiếp. Quan sát payment rows bị nhân lên thành sáu combinations.

Nhiệm vụ:

1. Vẽ bảng trung gian sáu dòng.
2. Giải thích vì sao `DISTINCT amount` không phải bản sửa tổng quát.
3. Sửa bằng pre-aggregation/subquery hoặc tách query đúng mục đích.
4. Tạo data có hai payment cùng amount để chứng minh `SUM(DISTINCT amount)` sai.

Transfer công ty

- Chỉ dùng account/database được cấp quyền; ưu tiên read-only.
- Xác nhận environment/database/schema trước khi query.
- Không chạy UPDATE/DELETE để “làm cho test pass”.
- Nếu automated test dùng DB oracle, giải thích vì sao observation công khai chưa đủ và giới hạn coupling.
- Khi báo bug, cung cấp business key/correlation ID và query result tối thiểu đã khử PII; không dump cả bảng.

Transfer Laundry

Thực hành trên disposable local/test PostgreSQL được tạo bằng Flyway:

- UC-09: snapshot service/price/quantity/line amount và initial history.
- UC-20/23: immutable collection/refund và derived balance.
- UC-24: pickup success/failed no-partial-write.
- UC-27: transaction-date revenue.
- UC-28: cohort/order performance và service demand không trộn KG/ITEM.

Không dùng H2 để chứng minh PostgreSQL-specific query/time/locking semantics. Không thêm cột tổng tiền đã trả/còn lại vì điều đó vi phạm ledger source of truth.

Gate J03

Bảng chứng	Điều kiện pass
Năm query read-only	Kết quả đúng với zero/one/many child rows; có stable ordering khi cần
Cardinality explanation	Vẽ/giải thích được row multiplication và sửa đúng, không dùng <code>DISTINCT</code> như bùa
Ledger oracle	Tính đúng collected/refunded/net/remaining từ immutable rows
Time-boundary oracle	Dùng half-open instant range và shop timezone đúng
Safety check	Chứng minh đang ở disposable/read-only database; không dùng data thật nhạy cảm

Chưa pass nếu: chỉ copy query mà không dự đoán cardinality; sửa data bằng tay; aggregate sau một join nhân dòng; hoặc coi UI và DB trùng nhau là đủ mà không truy về rule.

J04 - Git workflow: thay đổi có thể hiểu, review và phục hồi

Mục tiêu: Làm việc bằng branch/commit có chủ đích, đọc được trạng thái repository, xử lý conflict an toàn và bàn giao một vertical change có traceability.

Mental model

Git lưu một graph các snapshot, không phải “Google Drive cho code”. Tại mọi thời điểm cần phân biệt:

```
working tree - file đang sửa
staging area - nội dung dự kiến vào commit kế tiếp
local commit - snapshot đã ghi vào graph local
remote branch - graph được chia sẻ
```

Ba câu hỏi chẩn đoán trước mọi thao tác:

1. Tôi đang ở branch nào?
2. Working tree/staging area có gì?
3. Commit/remote nào là base của thay đổi này?

Commit tốt có một mục đích, đủ nhỏ để review/revert và vẫn giữ repository ở trạng thái hợp lý. “Update code”, “fix stuff” hoặc một commit trộn feature, format, generated binary và tài liệu không liên quan làm mất khả năng hiểu lịch sử.

Worked example - một vertical test change

```
git switch -c test/uc25-public-tracking
git status --short

# Sửa test và fixture liên quan.
git diff

git add tests/public-tracking.spec.ts tests/fixtures/tracking.ts
git diff --staged
git commit -m "test: cover public tracking mismatch privacy"

# Nếu thay đổi canonical docs là thật sự cần thiết và đã được duyệt,
# cập nhật trong commit riêng có mục đích rõ.
git status
git log --oneline --decorate -5
```

`git diff` đọc phần chưa stage; `git diff --staged` đọc đúng nội dung sắp commit. Không commit `.env`, `auth state`, trace chứa secret, database dump hay credentials.

Trước khi tích hợp thay đổi từ team:

```
git fetch origin
git status
git log --oneline --graph --decorate --all -12
```

Chỉ merge/rebase sau khi hiểu branch state và theo workflow của team. Không dùng destructive reset để che một working tree chưa hiểu.

Prediction

Với mỗi trạng thái, dự đoán output của `git status --short`, `git diff` và `git diff --staged`:

1. File mới chưa tracked.
2. File đã sửa nhưng chưa stage.
3. File đã stage rồi sửa thêm lần nữa.

4. File secret vô tình đã stage.
5. Local branch có hai commit chưa có trên remote.

Sau đó tạo tình huống trong toy repository và kiểm tra dự đoán.

Completion

Chạy `npm run lab:j04` để tạo toy repository mới trong OS temp. Command in exact path; mọi branch/conflict/unstage của J04 phải diễn ra trong repo đó, không phải curriculum repo.

Trong toy repository:

1. Tạo branch từ base sạch.
2. Thực hiện một test change gồm fixture và assertion.
3. Stage chọn lọc, để một file note ngoài commit.
4. Viết commit message có scope và intent.
5. Tạo commit thứ hai sửa documentation liên quan.
6. Dùng log graph giải thích quan hệ hai commit với base.

Không dùng `git add .` cho bài này; mục tiêu là luyện chọn scope có ý thức.

Independent

Tự tạo conflict có kiểm soát trong toy repository:

- Hai branch sửa cùng một đoạn requirement-to-test mapping.
- Merge theo workflow đã chọn.
- Đọc conflict markers, quyết định nội dung dựa trên requirement, không chọn “ours/theirs” mù.
- Chạy test sau resolution.
- Commit resolution và giải thích vì sao bản cuối đúng.

Failure injection

Thực hiện ba sự cố an toàn trong toy repository:

1. Stage nhầm file `.env.example.local` chứa fake secret.
2. Commit thiếu fixture làm test fail trên clean checkout.
3. Reformat toàn file chung với một logic change nhỏ làm diff nhiều.

Nhiệm vụ:

- Phát hiện bằng `status`, staged diff và clean-checkout test.
- Gỡ file khỏi staging mà không xóa working copy.
- Tách hoặc làm lại commit trong toy repo.
- Ghi prevention rule cho dự án thật.

Transfer công ty

- Theo branch/PR convention của team; không tự áp workflow khác.
- Một change phải có requirement/ticket context, tests và evidence đủ review.
- Trước khi yêu cầu review, đọc lại full diff và chạy focused checks.
- Conflict được giải theo product intent; nếu intent không rõ thì hỏi owner, không tự chọn cho build xanh.
- Không đưa artifact nhạy cảm vào Git history; xóa file ở commit sau không làm secret biến khỏi history.

Transfer Laundry

Một branch nên đại diện một UC hoặc một vertical slice gắn chặt:

```
form/view
+ Controller
+ service rule/transaction
+ repository/migration nếu có
+ automated tests
+ canonical traceability/document update khi behavior thật sự đổi
```

Không chia ownership kiểu “một người chỉ frontend, một người chỉ backend” khiến không ai trace được end-to-end. Không commit generated PDF/diagram trước khi behavior/code/schema được freeze theo delivery gate.

Gate J04

Bảng chứng	Điều kiện pass
Branch có 2-3 atomic commits	Mỗi commit có một intent, diff review được và repository hợp lệ
Status/diff oral check	Phân biệt working tree, staging, commit và remote không cần đoán
Conflict lab	Resolve đúng theo requirement và test xanh
Clean-checkout proof	Dependency install/test chạy từ trạng thái sạch, không dựa file untracked local
Secret hygiene	Không credential/auth state/dump/trace nhạy cảm trong history

Chưa pass nếu: chỉ biết `add/commit/push`; không đọc staged diff; commit phụ thuộc file untracked; hoặc xử lý conflict bằng cách chọn một phía không hiểu nội dung.

J05 - CI/CD đủ dùng: test phải sống được trên máy sạch

Mục tiêu: Đưa automated checks lên CI, giữ evidence khi fail và chẩn đoán được khác biệt local/runner. Không yêu cầu triển khai hệ thống hay trở thành DevOps.

Mental model

CI là một thí nghiệm lặp lại trên môi trường sạch:

```
trigger
→ checkout exact commit
→ provision runtime/dependencies
→ start required services
→ wait for readiness
→ run deterministic checks
→ preserve report/trace/log
→ return trustworthy status
```

CD là bước đưa artifact qua môi trường/release sau khi các gate đạt. Trong lộ trình này, chỉ cần hiểu tests có thể làm deployment gate; không tự xây deployment pipeline.

Một pipeline xanh chỉ đáng tin khi:

- Dùng lockfile và clean install.
- Không phụ thuộc file/secret chỉ có trên laptop.
- Service readiness rõ, không ngủ cố định rồi hy vọng.
- Test isolated và time/locale/config được kiểm soát.
- Failure vẫn giữ trace/report/log.
- Secrets đi qua secret store và không bị print.

Worked example - Playwright CI tối thiểu

Ví dụ GitHub Actions; kiểm tra phiên bản action theo official docs lúc triển khai thật.

```
name: playwright-check

on:
  pull_request:
  push:
    branches: [main]

jobs:
  e2e:
    runs-on: ubuntu-latest
    timeout-minutes: 20

    steps:
      - name: Checkout exact commit
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Install dependencies from lockfile
        run: npm ci

      - name: Install Chromium and OS dependencies
        run: npx playwright install --with-deps chromium
```



```

- name: Run Chromium tests
  env:
    PLAYWRIGHT_BASE_URL: ${vars.PLAYWRIGHT_BASE_URL}
  run: npx playwright test --project=chromium

- name: Upload Playwright evidence
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: playwright-report
    path: |
      playwright-report/
      test-results/
    if-no-files-found: ignore
    retention-days: 7

```

Điểm cần giải thích:

- `npm ci` tái tạo dependency từ lockfile; nếu lockfile lệch, CI phải fail thay vì tự sửa.
- Cài đúng browser project cần chạy để giữ pipeline nhỏ.
- Base URL là configuration, không hard-code environment nội bộ.
- `if: always()` giữ evidence cả khi test đổ.
- Retry, worker count và timeout không được tăng để che flaky test.

Nếu test app được khởi động trong cùng repository, ưu tiên Playwright `webServer` hoặc readiness command có health check. Không dùng một `sleep 30` cố định làm điều kiện sẵn sàng.

Worked example - phân lớp pipeline Laundry

Không viết một job khổng lồ ngay. Tư duy theo feedback cost:

```

Fast checks: compile/lint/unit/service tests
→ Spring web/repository integration tests với PostgreSQL
→ browser smoke trên vertical critical path
→ report/artifact

```

Lệnh cụ thể phải lấy từ repository thật. Baseline tối thiểu cho backend là clean Maven test; browser smoke chỉ chạy sau khi Flyway dựng được disposable PostgreSQL và application readiness được chứng minh. Không commit database credential.

Prediction

Với mỗi failure, dự đoán pipeline stage và evidence cần đọc đầu tiên:

1. `package.json` đổi nhưng lockfile không đổi.
2. Browser binary chưa được cài.
3. Base URL thiếu.
4. App process đã start nhưng DB chưa healthy.
5. Test dùng machine local timezone, runner dùng UTC.
6. UI test fail nhưng artifact upload step dùng mặc định `if: success()`.

Completion

Artifact: `labs/ci/broken/playwright.yml`, `labs/ci/fixed/playwright.yml` và `tests/work/p10_ci.spec.ts`. Active reference baseline không chạy work specs chưa làm.

Cho một toy Playwright project:

1. Thêm workflow CI.
2. Chạy suite xanh trên pull request/branch.

3. Cố ý làm một assertion đổ.
4. Xác minh job đổ nhưng report/trace vẫn được upload.
5. Đọc trace để viết expected/actual và phân loại failure.
6. Khôi phục assertion, chạy lại và đạt xanh.

Không coi bước 6 là pass nếu không giải thích được lần đổ ở bước 4.

Sau attempt thật, enable ba contract tests và chạy `npm run lab:p10`; command còn `test.fixme` phải fail thay vì tạo tín hiệu xanh giả.

Independent

Tự thiết kế CI contract cho một project có app + test:

- Trigger nào cần chạy fast checks và browser smoke?
- Runtime/version nào phải pin?
- Dependency nào là service?
- Readiness được chứng minh thế nào?
- Configuration nào là variable, configuration nào là secret?
- Artifact nào cần giữ khi fail và bao lâu?
- Failure nào block integration?

Viết diagram pipeline và workflow skeleton; sau đó hiện thực một happy path nhỏ.

Failure injection

Chọn ba lỗi, mỗi lỗi phải được tạo và phục hồi riêng:

- Test phụ thuộc thứ tự.
- Test data cố định va chạm khi chạy hai workers.
- Timezone/locale khác local.
- Missing environment variable.
- App chưa ready nhưng test đã chạy.
- Artifact không được giữ khi job fail.

Không sửa bằng cách tăng sleep/retry trước khi có evidence nguyên nhân.

Transfer công ty

- Đọc pipeline hiện có và tuân theo owner/security policy; không tự thêm secret hay deployment.
- Khi CI đổ, bắt đầu từ exact commit, failed step, first meaningful error và artifact.
- Reproduce bằng cùng command/config trong khả năng được phép.
- Phân biệt product failure, test failure, infrastructure failure và known external outage.
- Một rerun xanh không tự chứng minh lỗi đã biến mất; nếu rerun thay đổi outcome, điều tra flakiness.

Transfer Laundry

CI target theo release gate:

- Maven tests từ clean checkout.
- Flyway dựng được empty PostgreSQL từ V1 trở đi.
- Hibernate validate schema; không `ddl-auto=create/update`.
- Focused repository tests dùng PostgreSQL cho query/time/locking.

- Browser smoke bao phủ public, Staff và Manager critical paths ở desktop width.
- Không raw ID, secret, persistent toast hoặc empty demo report trong evidence.

CI không thay thế traceability audit, rendered document QA hay defense rehearsal; nó chỉ là một lớp gate.

Gate J05

Bằng chứng	Điều kiện pass
Green clean-machine run	Checkout + clean install + test không phụ thuộc file local
Deliberate red run	Failure đúng dự đoán và report/trace/log vẫn còn
Triage note	Nêu exact commit, failed step, first error, classification và fix evidence
Configuration safety	Không hard-code secret; biết phân biệt variable/secret
Oral check	Giải thích CI vs CD, readiness vs sleep, retry vs flaky diagnosis

Chưa pass nếu: pipeline chỉ xanh trên rerun; artifact mất khi fail; dùng sleep/retry để che root cause; hoặc pipeline phụ thuộc thao tác tay không được ghi lại.

J06 - Docker operational literacy: vận hành dependency, không học lan sang DevOps

Mục tiêu: Hiểu image/container, khởi động dependency có cấu hình, kiểm tra health/log/port và xử lý các lỗi phổ biến. Chưa học Kubernetes, orchestration sâu hay tự tối ưu production image.

Mental model

```
Image      = mẫu filesystem + metadata bất biến
Container  = một process đang chạy từ image
Port map   = host_port → container_port
Environment = configuration truyền vào process
Volume     = dữ liệu sống ngoài writable layer của container
Network    = cách containers tìm/gọi nhau
Health     = process có thực sự sẵn sàng phục vụ hay chưa
Log        = evidence đầu tiên khi process không như mong đợi
```

“Container đang Up” không đồng nghĩa database đã ready. “Chạy được trên máy tôi” cũng không chứng minh app trong container truy cập được localhost của host/container khác.

Worked example - disposable PostgreSQL cho lab

Ví dụ dùng version lab đã pin. Khi chuyển sang Laundry, dùng version/configuration được repository quyết định.

```
services:
  postgres:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: laundry_dev
      POSTGRES_USER: laundry
      POSTGRES_PASSWORD: local-only-password
    ports:
      - "5433:5432"
    volumes:
      - laundry_pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U laundry -d laundry_dev"]
      interval: 3s
      timeout: 3s
      retries: 10

volumes:
  laundry_pg_data:
```

Các lệnh quan sát:

```
docker compose up -d
docker compose ps
docker compose logs postgres
docker compose exec postgres pg_isready -U laundry -d laundry_dev
docker compose stop
```

Host application kết nối localhost:5433; process trong cùng Compose network sẽ gọi service name postgres:5432. Mật khẩu ví dụ chỉ dành cho disposable local lab. Project thật dùng ignored local environment/secret mechanism phù hợp.

Không xóa volume khi chưa hiểu dữ liệu nào sẽ mất. Reset disposable lab database phải là thao tác rõ target và có thể tái tạo bằng Flyway.

Prediction

Trước khi chạy, dự đoán symptom và command kiểm tra đầu tiên:

1. Host port 5433 đã bị process khác dùng.
2. App trong container dùng `localhost:5432` để gọi Postgres container khác.
3. Password app khác password database.
4. Container Up nhưng healthcheck `starting/unhealthy`.
5. Stop rồi start container khi có named volume.
6. Xóa named volume của disposable DB rồi khởi động lại.

Completion

Đọc `labs/docker/README.md`, kiểm tra topology bằng `npm run lab:j06:check`, sau đó mới dùng `labs/docker/compose.yaml` cho PostgreSQL disposable.

1. Khởi động PostgreSQL lab.
2. Dùng `ps` và healthcheck chứng minh sẵn sàng.
3. Kết nối từ host bằng port đã map.
4. Dừng/start và dự đoán data còn hay mất.
5. Đọc log để tìm startup line và một lỗi authentication cố ý.
6. Ghi lại config contract: image version, database, user, host port, container port, volume, health command.

Independent

Nhận một Compose file chưa từng thấy. Không chạy ngay. Vẽ:

```
host
→ mapped ports
→ services
→ internal service-name connections
→ volumes
→ environment/secrets
→ health dependencies
```

Sau đó chạy, xác minh từng dự đoán và sửa một configuration mismatch bằng evidence.

Failure injection

Tạo ba lỗi lần lượt:

- Đổi host port sang một port đang bị chiếm.
- Đổi password phía app nhưng không đổi DB.
- Cho test chạy trước health readiness.

Với mỗi lỗi, ghi:

1. Symptom bên ngoài.
2. `ps/health/log` evidence.
3. Root cause.
4. Minimal fix.
5. Proof sau fix.

Không “sửa” bằng cách xóa toàn bộ Docker data khi chưa biết root cause.

Transfer công ty

- Dùng setup được team cung cấp; không đổi image/tag/port/volume chung tùy ý.
- Biết tìm app log, service log, health state và connection configuration.
- Phân biệt failure của product/test với dependency chưa ready.

- Không đưa production dump hoặc credential vào local container.
- Nếu project dùng containerized Playwright, hiểu mount/workdir/user/artifact path trước khi copy command.

Transfer Laundry

Docker chỉ phục vụ operational setup, trước hết là PostgreSQL local/test. Spring Boot app vẫn theo fixed Maven JAR stack; không tạo một frontend/API/container architecture thứ hai.

Evidence cần có:

- Disposable DB khởi động healthy.
- Flyway chạy từ V1 và Hibernate validate thành công.
- App dùng shop timezone/config đúng, không dựa machine default.
- Reset disposable DB có quy trình rõ; không hand-edit row để demo pass.
- Browser/service/repository tests trở đúng environment.

Gate J06

Bảng chứng	Điều kiện pass
Topology sketch	Phân biệt host/container port, service name, volume và health
Healthy dependency proof	Tự khởi động/stop/start và kết nối được mà không đoán
Three-failure log	Chẩn đoán port/auth/readiness bằng evidence, không xóa mù
Data-safety oral check	Giải thích khi nào data tồn tại, khi nào volume reset làm mất data
Scope discipline	Không mở rộng sang Kubernetes/cloud/Docker production optimization

Chưa pass nếu: chỉ thuộc `docker compose up`; coi Up là ready; nhầm `localhost`; hoặc xóa container/volume như phản xạ đầu tiên.

J07 - English for QA: truyền evidence không làm người khác đoán

Mục tiêu: Đọc requirement/tool output, hỏi rõ ambiguity, viết bug report/test note và giải thích failure ngắn gọn bằng English kỹ thuật.

Mental model

English trong QA không phải thi văn phạm. Nó là giao thức truyền:

```
context
→ observation
→ expected/actual delta
→ evidence
→ impact
→ next action/question
```

Thông điệp tốt ngắn nhưng không thiếu dữ kiện. Tránh từ mơ hồ: “not working”, “sometimes”, “wrong”, “please check” nếu không có điều kiện tái hiện/evidence.

Mẫu câu tối thiểu

- **Clarify:** “When the order is cancelled with a positive net-paid amount, should the customer view show refund due or remaining payment?”
- **Prediction:** “I expect the POST to be rejected before any payment row is inserted.”
- **Observation:** “The response is 302, but the redirected page shows no new transaction and the server log contains correlation ID ...”
- **Delta:** “Expected one COLLECTION row; actual result contains two rows with the same user action.”
- **Impact:** “This may overstate paid amount and allow pickup before the real balance reaches zero.”
- **Next step:** “I can reproduce it with two concurrent submissions; shall I add the trace and timestamps to the ticket?”

Worked example - bug report mô phỏng

Đây là lỗi được cài trong lab, không phải claim về implementation thật.

```
Title
[Pickup] Order can be completed while an issue is OPEN

Environment
Local disposable environment, build 8f31c2a, Chromium, 13 Aug 2026

Preconditions
- Staff is authenticated.
- Order LD-20260813-000321 is READY_FOR_PICKUP.
- Remaining amount is 0 VND.
- The order has one OPEN issue.

Steps
1. Open the order detail page.
2. Submit "Complete pickup".
3. Reopen the order detail and status history.

Expected
The command is rejected. The order remains READY_FOR_PICKUP and no COMPLETED history row is created.

Actual
The order changes to COMPLETED and a COMPLETED history row is created.

Evidence
```

```
- Playwright trace: <sanitized artifact name>
- Screenshot: <sanitized artifact name>
- Correlation ID: <id>
```

Impact

Staff can hand over an order before its operational issue is resolved.

Reproducibility

3/3 in the disposable local environment.

Report không suy đoán root cause nếu chưa có evidence. “Service forgot the guard” chỉ nên viết sau khi đã trace code/log hoặc xác nhận với developer.

Prediction

Đọc report sau và liệt kê ít nhất năm câu reviewer buộc phải hỏi:

```
Login is broken. It sometimes redirects wrong. Please check ASAP.
```

Sau đó viết lại thành report có environment, account state, exact steps, expected, actual, frequency và evidence.

Completion

Điền `labs/j07/FAILURE_REPORT.md` bằng một failure thật từ J02/J05. Clarification artifact của independent task là `labs/j07/REQUIREMENT_QUESTION.md`.

Điền template từ một deliberate failure đã làm ở J02/J05:

```
Summary:
Context/environment:
Preconditions:
Steps/command:
Expected:
Actual:
First meaningful error:
Evidence/artifact:
Classification hypothesis:
Impact:
Next check:
```

Giới hạn 180 từ, nhưng không bỏ exact command/route/status cần thiết.

Independent

Thực hiện ba artifact:

1. Một clarification question cho requirement có ambiguity.
2. Một 90-second spoken failure walkthrough: context → expected → actual → evidence → hypothesis → next check.
3. Một test review comment giải thích vì sao status-only assertion chưa đủ.

Thu âm/phát lại phần nói. Nếu chính người học nghe lại không xác định được “lỗi gì, ở đâu, bằng chứng nào”, làm lại.

Failure injection

Lấy bug report tốt rồi lần lượt bỏ:

- Preconditions.
- Exact expected result.
- Environment/build.
- Evidence identifier.

Với mỗi phiên bản, đóng vai reviewer và ghi câu hỏi phát sinh. Mục tiêu là hiểu chi phí giao tiếp của dữ kiện bị thiếu, không phải viết report dài nhất.

Transfer công ty

Luyện một nhịp ngắn hằng ngày:

- Đọc một requirement/test failure tiếng Anh và tóm tắt ba dòng.
- Viết một prediction trước khi test.
- Khi report failure, dùng exact noun/verb/status thay cho “it/thing/wrong”.
- Trong standup: done, evidence, blocker, next action; không kể toàn bộ timeline.
- Khi chưa hiểu domain term, hỏi định nghĩa/example thay vì giả vờ hiểu.

Không đưa nội dung công ty thật vào artifact công khai hoặc công cụ không được phép.

Transfer Laundry

Người học phải giải thích được một vertical flow bằng English đơn giản:

```
The browser submits ...
Spring Security checks ...
The controller binds ...
The service enforces ... inside one transaction.
The repository reads/writes ...
The observable result is ...
The test proves ...
```

Tập trung vào UC-20/24/25 hoặc một UC mình sở hữu. Không cần accent hoàn hảo; cần thuật ngữ nhất quán, flow đúng và trả lời được một “why” cùng một failure variant.

Gate J07

Bằng chứng	Điều kiện pass
Bug report ≤ 220 từ	Người khác có thể tái hiện; expected/actual/evidence/impact rõ
Clarification question	Chỉ ra đúng ambiguity và đưa ví dụ boundary cụ thể
3-minute technical walkthrough	Flow đúng, không đọc script toàn bộ, trả lời được hai follow-up
Failure summary	Nêu first meaningful error và hypothesis như hypothesis, không biến thành fact
Vocabulary consistency	Dùng đúng request/response/status/session/transaction/oracle/fixture/trace

Chưa pass nếu: report chỉ có ảnh; dùng “not working”; trộn observation với phỏng đoán; hoặc không nói được expected behavior dựa trên rule nào.

4. Runway integration gate - kết thúc ba tuần

Runway gate dùng practice app/disposable environment, không cần chờ Laundry hoàn thiện.

Nhiệm vụ tích hợp

Chọn một business flow nhỏ có UI và API lab support:

1. Phân tích requirement thành risk map và decision/state tests.
2. Tạo test data isolated.
3. Viết một API test và một Playwright UI test, tránh duplicate coverage vô lý.
4. Dùng SQL read-only để xác minh một invariant thực sự cần DB oracle.
5. Commit thành 2-3 commits review được.
6. Chạy trên CI từ clean checkout, giữ artifact.
7. Khởi động dependency bằng Docker và chứng minh health/readiness.
8. Cài một failure, triage bằng trace/log/query.
9. Viết failure summary bằng English và giải thích oral 3 phút.

Rubric

Dimension	Fail	Pass	Strong
Test design	Script theo thao tác, không risk/oracle	Test truy về rule, có negative/boundary	Chọn đúng level, giảm duplicate E2E
Automation	Phụ thuộc order/shared data	Isolated, deterministic, web-first	Failure evidence và teardown/data strategy rõ
HTTP/API	Chỉ status	Contract + business invariant	Nối được UI request với server-side failure layer
SQL	Query copy, không hiểu join	Read-only oracle đúng	Chứng minh cardinality/time boundary/no-partial-write
Git	Một commit lộn xộn	Atomic commits, clean diff	Conflict/recovery được giải thích
CI	Chỉ local xanh	Clean runner xanh, artifact khi đỏ	Phân loại failure và loại flake root cause
Docker	Chỉ up	Hiểu port/env/health/log	Chẩn đoán ba failure không reset mù
English	Mơ hồ	Reproducible, evidence-led	Ngắn, đúng thuật ngữ, trả lời follow-up

Pass yêu cầu tất cả dimensions đạt mức **Pass**. Mức Strong là mục tiêu trong hai tháng SWP, không phải điều kiện để bắt đầu dự án.

5. SWP integrated technical gate - cuối hai tháng

Nhiệm vụ

Chọn một canonical Laundry vertical slice có đủ UI, business rule và persistence, ưu tiên flow order/payment/pickup:

```
login
→ create/select customer
→ create mixed KG + ITEM order
→ verify snapshots/rounded total
→ record collection
→ transition/process issue
→ ready/pickup guard
→ verify history/ledger/report or public tracking
```

Không nhất thiết một người implement toàn hệ thống, nhưng người học phải tự trace và test được slice mình trình bày.

Evidence pack kỹ thuật

- UC/BR risk and coverage map.
- Focused service/controller/repository proofs theo đúng test pyramid.
- Một Playwright critical-path test với semantic locators, web-first assertions, isolation và trace-on-failure.
- HTTP contract map cho form/session/CSRF/PRG.
- SQL oracle cho snapshot/history/ledger/report phù hợp.
- Git history review được.
- CI clean run và deliberate-failure triage artifact.
- Docker dependency topology/health proof.
- English technical walkthrough và một failure variant.

Final oral gate

Không nhìn script, trả lời:

1. Risk quan trọng nhất của slice là gì?
2. Vì sao test này ở service/API/UI/repository level?
3. Browser gửi request nào và security/binding xử lý ra sao?
4. Business invariant nằm ở đâu và transaction boundary nào bảo vệ nó?
5. SQL evidence nào chứng minh success hoặc no-partial-write?
6. Nếu test chỉ fail trên CI, đọc evidence theo thứ tự nào?
7. Một thay đổi ngẫu nhiên nào sẽ làm coverage hiện tại thiếu?

Sau oral gate, mentor đưa một **random change/no-agent drill** nhỏ. Ví dụ: đổi collection boundary, thêm một invalid state, đổi locator text nhưng giữ accessible role, hoặc làm một test data collision. Chỉ đạt mastery khi người học tự dự đoán, sửa và giải thích được; AI-generated green code không được tính là bằng chứng hiểu.

6. Recovery khi lệch nhịp

Nếu nghỉ học hoặc fail gate:

- Không quay lại đọc toàn bộ module.
- Mở evidence gần nhất và chọn failure nhỏ nhất chưa giải thích được.

- Làm lại theo thứ tự: prediction → focused run → evidence → explanation.
- Nếu fail cùng nguyên nhân hai lần, dùng hint bậc 1; lần ba mới dùng worked patch/hint sâu.
- Không chuyển sang công nghệ mới để né nền tảng đang yếu.

Ưu tiên phục hồi:

```
testing/oracle  
→ JS/TS/Playwright core  
→ HTTP/API  
→ SQL/Git  
→ CI  
→ Docker
```

English QA tiếp tục 10-15 phút trong mọi ngày học vì nó dùng chính failure/evidence đang có, không cần một “buổi học tiếng Anh” tách rời.

PHẦN III - Playwright từ bài test đầu đến suite có thể tin

Phần này giả định m đã pass cả JS08 và TS04: đọc được function, object, destructuring, Promise, `await`, type annotation và import. Nếu syntax vẫn chặn việc giải thích test, quay lại đứng foundation gate; không copy thêm Playwright code để che lỗ hổng.

Mỗi chapter có một command `npm run lab:pXX`. Starter dùng `test.fixme(...)` để không làm full suite đổ trước khi đến bài, nhưng focused command cố ý thoát non-zero khi marker còn tồn tại. Sau attempt thật, đổi đúng marker của bài thành `test(...)`; test bị skip không bao giờ được tính là completion.

Playwright không chỉ là API click browser. Năng lực thật gồm bốn lớp:

```
testing intent
-> browser/web mental model
-> Playwright synchronization + isolation
-> evidence-driven debug
```

Mỗi test phải trả lời được: risk nào đang được kiểm soát, state đầu vào thuộc về ai, action nào thay đổi hệ thống, oracle nào chứng minh outcome và failure evidence nằm ở đâu.

P01 - Anatomy của một test và oracle

Problem

Test sau chạy xanh:

```
test('create order', async ({ page }) => {
  await page.goto('/orders/new');
  await page.getByRole('button', { name: 'Tạo đơn' }).click();
});
```

Nó chỉ chứng minh browser đã thử click. Nó chưa chứng minh một đơn hợp lệ được tạo, URL đúng, dữ liệu persist, total đúng hay user thấy outcome.

Mental model

Test runner đăng ký một test case. Khi chạy, runner tạo fixture, gọi async callback và quyết định pass/fail từ assertion/rejection.

```
Arrange: tạo state và dữ liệu đầu vào có chủ
Act:      một hành vi hoặc command chính
Assert:   oracle gần outcome người dùng nhất
```

Một test có thể có nhiều check liên quan, nhưng tên test chỉ nên mô tả một behavior/risk. Nếu một assertion fail mà tên test không giúp biết contract nào hỏng, scope đang quá mơ hồ.

Worked example

```
import { test, expect } from '@playwright/test';

test('visitor sees one active KG service with its VND price', async ({ page }) => {
  // Arrange is server seed plus navigation.
  await page.goto('/services');

  // Find the business object, not a screen coordinate.
  const row = page
    .getByRole('row')
    .filter({ hasText: 'Giặt sấy quần áo' });

  // Oracles prove the user-facing contract.
  await expect(row).toContainText('KG');
  await expect(row).toContainText('25.000 đ');
});
```

Prediction

Trước khi chạy:

1. Nếu heading đổi nhưng row đúng, test có fail không?
2. Nếu service bị ẩn khỏi DOM, assertion nào báo lỗi đầu tiên?
3. Nếu giá hiển thị 25000, risk nào bị bắt?

Completion task

Mở `tests/work/p01_anatomy.spec.ts`, đổi hai marker của bài từ `test.fixme` thành `test`, rồi chạy `npm run lab:p01`. Điền ba chỗ:

- navigation;
- locator cho một service row theo tên;
- assertion unit và price.

Không dùng CSS, XPath, `nth()` hoặc snapshot cả trang.

Independent task

Viết test cho LD-002 trên `/orders?order=desc`:

- action “Chi tiết” xuất hiện ở nhiều row;
- test phải mở đúng order;
- oracle là heading business code ở trang detail.

Chỉ nhận contract này, không nhìn reference test.

Failure injection

Đổi expected code thành LD-003. Trước khi chạy, dự đoán call log và first meaningful error. Sau đó phân biệt:

- locator không tìm thấy;
- locator match sai object;
- action thành công nhưng assertion sai.

Transfer

Công ty

Chọn một spec được phép đọc. Ghi bốn dòng, không chép source ra ngoài:

```
Risk/behavior:
Owned starting state:
Main action:
Observable oracle:
```

Nếu không tìm thấy oracle, đó là câu hỏi review hợp lệ.

Laundry

Với UC-05, oracle là active service name/unit/price. Với UC-09, oracle không chỉ là redirect: detail phải hiển thị order code, snapshots, rounded line amounts và total do server tính. Không automate UC-09 trước khi vertical slice tồn tại; viết test condition trước.

Gate P01

Từ một manual checklist, tự viết một test có tên theo behavior, arrange có data owner, một act chính và oracle user-visible. Gây một assertion fail và chỉ đúng first meaningful line.

P02 - Locator là query recipe, không phải tọa độ

Problem

Selector sau rất cụ thể nhưng dễ gãy:

```
page.locator('main > div:nth-child(3) table tr:nth-child(2) td:last-child a')
```

Nó mô tả layout hiện tại chứ không mô tả user intent. Thêm một row hoặc wrapper có thể làm test click nhầm dù feature vẫn đúng.

Mental model

Locator giữ **công thức tìm element tại thời điểm action/assertion**, không giữ sẵn một DOM node. Playwright dùng locator làm trung tâm cho auto-wait và retry. Ưu tiên locator theo cách người dùng/assistive technology nhận biết UI:

1. role + accessible name;
2. label cho form control;
3. text khi text chính là business identity;
4. test id khi semantics không đủ và team có contract;
5. CSS nhỏ, có chủ đích cho cấu trúc kỹ thuật không biểu diễn được bằng semantics.

Official reference: [Playwright Locators](#).

Accessible name vừa đủ

```
<label for="phone">Số điện thoại</label>
<input id="phone" name="phone">
<button type="submit">Lưu</button>

page.getByLabel('Số điện thoại');
page.getByRole('button', { name: 'Lưu' });
```

name của role không nhất thiết là thuộc tính HTML name. Nó là accessible name được tính từ text, label, aria-label, aria-labelledby và các rule accessibility.

Strictness

Action cần một target duy nhất. Nếu `getByRole('link', { name: 'Chi tiết' })` match ba link, Playwright báo strictness thay vì tự chọn đại. Đây là feedback rằng intent chưa đủ rõ.

Scope business object trước, action sau:

```
const row = page.getByRole('row').filter({ hasText: 'LD-002' });
await row.getByRole('link', { name: 'Chi tiết' }).click();
```

Worked example

Cách	Test đang nói gì	Chịu được
<code>nth(1)</code>	lấy item DOM thứ hai	gần như không chịu reorder/filter
<code>.first()</code>	lấy match đầu	che ambiguity
row có LD-002 rồi link	action thuộc business order nào	reorder, thêm row, filter
test id order-LD-002	UI có contract kỹ thuật	layout đổi, nhưng ít phản hồi semantics

Prediction

Trước khi chạy `/orders?order=desc`, dự đoán:

- `nth(1)` sẽ chọn code nào;
- row-scope chọn code nào;
- nếu hai row cùng chứa chuỗi `LD-002`, lỗi gì nên xảy ra.

Completion task

Làm `tests/work/p02_locators.spec.ts`; sau attempt, enable đúng hai test và chạy `npm run lab:p02`:

1. tìm row service theo name;
2. assert unit và price trong row;
3. đảo order rows;
4. mở đúng `LD-002`.

Independent task

Tự tạo một HTML snippet hoặc dùng app được phép có:

- hai button cùng text ở hai section;
- một field có label;
- một status message.

Viết locator chịu được việc đổi thứ tự section.

Failure injection

Xóa `<label>` khỏi sandbox copy hoặc đổi accessible name. Không lập tức chuyển sang CSS. Ghi:

- đây là test locator sai hay UI accessibility contract yếu;
- user nào bị ảnh hưởng;
- fix UI hay fix test là quyết định đúng trong context nào.

Transfer

Order detail có nhiều financial/operational action. Scope bằng section và business effect, không dùng button index. List/search dùng `order_code`, customer snapshot và status label; numeric DB ID không phải user-facing identity.

Gate P02

Sửa năm brittle selectors. Mỗi selector phải kèm một câu: “UI thay đổi nào test vẫn sống?” và một case cố ý gây strictness.

P03 - Actionability, auto-wait và web-first assertion

Problem

Khi test flaky, phần xạ thường là:

```
await page.waitForTimeout(3000);
```

Nếu app nhanh, test lãng phí ba giây. Nếu app chậm hơn, test vẫn fail. Nếu locator sai hoặc overlay vĩnh viễn, sleep không chữa nguyên nhân.

Mental model

Trước action như click, Playwright chờ các điều kiện actionability phù hợp: locator resolve đúng số target, visible, stable, receives events và enabled khi cần. Assertion async như `toBeVisible` hoặc `toHaveText` lặp lại query/check cho tới khi condition đúng hoặc timeout.

Hai loại wait trả lời hai câu khác nhau:

```
actionability: control đã có thể thao tác chưa?
assertion retry: outcome người dùng đã đạt chưa?
```

Official references: [Auto-waiting](#) và [Assertions](#).

Worked example

```
await page.goto('/slow-form');
await page.getByLabel('Số điện thoại').fill('0912345678');
await page.getByRole('button', { name: 'Lưu' }).click();

await expect(page).toHaveURL(/saved=1/);
await expect(page.getByRole('status')).toHaveText('Đã lưu 0912345678');
```

Overlay ban đầu chặn click. Playwright đợi button nhận event. Sau POST/redirect, assertion đợi user-visible message. Không có timeout cứng tùy tiện.

Locator re-resolution

Ở `/rerender`, button DOM cũ bị thay bằng node mới. Locator có thể resolve lại:

```
const save = page.getByRole('button', { name: 'Lưu' });
await expect(save).toHaveAttribute('data-generation', '2');
await save.click();
await expect(page.getByRole('status')).toHaveText('Đã lưu bởi node mới');
```

Đừng cache element handle khi không có lý do; handle có thể detached sau re-render.

Prediction

Với từng tình huống, chọn evidence chứ chưa chọn fix:

Symptom	Hypothesis A	Hypothesis B	Evidence phân biệt
click timeout	overlay chặn	locator match hidden duplicate	call log + trace snapshot
click pass, message timeout	backend fail	UI không render outcome	Network + console + DOM
local xanh, CI đỏ	environment/data	race	trace, worker/retry pattern, logs

Completion task

Làm `tests/work/p03_sync.spec.ts`: xóa `sleep`, dùng `actionability` và một `user-visible assertion`. Enable ba test sau attempt và chạy `npm run lab:p03`.

Independent task

Tạo một route/page nhỏ có button bị disable 400-800ms rồi enabled. Test không được biết delay cụ thể; phải đợi condition/outcome.

Failure injection

Ba lần riêng biệt:

1. đổi locator thành target không tồn tại;
2. giữ overlay vĩnh viễn;
3. server trả 200 nhưng client không render success.

Ghi call log/trace khác nhau thế nào. Không dùng cùng một “tăng timeout” cho ba root cause.

Transfer

POST thành công dùng PRG và one-request flash. Test cần quan sát POST/302/GET khi debug, nhưng oracle chính vẫn là detail/status/row người dùng thấy. Response 200 không chứng minh business outcome nếu view sai.

Gate P03

Sửa một race mà không dùng `waitForTimeout`, giải thích `actionability` khác `assertion retry`, và chẩn đoán được một permanent blocker bằng trace.

P04 - Form, table, file, dialog và browser context

Problem

Biết `click()` chưa đủ để automate flow thật. Form có label, validation, select/checkbox, submit; table có nhiều business row; browser có upload/download, popup và dialog. Nhưng học mọi API theo danh sách sẽ nhanh quên vì không gắn risk.

Mental model

Mỗi interaction có ba lớp:

```
semantic target -> browser action -> observable outcome
```

Chọn action gần hành vi user và để Playwright phát event đúng cách:

- `fill()` thay giá trị text theo contract;
- `pressSequentially()` chỉ khi app phụ thuộc từng key event và m có evidence;
- `selectOption()` cho native select;
- `check()/uncheck()` cho checkbox/radio;
- `setInputFiles()` cho file input;
- chờ download/popup event khi action tạo object mới;
- đăng ký dialog handler trước action nếu dialog được mong đợi.

Worked example

Form

```
test('invalid phone is rejected beside the field', async ({ page }) => {
  await page.goto('/customers/new');
  await page.getByLabel('Họ tên').fill('Nguyễn An');
  await page.getByLabel('Số điện thoại').fill('abc');
  await page.getByRole('button', { name: 'Lưu khách hàng' }).click();

  await expect(page.getByText('Số điện thoại không hợp lệ')).toBeVisible();
  await expect(page.getByLabel('Họ tên')).toHaveValue('Nguyễn An');
});
```

Oracle không chỉ kiểm tra error; safe input phải được giữ để user sửa. Nếu app dùng browser native validation, xác định request có thực sự được gửi không trước khi kết luận server validation.

Table

```
const row = page
  .getByRole('row')
  .filter({ hasText: 'LD-20260810-000123' });

await expect(row).toContainText('Sẵn sàng trả');
await row.getByRole('link', { name: 'Chi tiết' }).click();
```

Table test theo business row. Không assert toàn bộ table text nếu risk chỉ là filter/status của một row; assertion quá rộng tạo noise.

File và download

```
const downloadPromise = page.waitForEvent('download');
await page.getByRole('link', { name: 'Xuất báo cáo' }).click();
const download = await downloadPromise;
expect(download.suggestedFilename()).toMatch(/revenue.*\.csv/);
```

Promise chờ event phải được tạo trước click để không bỏ lỡ event nhanh.

Với upload:

```
await page.getByLabel('Tập đính kèm').setInputFiles('fixtures/sample.txt');
await expect(page.getByText('sample.txt')).toBeVisible();
```

Đừng automate upload nếu product không có risk đó. Canonical Laundry không cần upload chỉ để luyện tool; dùng sandbox.

Prediction

Trước khi chạy code, ghi thứ tự cho hai case:

1. tạo download promise trước click so với sau click: case nào có thể bỏ lỡ event;
2. tìm “Chi tiết” bằng index sau khi đảo table so với scope row bằng business code: mỗi cách mở order nào.

Với form invalid, dự đoán request có được gửi và safe input nào phải còn lại. Sau run, dùng Network/DOM evidence để sửa prediction, không sửa expected theo actual một cách máy móc.

Completion task

Làm `tests/work/p04_controls.spec.ts`; enable hai test sau attempt và chạy `npm run lab:p04`:

- fill phone;
- chọn unit KG;
- check acknowledgement;
- set một file nhỏ;
- assert summary theo label/status.

Independent task

Viết hai test:

1. filter order table và mở đúng row khi DOM đảo thứ tự;
2. một invalid form giữ safe input và không tạo outcome success.

Failure injection

- dùng `pressSequentially` không cần thiết và đo thời gian;
- tạo download promise sau click;
- dùng file path phụ thuộc current working directory;
- scope action bằng `nth()` rồi đảo row.

Mỗi failure ghi root cause thuộc event order, environment hay business selection.

Transfer

Công ty

Chỉ học iframe, popup, download hoặc mobile emulation khi task thật có chúng. Không biến roadmap thành bingo API.

Laundry

Ưu tiên form/customer/order/table/status/print flows. Receipt có thể kiểm tra print-friendly content/route; không cần pixel-perfect PDF nếu app chỉ render HTML print view.

Gate P04

Tự automate một form gồm happy + invalid case, một table row có duplicate action và một event-based interaction; không sleep, không index coupling.

P05 - Isolation, hooks và data ownership

Problem

Hai test đều pass khi chạy riêng nhưng fail khi chạy cả suite:

Test A đổi phone của customer mặc định.
Test B giả định phone vẫn là giá trị seed.

Browser context mới chỉ cô lập cookie/local storage/cache của browser. Nó không reset PostgreSQL hoặc state backend.

Mental model

Mọi state phải có owner và lifecycle:

State	Ví dụ	Owner/reset thường dùng
browser	cookie, localStorage	context/fixture per test
server	session, in-memory cache	app contract/test endpoint có kiểm soát
database	customer/order/payment	unique data + cleanup, transaction/seed reset
external	email/provider/queue	fake/sandbox/controlled environment

Test isolation không đồng nghĩa reset toàn hệ thống sau mỗi test. Chọn strategy theo contract:

- unique identity per test;
- API/helper tạo và cleanup resource do test sở hữu;
- deterministic DB seed/reset trong test environment;
- serial execution chỉ khi business scenario thật sự có chuỗi state và trade-off được ghi rõ.

Worked example

```
test('updates only its owned customer state', async ({ page, request }, testInfo) => {
  const owner = `customer-${testInfo.workerIndex}-${testInfo.title.replaceAll(/\W/g, '-')}`;
  await request.post(`/test-support/reset?owner=${encodeURIComponent(owner)}`);

  await page.goto(`/customer-state?owner=${encodeURIComponent(owner)}`);
  await page.getByLabel('Số điện thoại mới').fill('0987654321');
  await page.getByRole('button', { name: 'Cập nhật' }).click();

  await expect(page.getByTestId('current-phone')).toHaveText('0987654321');
});
```

Identity không chỉ dùng timestamp chung chung. Nó phải tránh collision giữa worker/retry và vẫn cho phép tìm cleanup target.

Hook hay fixture?

`beforeEach` phù hợp setup đơn giản cho nhóm test. Custom fixture phù hợp khi dependency có contract, lifecycle, reuse hoặc cleanup rõ.

```
type OwnedFixtures = { owner: string };

export const test = base.extend<OwnedFixtures>({
  owner: async ({ request }, use, testInfo) => {
    const owner = `w${testInfo.workerIndex}-${testInfo.testId}`;
    await request.post(`/test-support/reset?owner=${encodeURIComponent(owner)}`);
    await use(owner);
    await request.post(`/test-support/reset?owner=${encodeURIComponent(owner)}`);
  }
});
```

```
    },  
  });
```

Fixture này có setup -> use -> teardown. Helper bình thường chỉ là function; không cần biến mọi helper thành fixture.

Prediction

Với mỗi state, trả lời trước khi code:

1. Ai tạo?
2. Identity nào phân biệt?
3. Test khác có thể nhìn/sửa không?
4. Retry có tạo trùng không?
5. Cleanup chạy khi assertion fail không?

Completion task

`tests/work/p05_isolation.spec.ts` chứa hai test executable cố ý dùng cùng owner. Enable đúng hai test đó, chạy trên server state mới: chạy riêng để lấy baseline rồi chạy chung để thấy pollution. Chỉ sau evidence mới sửa bằng owner riêng + cleanup. Reference ownership proof nằm trong

`tests/reference/foundation/p05_isolation.reference.spec.ts` và chỉ mở ở H5.

Focused command sau khi enable: `npm run lab:p05`. Command phải đỗ ở baseline combined run, rồi xanh sau repair; một lần chạy riêng xanh không đạt gate.

Independent task

Thiết kế data contract cho ba test order:

- create order;
- search order;
- cancel order.

Không cho test B phụ thuộc order do test A tạo, trừ khi m chủ đích viết một scenario test duy nhất có toàn flow.

Failure injection

- dùng `Date.now()` nhưng hai worker mock cùng clock;
- cleanup toàn bộ DB thay vì resource owned;
- lưu page/context vào biến global;
- retry tạo duplicate business code.

Transfer

Canonical Laundry có state DB thật. Trong test suite, không copy production endpoint `/test-support/reset` vào app chính. Dùng test profile, fixture/migration/seed có kiểm soát hoặc unique business data; khóa endpoint test-only nếu thật sự cần. Payment/status history là immutable business history, cleanup không được trở thành production feature.

Gate P05

Hai test chạy đơn, chạy chung, đảo thứ tự và repeat vẫn xanh. M giải thích browser isolation khác DB isolation và vẽ data owner/cleanup.

P06 - Authentication, authorization, session và CSRF

Problem

“Login thành công” không chứng minh authorization đúng. Một app có thể hiện menu đúng nhưng server vẫn cho STAFF gọi route Manager. Hoặc test lưu auth state nhưng bỏ qua forced-password/session rule.

Mental model

Phân biệt bốn câu hỏi:

```
Authentication: m là ai?
Authorization:  m được làm gì?
Session:       server nhớ identity giữa request bằng contract nào?
CSRF:          request thay đổi state có đến từ browser session hợp lệ không?
```

UI hide button chỉ là UX. Server route và service guard mới là authority.

Prediction

Actor/state	Authn request	Authn result	POST Manager có CSRF	POST thiếu CSRF
Anonymous	302 login	302 login	không có session	403/redirect theo stack
STAFF	200	403	403	403
MANAGER	200	200	success/redirect	403

Expected status phải theo app contract, không học thuộc bảng chung cho mọi framework.

Trước khi chạy, điền cả matrix và dự đoán redirect chain, status cuối, button visibility. Sau đó đánh dấu oracle nào chỉ chứng minh UI và oracle nào chạm server route.

Worked example

```
async function login(page: Page, username: 'staff' | 'manager') {
  await page.goto('/login');
  await page.getByLabel('Tên đăng nhập').fill(username);
  await page.getByLabel('Mật khẩu').fill('lab');
  await page.getByRole('button', { name: 'Đăng nhập' }).click();
  await expect(page.getByRole('heading', { name: 'Bảng điều khiển' })).toBeVisible();
}
```

Helper không được nuốt assertion/diagnostic quan trọng. Type union ngăn typo actor trong compile-time, nhưng runtime server vẫn phải kiểm tra credentials.

Storage state

Với suite lớn, setup project có thể login một lần cho mỗi role và lưu browser state vào ignored output. Không commit cookie/token thật.

```
await page.context().storageState({ path: authFile });
```

Storage state là snapshot browser-side. Nó không đảm bảo server session chưa expire, account còn active hoặc password-reset flag không đổi. Test nhảy auth vẫn cần negative/fresh-session coverage.

Official reference: [Playwright Authentication](#).

Completion task

Tách comment auth matrix trong v3 thành test độc lập tại `tests/work/p06_auth.spec.ts`:

1. anonymous redirect;
2. STAFF forbidden;
3. MANAGER allowed;
4. missing CSRF rejected.

Enable bốn test sau attempt và chạy `npm run lab:p06`.

Independent task

Viết một config có setup project cho STAFF và MANAGER, hai dependent projects dùng state tương ứng. File state phải nằm trong ignored directory.

Failure injection

- dùng MANAGER state cho STAFF test;
- commit auth file giả rồi chạy secret scan;
- reuse expired state;
- chỉ kiểm tra menu ẩn mà không gọi route;
- POST bằng request context thiếu cookie hoặc CSRF.

Transfer

Canonical rules phải được test ở nhiều tầng:

- UC-01/02/03/04 và BR-01..05;
- MANAGER kế thừa Staff operations;
- temporary-password session chỉ vào change-password/logout/static;
- session ID rotate sau login/password change;
- mọi POST có CSRF;
- refund/manager routes trả 403 cho STAFF;
- actor ID lấy từ session, không tin form.

Playwright chứng minh behavior user-visible và route boundary; MockMvc/service tests chứng minh sâu hơn. Không bắt E2E gánh hết security matrix.

Gate P06

Tự xây role matrix, automate ít nhất anonymous/STAFF/MANAGER/CSRF, giải thích storage state không phải authorization proof và tìm được một wrong-role setup bug.

P07 - Config, project, fixture và suite architecture

Problem

Suite năm test chưa cần framework lớn. Nhưng suite 30 test copy login, test data và selectors cũng không bền. Hai lỗi đối nghịch là không tách gì hoặc tạo Page Object cho mọi page từ ngày đầu.

Mental model

Tách abstraction khi có một trong ba tín hiệu:

1. duplication có cùng meaning và thay đổi cùng nhau;
2. lifecycle/dependency cần fixture quản lý;
3. domain action cần tên rõ hơn low-level steps.

Không tách chỉ vì code dài ba dòng.

config	policy toàn run: testDir, projects, retries, reporter, webServer
fixture	dependency + setup/use/teardown lifecycle
helper	function thuần hoặc action nhỏ
data builder	tạo data shape có default/override
page object	interface theo user task khi page behavior đủ ổn định

Worked example

Config

```
export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: Boolean(process.env.CI),
  retries: process.env.CI ? 1 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: [['list'], ['html', { open: 'never' }]],
  use: {
    baseUrl: process.env.BASE_URL ?? 'http://127.0.0.1:4173',
    trace: 'retain-on-failure',
    screenshot: 'only-on-failure',
  },
});
```

retries=1 không phải thuốc chữa flake. Nó cho thêm evidence và giảm noise có kiểm soát trong CI; root cause vẫn phải được xử lý.

Task helper

```
async function openOrder(page: Page, orderCode: string) {
  await page.goto('/orders');
  const row = page.getByRole('row').filter({ hasText: orderCode });
  await row.getByRole('link', { name: 'Chi tiết' }).click();
  await expect(page.getByRole('heading', { name: `Đơn ${orderCode}` })).toBeVisible();
}
```

Helper giữ oracle để không silently click nhầm. Nếu behavior thay đổi, một contract có chỗ sửa rõ.

Data builder

```
type CustomerInput = { fullName: string; phone: string; note?: string };

function customerInput(overrides: Partial<CustomerInput> = {}): CustomerInput {
  return {
    fullName: 'Nguyễn An',
    phone: '0912345678',
    ...overrides,
  };
}
```

Builder không sinh magic random data không debug được. Override phải nhìn ra case intent.

Prediction

Cho từng abstraction định thêm, ghi trước: duplication/lifecycle nào chứng minh nó cần tồn tại, ai gọi nó và failure sẽ hiện ở đâu. Nếu không chỉ được evidence, giữ code tại chỗ. Dự đoán thêm `test.only` sẽ làm local run và CI forbidOnly khác nhau thế nào.

Completion task

Sau khi có tối thiểu tám test, mở `tests/work/p07_refactor/`. Tìm đúng ba duplication có meaning, tách helper/builder/fixture phù hợp; giữ assertion gần behavior. Enable ba starter tests theo thứ tự ghi trong file và chạy `npm run lab:p07` sau mỗi refactor nhỏ.

Independent task

Thiết kế folder cho portfolio suite khoảng 25 test. Giải thích vì sao mỗi abstraction tồn tại. Không dùng `utils.ts` chứa mọi thứ.

Failure injection

- mega Page Object vừa query DB vừa click UI vừa assert mọi thứ;
- fixture auto chạy nặng cho mọi test;
- environment variable thiếu nhưng config silently dùng production URL;
- helper catch error rồi return `false`, khiến test false green;
- `test.only` lọt CI.

Transfer

Công ty

Đọc convention repo hiện tại trước. Không refactor architecture trong task đầu. Sửa tối thiểu, hỏi reviewer về fixture/data/selector contract và lưu diff nhỏ.

Laundry

Không tạo Playwright Page Object mirror toàn bộ 16 views. Bắt đầu bằng domain task helper ở flow lặp thật: login, select owned customer, open order by business code. Business rules vẫn nằm trong Spring service, không copy vào test helper làm source of truth.

Gate P07

Từ suite có duplication thật, tách đúng một fixture, một data builder và tối đa hai task helpers; test behavior không đổi, chạy repeat/reorder được, m giải thích lifecycle.

P08 - APIRequestContext và UI-API hybrid

Boundary quan trọng

API testing là kỹ năng nghề nghiệp có giá trị, nhưng canonical Laundry là Spring MVC server-rendered, không có REST-first UI. Không dựng `/api/*` vào Laundry chỉ để tick box. P08 dùng API sandbox của lab hoặc API công ty được phép.

Mental model

UI test chứng minh browser/user flow. API test chứng minh HTTP contract nhanh và trực tiếp. Hybrid thường dùng API để setup/cleanup data rồi UI để chứng minh outcome, hoặc UI act rồi API/DB evidence để debug.

```
API arrange -> UI act/assert
UI act -> HTTP/API inspect -> UI assert
```

Đừng thay toàn bộ user flow bằng API rồi tuyên bố UI được test.

Official reference: [Playwright API testing](#).

Worked example

```
test('active services API returns stable business fields', async ({ request }) => {
  const response = await request.get('/api/services');
  expect(response.status()).toBe(200);
  expect(response.headers()['content-type']).toContain('application/json');

  const body: unknown = await response.json();
  const services = parseServices(body); // runtime validation from TS03
  expect(services).toEqual(
    expect.arrayContaining([
      expect.objectContaining({ code: 'GIAT_SAY_KG', unit: 'KG' }),
    ]),
  );
});
```

Type annotation không validate JSON runtime. `parseServices` phải narrow/validate `unknown` hoặc dùng schema library nếu project đã chọn.

Contract dimensions

Dimension	Example oracle
status	200/201/400/401/403/404/409 theo contract
headers	content type, location, cache, correlation ID
shape	required/optional fields and types
semantics	active only, correct calculation, authorization
side effect	resource actually created/changed once
privacy	fields/records not leaked
timing	only when explicit SLA/risk exists

Không assert mọi field trong mọi test. Chọn dimension theo risk.

Prediction

Không chạy request, điền matrix cho success, invalid phone, missing token và not-found: expected status, minimal body shape, side effect có/không và cleanup owner nào. Với một body sai shape nhưng status 200, dự đoán cast-only parser và runtime validator cho hai kết quả khác nhau thế nào.

Completion task

Làm `tests/work/p08_api.spec.ts`:

- GET services status/header/shape;
- invalid customer update trả 400;
- owned customer update rồi UI thấy đúng phone;
- cleanup đúng owner.

Enable bốn test sau attempt và chạy `npm run lab:p08`.

Independent task

Từ một API contract nhỏ, thiết kế matrix positive, boundary, invalid shape, unauthorized và not-found. Implement tối thiểu sáu case không duplicate boilerplate quá mức.

Failure injection

- cast JSON bằng `as Service[]` dù body sai;
- hardcode shared resource;
- chỉ assert 200;
- cleanup toàn bộ state;
- reuse auth context sai role;
- retry POST không idempotent mà không hiểu side effect.

Transfer

Với Laundry, dùng Network/Playwright request để hiểu form POST/redirect/status/CSRF, không thêm JSON adapter. API portfolio có thể nằm ở sandbox/repo riêng. SQL/service/MockMvc mới là proof sâu của Laundry.

Gate P08

Tự viết một API matrix có runtime shape check, negative auth/input case, owned data và một UI-API hybrid flow. Giải thích rõ layer nào được và chưa được chứng minh.

P09 - Debug bằng evidence, không bằng ritual

Problem

Test fail ở dòng click không có nghĩa click là root cause. Login trước đó có thể fail, data không tồn tại, overlay che button, locator match sai, app trả 500 hoặc CI thiếu environment.

Mental model

1. Read the first meaningful error
2. State expected versus actual
3. Classify likely layer
4. Write 2-3 falsifiable hypotheses
5. Choose the cheapest decisive evidence
6. Reproduce narrowly
7. Apply the smallest fix
8. Run regression proportional to risk

Evidence map

Layer	Evidence chính	Câu hỏi
test code	stack, assertion diff, call log	test đang yêu cầu gì?
locator/DOM	trace snapshot, accessibility view	match 0, 1 hay nhiều?
timing/actionability	action log, video/trace timeline	check nào chưa pass?
browser JS	console, page error	client crash?
HTTP	Network, status, redirect, request body	request/response contract?
server	structured log/correlation ID	handler/service nào fail?
DB/data	owned records/query evidence	seed/state/constraint?
environment	versions/env/ports/artifacts	local và CI khác gì?

Trace workflow

1. mở trace từ failure, không click ngẫu nhiên;
2. chọn action đầu tiên lệch prediction;
3. so DOM before/after, call log và Network;
4. ghi một evidence loại trừ ít nhất một hypothesis;
5. chỉ sau đó mở source/patch.

Official references: [Trace Viewer](#) và [Debugging tests](#).

Worked diagnosis

Symptom:

```
Timeout 5000ms exceeded while waiting for getByRole('button', { name: 'Luu' })
```

Reasoning không hợp lệ: “CI chậm, tăng lên 30s.”

Reasoning kiểm chứng được:

```
Expected: one enabled visible Luu button after login.
Actual: locator resolved zero nodes.
H1: still on login because auth state expired.
H2: accessible name changed to Cập nhật.
H3: route returned error page.
```

Evidence: current URL + trace DOM + response status.

Nếu URL là `/login`, locator không phải vấn đề chính.

Flake taxonomy

Nhóm	Ví dụ	Fix class
test race	missing await, wrong signal	dependency/assertion
app race	async save/duplicate command	product synchronization/idempotency rule
shared data	order reused across workers	ownership/reset
selector	DOM order/duplicate text	semantic scope
environment	timezone/version/secret/port	reproducible config
external dependency	unstable service	fake/contract/environment strategy

Retry pattern là evidence: pass ở retry có thể là flake, nhưng cũng có thể là test data side effect khiến lần hai khác lần một.

Prediction

Trước khi mở source của từng seeded bug, ghi expected/actual, hai hypothesis và một evidence rõ nhất có thể loại trừ ít nhất một hypothesis. Mười phút đầu chỉ được đọc error, trace, URL/DOM, Network hoặc state evidence; chưa patch.

Completion task

Lab `tests/work/p09_seeded_bugs/` có ít nhất bốn bug:

- missing await;
- duplicate locator;
- shared owner;
- wrong environment/base URL.

Mỗi bug có note riêng trong `tests/work/p09_seeded_bugs/bug-notes/`: expected, actual, hypotheses, decisive evidence, root cause, smallest fix, regression scope. Không gộp bốn root cause vào một file chung.

Chạy riêng bằng `npm run lab:p09:missing-await`, `npm run lab:p09:duplicate-locator`, `npm run lab:p09:shared-owner` và `npm run lab:p09:wrong-environment`. Sau khi cả bốn đã được enable/sửa, chạy regression `npm run lab:p09`.

Independent task

Lấy một failure thật được phép ở internship hoặc Laundry. Không gửi source/secret ra ngoài. Viết postmortem đủ để một teammate hiểu cách tái hiện và lý do fix.

Transfer

Chuyển triage card sang một failure mới nhưng giữ bí mật: chỉ ghi symptom đã sanitize, layer, evidence, root cause class và regression. Với Laundry, map evidence qua browser → controller/service → PostgreSQL thay vì mặc định đổ cho locator.

Failure injection

Sau khi sửa, tự gây lại lỗi bằng cách đảo một biến. Nếu regression test vẫn xanh khi bug quay lại, oracle chưa bắt đúng failure.

Gate P09

Trong 30 phút, chẩn đoán một seeded bug lạ. Không patch trong mười phút đầu. Diagnose đạt 2 khi evidence loại trừ được hypothesis và regression fail đúng trước/fix xanh sau.

P10 - CI cho test automation ở mức đi làm

Mental model

CI là một máy/runner bắt đầu gần clean state và thực hiện workflow lặp lại. Nó không phải phép màu làm test chuyên nghiệp hơn.

```
checkout
-> install pinned dependencies
-> install browser/system dependencies
-> start target or point to safe environment
-> run typecheck/tests
-> preserve report/trace/log when needed
-> publish pass/fail signal
```

Worked GitHub Actions baseline

```
name: Playwright checks
on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npm run typecheck
      - run: npm run check:kit
      - run: npm test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report
```

Pin action major versions per current official guidance and review third-party action trust. Secrets đi qua repository/environment secrets, không echo ra log và không commit `.env`.

CI không dạy gì thay test design

- Cache sai có thể giữ artifact/dependency cũ.
- `workers=1` là baseline chuẩn đoán, không chứng minh suite parallel-safe.
- Retry che noise tạm thời nhưng phải track flake.
- Artifact `if: always()` giúp debug cả failure; retention phải phù hợp dữ liệu.
- Test trên production cần explicit approval/risk boundary, không phải default.

Prediction

Đọc workflow broken nhưng chưa sửa. Với từng branch, dự đoán stage fail đầu tiên, log đặc trưng và artifact nào còn/không còn. Phân biệt “test đố” với “runner chưa đủ dependency để chạy test”.

Completion task

Phân biệt hai workflow trước khi sửa:

- `labs/ci/broken/playwright.yml` và `fixed/` là vật liệu chẩn đoán;
- executable starter kiểm tra contract YAML ở `tests/work/p10_ci.spec.ts`;
- `.github/workflows/playwright.yml` là baseline an toàn chỉ gate reference, vì work specs

còn `fixme`. Trong bài P10, m enable một work spec trên branch disposable để chứng minh deliberate red + artifact, rồi phục hồi; không commit một pipeline luôn đổ vào pack.

Workflow lab có ba failure branch:

1. bỏ browser install;
2. dùng `npm install` khi lockfile drift;
3. chỉ upload report khi success.

Sửa từng branch và ghi failure nằm ở infrastructure hay test.

Enable ba contract tests trong `tests/work/p10_ci.spec.ts` sau attempt và chạy `npm run lab:p10`.

Independent task

Từ clean clone/folder mới, chạy đúng command README. Sau đó dựng một pipeline xanh và tải artifact của một run cố ý đổ.

Failure injection

- thiếu env;
- port collision/webServer không ready;
- Node version khác local;
- timezone khác;
- `test.only` lộn branch;
- artifact path sai;
- secret in log.

Transfer

Docker ở đây dùng thế nào?

Nếu Laundry dùng Compose để chạy PostgreSQL/app, CI hoặc local setup có thể gọi file đã được project kiểm soát. M chỉ cần đọc service, image/build, port, env, volume, healthcheck và logs. Không dành tuần riêng tối ưu image hoặc học orchestration.

Gate P10

Pipeline chạy từ clean checkout, typecheck + test, giữ artifact khi fail. M cố ý làm đổ, chỉ đúng stage/root cause và sửa mà không thử mò YAML.

P11 - Company-style capstone ticket

Artifact bắt buộc đã có sẵn: `tests/work/p11_capstone.spec.ts` và bốn card trong `labs/p11/`. Chạy `focused` bằng `npm run lab:p11`; không viết test trước khi `REQUIREMENT_PARAPHRASE.md`, `RISK_AND_ORACLE.md` và `DATA_OWNERSHIP.md` có attempt thật.

Mental model

Capstone không thêm API mới; nó buộc m ghép cùng một chain đã học:

```
ticket -> paraphrase -> risk/oracle -> owned state -> focused tests
      -> failure evidence -> smallest fix -> regression/CI -> teach-back
```

Mỗi artifact khóa một kiểu brainrot: code trước khi hiểu requirement, expected lấy từ actual, data không owner, debug thử mò hoặc “CI xanh” vì test bị skip.

Worked example

Với một ticket khác “user đổi display name”, một paraphrase tốt phải nói actor, precondition, command, observable success và forbidden outcome. Risk card tách happy save, invalid input, unauthorized actor và duplicate submit. Đây chỉ là mẫu cách suy nghĩ; không dùng nó làm answer cho order/phone ticket bên dưới.

Prediction

Trước code, ghi expected route/status/UI outcome cho ba actor: anonymous, STAFF và MANAGER. Vẽ POST → redirect → GET cho happy path, rồi dự đoán test nào sẽ bắt wrong order, shared owner, missing await và server authorization bị bỏ quên.

Completion task

Ticket

Sandbox nhận một thay đổi: “Staff tìm order theo business code, mở detail, cập nhật phone thuộc data owner và thấy success sau PRG. Anonymous không được vào protected route; STAFF không được gọi Manager action.”

M không được bắt đầu bằng code. Tạo artifact theo thứ tự:

1. requirement paraphrase;
2. risk/test-condition matrix;
3. data ownership plan;
4. locator/oracle plan;
5. implementation tối thiểu;
6. một seeded failure và bug journal;
7. CI evidence;
8. teach-back ba phút.

Constraints

- TypeScript strict;
- semantic locators;
- không `waitForTimeout`;
- test chạy độc lập/reordered;
- auth state không commit;
- API dùng cho setup chỉ khi boundary rõ;

- ít nhất một case âm;
- trace/report khi fail;
- không mega-POM.

120-minute simulation

Phân	Thời gian	Output
đọc ticket + test design	15 phút	conditions/risks
code happy + negative	45 phút	focused specs
seeded bug	25 phút	evidence note
clean/repeat/CI	20 phút	commands/artifact
teach-back	15 phút	explanation

Sau attempt thật ở ba test, bỏ marker `fixme` và chạy `npm run lab:p11`. Không được coi command bị block hoặc test skipped là tiến độ.

Independent task

Giữ cùng contract nhưng đổi một dimension không có trong starter: DOM order đảo, phone invalid hoặc session hết hạn. Viết một test condition mới từ risk trước khi thêm code và nói regression scope nào đủ, scope nào quá rộng.

Failure injection

Sau khi suite xanh, tự cài đúng một lỗi trong bốn nhóm: locator theo index, shared owner, missing await hoặc UI-only authorization. Ghi vào `labs/p11/BUG_JOURNAL.md`, chứng minh test đỏ, restore fix rồi chạy focused + typecheck. Không giữ bug trong learner work sau evidence.

Transfer

Lấy một ticket được phép ở công ty, sanitize mọi name/URL/secret, rồi tái dùng bốn card requirement, risk/oracle, ownership và bug journal. Không copy canonical Laundry behavior sang hệ thống công ty.

Gate P11 - Graduation Playwright

Pass khi:

- không AI trong prediction và transfer;
- tự viết được flow mới 6-10 bước;
- locator chịu reorder/duplicate text;
- assertion chứng minh outcome;
- data owner rõ và repeat/reorder xanh;
- auth negative đúng;
- tìm seeded bug bằng evidence;
- pipeline có artifact;
- giải thích code, state và failure path.

Nếu test xanh nhưng không giải thích được missing `await`, fixture lifecycle hoặc root cause seeded bug, quay lại đúng P lesson bị lộ.

PHẦN IV - Java/Spring reverse và tự làm toàn bộ SWP391 Laundry

Mục tiêu phần này không phải biến m thành Java backend specialist trong ba tuần. Mục tiêu là đủ Java/Spring để tự xây, test, lần trace và defense toàn bộ canonical Laundry trong kỳ SWP hai tháng.

Đây là một khối lượng lớn nếu vừa internship vừa học từ zero. Vì vậy mỗi feature đi theo vertical slice và Definition of Done; không scaffold 30 controller rồi gọi là tiến độ.

Evidence cards khi repo Laundry thật xuất hiện

Part IV không scaffold một Spring app giả. Các completion chỉ chạy trên repo Laundry thật m cung cấp, nhưng card ghi evidence đã có trong `lab-kit/labs/spring-reverse/`:

Module	Card thật
J00	J00_JAVA_READING.md
S01	S01_HTTP_CONTAINER_TRACE.md
S02	S02_MVC_PRG_TRACE.md
S03	S03_TRANSACTION_TRACE.md
S04	S04_PERSISTENCE_TRACE.md
S05	S05_SECURITY_TRACE.md
S06	S06_TEST_LAYER_MATRIX.md
S07	S07_FULL_VERTICAL_TRACE.md

Card compact `REVERSE_TRACE_CARD.md` và disturbance card UC-09 vẫn được giữ. Chưa có repo thì chỉ làm reading/model task; không giả vờ completion đã có implementation evidence.

J00 - Java reading bridge cho người zero

Mental model

- TypeScript chạy automation tests.
- Java chạy production Laundry.

M học cả hai qua cùng mental model: values, types, functions/methods, objects, errors và modules/packages. Không cố dịch từng cú pháp một-một.

Worked example

```
public OrderDetailView getOrderDetail(String orderCode) {
    LaundryOrder order = orderRepository.findByOrderCode(orderCode)
        .orElseThrow(() -> new OrderNotFoundException(orderCode));
    return orderViewMapper.toDetail(order);
}
```

Đọc từ ngoài vào:

- `public`: method được gọi từ object khác;
- `OrderDetailView`: kiểu trả về;
- `getOrderDetail`: tên hành vi;
- `(String orderCode)`: input typed;
- `local variable order` giữ reference tới entity;
- `repository` trả `Optional`; thiếu thì throw domain exception;
- `mapper` tạo deliberate view model;
- `return` gửi result về caller.

Prediction: nếu code không tồn tại, dòng mapper có chạy không? Exception đi về tầng nào?

Class, record, interface

```
public record CustomerForm(
    @NotBlank String fullName,
    @NotBlank String phone,
    String note
) {}
```

Record phù hợp data carrier bất biến theo field reference. Annotation cung cấp metadata cho framework/validator; annotation không tự chạy nếu pipeline không kích hoạt nó.

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    Optional<Customer> findById(Long id);
}
```

Interface mô tả contract. Spring Data tạo implementation runtime cho pattern được hỗ trợ; query phức tạp vẫn cần contract rõ và test PostgreSQL.

Constructor injection

```
@Service
public class CustomerService {
    private final CustomerRepository customerRepository;

    public CustomerService(CustomerRepository customerRepository) {
        this.customerRepository = customerRepository;
    }
}
```

Object không tự `new` repository. Spring container tạo bean và đưa dependency vào constructor. Lợi ích: dependency explicit, test thay bằng fake/mock khi phù hợp, object không giấu global lookup.

Collections và stream vừa đủ

```
BigDecimal total = items.stream()
    .map(OrderLine::lineAmount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
```

Đây là pipeline array-like: source -> map -> reduce. Nhưng money rule không được viết kiểu ngắn đến mức mất rounding/validation. Với code khó giải thích, loop rõ có thể tốt hơn stream clever.

Exception mental model

Expected business conflict khác system failure:

- invalid shape: binding/Bean Validation, trả field error;
- business conflict: service thấy state không hợp lệ, map thành page-level 409;
- not found: 404;
- wrong role/CSRF: 403;
- unexpected persistence/system error: generic 500 + correlation log.

Không catch `Exception` rồi return `null`. Exception phải giữ meaning và transaction rollback contract.

Prediction

Không chạy code, đọc method mẫu rồi ghi: mapper có chạy khi repository không tìm thấy không; exception đi qua caller nào; dependency nào là field, value nào là local; đổi return type hoặc quên return tạo compile feedback ở đâu. Với stream money snippet, tự tính hai line trước khi tin output.

Completion task

Từ một Controller/service/repository thật, gạch:

Ghi vào `lab-kit/labs/spring-reverse/J00_JAVA_READING.md`.

- input type;
- return type;
- dependency;
- branch;
- side effect;
- exception path.

Independent task

Viết Java thuần cho `PhoneNormalizer.normalizeStoredPhone` với tests: trim, remove space/dot/hyphen, +84 -> 0, đúng 10 digits bắt đầu 0, invalid throw. Chưa dùng Spring.

Failure injection

- thiếu `return`;
- dùng `double` cho VND;
- field mutable per-request trong singleton service;
- catch exception và nuốt;
- `Optional.get()` không check;
- entity dùng làm form.

Transfer

Chọn một class Java thật được phép đọc ở internship hoặc Laundry. Lập bảng “Java symbol → vai trò” cho constructor, field, method, parameter, return/throw và test caller. Không copy source riêng tư vào card; chỉ ghi symbol/path và flow đã sanitize.

Gate J00

Đọc được class khoảng 40-80 dòng, kể dependency/call flow, viết một method + JUnit test và sửa compile error mà không nhờ AI viết patch.

S01 - Từ browser tới Servlet container

Problem

Browser không “gọi Controller”. Nó gửi HTTP bytes tới port. Controller chỉ xuất hiện sau khi server/container/filter/framework đã xử lý nhiều bước.

Mental model

```

Browser
-> TCP/HTTP listener trong embedded Servlet container
-> HttpServletRequest / HttpServletResponse
-> Servlet FilterChain
-> DispatcherServlet
-> Spring MVC delegates
-> Controller
-> Service
-> Repository/JPA
-> PostgreSQL
-> Model/ViewResolver/Thymeleaf
-> HTTP response
-> DOM

```

Spring Boot auto-configure hạ tầng; nó không xóa Servlet API. Container quản lifecycle và concurrency. Nhiều request có thể đi qua cùng singleton Controller/Service bean, nên không lưu per-request mutable data trong bean field.

Worked example

Với GET /services:

1. browser gửi method/path/headers/cookie;
2. container tạo request/response objects;
3. filters chạy;
4. DispatcherServlet tìm handler;
5. `PublicCatalogController.listActiveServices` gọi `CatalogService`;
6. repository query active services;
7. Controller đưa typed view data vào model;
8. Thymeleaf render HTML;
9. browser parse thành DOM;
10. Playwright locator/assertion quan sát DOM.

Evidence card

```

User outcome:
HTTP method/path/query/body:
Request status/redirect/headers:
Selected handler:
Bound input object:
Service command/query:
Transaction boundary:
Repository/query/table:
Rendered view/redirect:
Playwright oracle:

```

Prediction

Với một GET public và một GET protected, dự đoán method/path, filter outcome, handler có/không được gọi, status/redirect và DOM oracle. 404, 302, 403, 500 phải có first differing layer riêng.

Completion task

Chạy một GET thật. Dùng Network và breakpoint/log ở Controller. Ghi method, URI, status, selected method, SQL/query và view.

Ghi vào `lab-kit/labs/spring-reverse/S01_HTTP_CONTAINER_TRACE.md`.

Independent task

Trace một GET có query filter như `/orders?status=RECEIVED&page=1`. Chỉ rõ query parameter khác path và body.

Failure injection

- route 404;
- unauthenticated 302;
- wrong role 403;
- Controller throw 500;
- view template missing;

Mỗi case phải chỉ tầng đầu tiên tạo khác biệt.

Transfer

Từ một Network entry ở app được phép, kể chain tới handler và response mà không dùng câu “Spring tự lo”. Nếu chưa có repo Laundry thật, chỉ điền prediction/model; không tự tạo Controller giả để lấy điểm.

Gate S01

Không dùng câu “Spring tự map”. Kể container -> request/response -> filters -> DispatcherServlet -> handler -> response bằng object/tầng cụ thể và evidence.

S02 - DispatcherServlet, binding, validation và PRG

Mental model

DispatcherServlet là front controller. Nó dùng delegate:

```
HandlerMapping -> chọn handler
HandlerAdapter -> invoke phù hợp
Argument resolvers/data binding -> tạo input
Validator -> shape validation
Controller -> navigation/delegation
ViewResolver -> view rendering
Exception resolvers -> error mapping
```

Worked example

```
@PostMapping("/customers")
public String createCustomer(
    @Valid @ModelAttribute("form") CustomerForm form,
    BindingResult bindingResult,
    RedirectAttributes redirectAttributes) {
    if (bindingResult.hasErrors()) {
        return "customers/form";
    }

    long customerId = customerService.createCustomer(form);
    redirectAttributes.addFlashAttribute("success", "Đã lưu khách hàng");
    return "redirect:/customers/detail?id=" + customerId;
}
```

Controller xử lý binding/navigation. Service revalidate business truth. Form không gửi trusted actor ID, calculated money, status hoặc timestamps.

Shape versus business validation

Layer	Câu hỏi	Ví dụ
binding/conversion	text có đổi được sang kiểu?	abc thành BigDecimal thất bại
Bean Validation	field shape đúng?	blank, length, pattern
service	state/trust/cross-row đúng?	service active, status eligible, amount <= remaining
database	integrity cuối	check, FK, unique

Database constraint không thay user-facing validation. Browser validation không thay server validation.

PRG

Successful mutation:

```
POST /orders
-> 302 Location: /orders/detail?code=...
-> browser GET detail
-> 200 HTML
```

Refresh lặp GET, không resubmit POST. Flash sống đúng một request redirect. PRG giảm duplicate submit nhưng không phải distributed idempotency; service vẫn khóa/revalidate.

Prediction

Với ba input: malformed number, field blank và value hợp shape nhưng vi phạm business state, dự đoán tầng reject đầu, service có được gọi không, response 200/302/409 và safe field nào còn lại. Vẽ riêng POST response và redirect GET; đừng gộp thành một request.

Completion task

Với `CustomerForm`, implement shape error giữ safe values và không gọi service. Valid input gọi command một lần rồi redirect.

Ghi request/binding/redirect evidence vào `lab-kit/labs/spring-reverse/S02_MVC_PRG_TRACE.md`.

Independent task

Với `CreateOrderForm`, vẽ field nào browser submit và field nào server phải reload/derive. Unit price, total, actor, status, order code và time không được trusted từ form.

Failure injection

- đặt `BindingResult` sai vị trí;
- redirect khi binding error làm mất field error;
- dùng entity làm form;
- tin `staffId` từ hidden input;
- return 200 success page sau POST thay vì PRG;
- persistent toast trong session.

Transfer

Đọc một form flow công ty/Laundry được phép và tách browser-owned text khỏi server-derived actor, money, code, status và time. Ghi route/form/service conflict/PRG oracle; không đổi app chỉ để khớp mẫu.

Gate S02

Tự implement một GET form + POST form với typed input, field error, service conflict, PRG và one-request flash. Trace được 200 validation versus 302 success versus 409 conflict.

S03 - Service, transaction và business invariant

Mental model

Service method biểu diễn một business command, không phải collection CRUD tùy ý. Transaction là all-or-nothing boundary cho command.

```
@Transactional
public OrderCode createOrder(CreateOrderCommand command, CurrentOperator actor) {
    // reload authoritative rows
    // validate active service and quantities
    // calculate/snapshot line values
    // persist order/items/initial history atomically
    // return business code
}
```

Controller không bắt đầu nửa transaction, repository không tự commit từng phần của cùng command.

Worked example

Canonical money model

Mọi amount dùng `BigDecimal`; VND scale 0. Line:

```
lineAmount = round(unitPrice * quantity, scale 0, HALF_UP)
orderTotal = sum(already-rounded positive line amounts)
```

Browser display không phải pricing source. Payment ledger immutable:

```
collected = sum(COLLECTION)
refunded  = sum(REFUND)
netPaid   = collected - refunded
remaining = max(orderTotal - netPaid, 0)
refundDue = cancelled ? netPaid : 0
```

Không persist `paid_amount`, `refunded_amount`, `remaining_amount` trên order.

Lifecycle

```
RECEIVED -> PROCESSING -> READY_FOR_PICKUP -> COMPLETED
|
-> CANCELLED
```

Chỉ RECEIVED được cancel. Không backward transition. Open issue chặn ready/pickup. Pickup cần READY + remaining zero + no open issue.

Concurrency

Command trên existing order phải lock row `laundry_order` bằng pessimistic write trước khi đọc lại status/ledger/issue guards. Hai full-balance collection không được cùng pass stale check.

UC-09 chưa có order row để lock. Nó lock selected active catalogue rows theo ascending `service_id`, validate/snapshot và insert order/items/history atomically.

Prediction

Trước code, điền decision table cho amount zero/negative/exceeds, terminal status, open issue và hai submit đồng thời. Với mỗi row, dự đoán lock/reload/guard/write order, exception và số history/ledger row sau failure.

Completion task

Viết service test cho `recordCollection`:

Ghi transaction/lock/no-partial-write evidence vào

lab-kit/labs/spring-reverse/S03_TRANSACTION_TRACE.md.

- positive eligible amount;
- zero/negative;
- exceeds remaining;
- cancelled/completed;
- two concurrent full-balance submits exactly one succeeds.

Independent task

Implement `completePickup` từ tests trước: lock -> reload balance/issues/status -> validate -> transition/history/operator/time atomically.

Failure injection

- validate trước khi lock;
- controller tính total;
- update status bằng generic `updateStatus(status)`;
- catch conflict rồi commit partial history;
- dùng `double`;
- lưu remaining duplicate;
- issue resolution tự move order backward.

Transfer

Chọn một command mutation khác trong repo thật. Viết invariant, authoritative rows, transaction boundary, lock target/order và no-partial-write oracle trước khi đọc implementation; sau đó đối chiếu.

Gate S03

Từ BR viết service tests và command có transaction/lock đúng order. Gây exception giữa hai writes và chứng minh rollback, rồi giải thích invariant chứ không chỉ annotation.

S04 - Repository, JPA, PostgreSQL và Flyway

Mental model

JPA mapping không phải schema source. Flyway migration sở hữu DDL; Hibernate `ddl-auto=validate` chỉ kiểm tra mapping tương thích.

```
Flyway versions -> PostgreSQL schema
JPA entity/repository -> mapping/query contract
Service transaction -> when reads/writes commit or rollback
```

Không sửa entity rồi chờ production database tự thay.

Chín bảng canonical

Table	Business purpose
user_account	Staff/Manager credentials/state
customer	counter contact, no login
laundry_service	catalogue/current price
laundry_order	order header/contact snapshot/total/status
laundry_order_item	immutable service/price snapshot per line
order_status_history	transition audit
payment_transaction	immutable collection/refund ledger
order_issue	operational problem/resolution
store_setting	singleton display/receipt info

Không thêm table vì generator gợi ý. Mỗi table mới cần business purpose + UC trace và baseline change approval.

Worked example

Một vertical persistence trace tối thiểu là: Flyway tạo table/constraint → entity map đúng column/type → repository query có grain/order rõ → service dùng trong transaction → PostgreSQL test chứng minh edge. Nếu chỉ thấy entity compile, schema/query contract vẫn chưa được chứng minh.

Query contract

Repository test cần PostgreSQL khi semantics phụ thuộc:

- `ILIKE`/escaped filters;
- timezone grouping;
- pessimistic lock;
- aggregate ledger/report;
- half-open instant range;
- stable ordering/pagination.

Trivial single-row CRUD không cần persistence test chỉ để tăng count.

Time

Business zone duy nhất Asia/Ho_Chi_Minh. Persist event instant bằng `timestampz/Java Instant`, DB session UTC. Browser local datetime chuyển bằng shop zone. Local date report thành range `[start, nextDayStart)`.

Tests inject fixed `Clock`; không dựa máy dev/CI default timezone.

Prediction

Với query order có items + transactions + history, dự đoán JOIN nào nhân row; viết output grain và stable sort trước SQL. Với local-date report, chuyển ngày shop thành range half-open và dự đoán hai event sát nửa đêm thuộc ngày nào.

Completion task

Tạo migration V1 cho identity/customer/catalog, entity mappings và repository smoke. App phải build empty database từ V1 và validate schema.

Ghi migration/query/cardinality evidence vào

`lab-kit/labs/spring-reverse/S04_PERSISTENCE_TRACE.md`.

Independent task

Viết repository query + PostgreSQL test cho payment aggregate hoặc order search có stable sort. Tự giải thích count query/page boundary.

Failure injection

- edit applied migration;
- `ddl-auto=update`;
- enum ordinal;
- eager mọi association;
- fetch join items x transactions x history gây cartesian product;
- group revenue theo order creation date;
- dùng server timezone mặc định;
- unique customer phone.

Transfer

Mở migration/entity/repository test của một feature thật và lập trace ba chiều schema ↔ mapping ↔ query. Nếu repo chưa có, dừng ở thiết kế card; không invent table/class để completion giả.

Gate S04

Từ empty DB chạy toàn migrations, Hibernate validate, repository query đúng edge cases. Chỉ table/index/constraint nào phục vụ BR/UC và chỉ ra SQL evidence.

S05 - Spring Security filter chain

Mental model

Security chạy trong Servlet FilterChain trước DispatcherServlet cho nhiều quyết định. Spring Security thường dùng DelegatingFilterProxy/FilterChainProxy để chọn security chain.

```
request
-> container filter chain
-> Spring Security filters
    authentication/session/CSRF/authorization
-> DispatcherServlet only if allowed/continued
```

403 có thể xuất hiện trước Controller breakpoint. 302 login có thể do entry point, không phải Controller redirect.

Worked example

Canonical boundary

- public GET /, /services, /track, /login và static/error;
- public POST /track, /login theo contract;
- remaining routes authenticated;
- /manager/** và refund Manager-only;
- MANAGER vẫn dùng Staff operations;
- every state-changing POST keeps CSRF;
- inactive account cannot start new session;
- temporary-password session restricted;
- actor derived from authenticated session;
- no remember-me; inactivity timeout 30 minutes.

Prediction

Điền actor × route × method × CSRF matrix trước test. Với mỗi 302/403, dự đoán Controller breakpoint có hit không và evidence filter/entry-point/access-denied nào phân biệt. Menu visibility chỉ là thêm UX oracle, không thay server oracle.

Completion task

MockMvc tests: anonymous redirect, STAFF 403 Manager route, MANAGER access, missing CSRF 403, valid CSRF reaches handler.

Ghi matrix vào `lab-kit/labs/spring-reverse/S05_SECURITY_TRACE.md`.

Independent task

Trace one login through filter/provider/user details/success handler/session fixation to dashboard. Trace forced-password redirect and session ID rotation after password change.

Failure injection

- permit all then hide menu;
- disable CSRF to make test easy;
- trust role in form;
- expose hash/log password;
- different error for unknown versus wrong password;

- reset Manager through Staff admin;
- stale principal keeps `mustChangePassword=true` after success.

Transfer

Trace một protected flow từ cookie/session qua filter/provider/authorization đến handler. Sanitize credential/cookie; không lưu auth state thật vào evidence hay portfolio.

Gate S05

Từ 302/403 symptom, chỉ đúng filter/authorization/CSRF layer bằng evidence. Viết security matrix ở MockMvc và one Playwright smoke without duplicating every case E2E.

S06 - Test pyramid của Laundry

Mental model

Một UI happy path rất có giá trị nhưng chậm và khó bao phủ mọi invariant/concurrency/query edge. Chọn tầng thấp nhất vẫn chứng minh đúng contract.

Tầng	Proof phù hợp	Không phù hợp
Java unit/service	pricing, transition, ledger cap, issue/pickup guards, rollback	HTML/route/real query semantics
MockMvc/web	auth, CSRF, binding, status, redirect, view, service delegation	JPA locks/report SQL thật
PostgreSQL repository	filters, aggregates, lock, timezone, sorting	user-visible navigation
Spring integration	components + transaction + DB	mọi permutation UI
Playwright	critical user journey, semantics, browser/session/PRG	toàn bộ BR matrix

Worked example

```
Service:
- eligible statuses
- positive amount/cap
- actor server-owned
- lock then recompute
- concurrent full balance
```

```
MockMvc:
- anonymous/role/CSRF
- invalid shape
- business conflict 409
- success redirect + flash
```

```
Repository:
- collection/refund aggregate order and exact sum
```

```
Playwright:
- Staff records a valid payment and sees derived balance
- one negative role/state smoke if high risk
```

Test names nói behavior, không lặp method name chung chung.

Prediction

Cho BR-31, UC-25 và một report query, điền trước layer rõ nhất đủ chứng minh, oracle, failure layer và reason không chọn E2E cho mọi permutation. Sau đó đánh dấu một critical journey thực sự cần browser.

Completion task

Lấy BR-31 pickup và phân tầng test. Tạo ít nhất một case fail-before/pass-after cho mỗi layer cần thiết.

Ghi layer/oracle vào `lab-kit/labs/spring-reverse/S06_TEST_LAYER_MATRIX.md`.

Independent task

Thiết kế full proof cho UC-25 public tracking privacy: exact code+phone, generic mismatch, limited fields, no Staff/internal issue leak, cancelled money wording.

Failure injection

- mock repository quá mức nên transaction/SQL never tested;
- repository test chạy H2 thay PostgreSQL;
- E2E chỉ assert URL;
- service test verify implementation calls thay business outcome;
- test fixture seed contradict canonical rules;
- suite order dependency.

Transfer

Lấy một test matrix công ty/Laundry và tìm một case đang ở layer quá đắt hoặc quá mock. Đề xuất move với trade-off rõ; không refactor repo nếu chưa được giao quyền.

Gate S06

Từ một BR, chọn đúng layer cho từng risk và giải thích trade-off. Full build + relevant Playwright proof chạy từ clean state.

S07 - Full reverse trace và random change

Dùng `lab-kit/labs/spring-reverse/S07_FULL_VERTICAL_TRACE.md` cho full chain và `UC09_DISTURBANCE_MATRIX.md` cho disturbance evidence; không để output chỉ nằm trong chat.

Mental model

Full trace không phải liệt kê framework. Nó nối một outcome/UC/BR với object/symbol/evidence thật ở mỗi boundary, rồi chỉ first rejecting layer khi một input thay đổi.

Full trace template

1. Actor + observable outcome + UC/BR
2. Browser DOM/action
3. HTTP request: method/path/query/body/cookie/CSRF
4. Servlet container + request/response
5. Security filters and decision
6. DispatcherServlet + mapping/adapter
7. Binding/conversion/validation
8. Controller navigation/delegation
9. Service command/query + transaction + lock
10. Repository/JPA/query
11. PostgreSQL tables/constraints/rows
12. Response: status/Location/model/view
13. Thymeleaf HTML -> DOM
14. Test oracle at each needed layer

Worked example

Disturb matrix

Với cùng một POST, đôi từng biến riêng:

Disturbance	Expected first rejecting layer	Evidence
missing CSRF	Security filter	403, Controller not hit
malformed number	binding/conversion	field error, service not hit
well-shaped but excessive payment	service	409, no ledger row
concurrent stale balance	transaction/lock/service	one success, one conflict
DB constraint violation bug	persistence/rollback	generic error, zero partial commit
template missing after successful command	view on redirect GET	DB changed, GET 500; regression must expose navigation issue

Hai row đầu minh họa cùng endpoint nhưng missing CSRF dừng trước Controller còn excessive payment đi tới service và phải để ledger unchanged. Đây là cách kể failure path, không chỉ happy arrows.

Prediction

Chọn một GET và một POST chưa trace. Điền full chain và disturbance matrix bằng prediction trước khi đặt breakpoint/chạy test; ít nhất một disturbance phải dừng ở security, một ở binding/service và một ở persistence/view.

Completion task

Hoàn thành `S07_FULL_VERTICAL_TRACE.md` bằng symbol/path thật cho một GET + POST và `UC09_DISTURBANCE_MATRIX.md` bằng evidence fail-before/pass-after. Chưa có implementation repo thì task vẫn open; planning/card không thay runtime evidence.

Independent task

Random change drill

Ví dụ: “Receipt phải hiển thị normalized phone bên cạnh entered phone cho Staff, nhưng public tracking vẫn chỉ dùng snapshot phone và không lộ internal IDs.”

Trước code:

1. affected UC/BR/screens;
2. trusted source/data model;
3. migration needed hay projection-only;
4. Controller/service/repository/view/tests;
5. privacy risk;
6. docs/traceability impacted.

Không sửa file đầu tiên AI gợi ý. Vẽ propagation trước.

Failure injection

Disturb đúng một variable mỗi lần: CSRF, malformed input, business guard, concurrent balance, constraint hoặc template. Ghi first differing layer và no-partial-write/response evidence; restore trước case kế.

Transfer

Trong defense hoặc code review, bốc ngẫu nhiên một route và kể actor → HTTP → security → MVC → service/transaction → DB → view → oracle trong ba phút. Với company repo, dùng terminology thật của repo, không ép canonical Laundry lên nó.

Gate S07

Trong 45 phút, trace một GET và một POST không note. Trong 45 phút khác, implement một approved random change nhỏ, run focused + related tests và explain blast radius.

PHẦN V - Tám tuần tự làm full canonical Laundry

Ranh giới cố định

Production stack:

```
Java 21
Spring Boot 4.1.x Maven JAR
Spring MVC + Thymeleaf
Spring Security session + CSRF
Spring Data JPA/Hibernate
Jakarta Bean Validation
PostgreSQL + Flyway
JUnit 5 + Spring Boot Test + MockMvc
Bootstrap 5 + small project CSS
```

UI tiếng Việt; code/schema/formal docs tiếng Anh. Không JSP/Servlet controllers/JDBC adapter, React/SPA, JWT, REST-first UI, customer account, delivery, promotion, payment provider, notification, multi-branch, inventory, IoT hoặc AI feature.

Canonical baseline có 30 UC, 39 BR, 9 core tables, 5 order statuses và 2 issue statuses. Đây là scope contract, không phải danh sách feature để chia đều cho đẹp.

Solo-full không có nghĩa code một mình trong im lặng

M muốn tự làm toàn bộ để hiểu. Trong team SWP, vẫn phải tuân contribution/process thật và không giành/xóa phần người khác. “Solo-full learning” nghĩa là:

- tự trace và tái hiện được mỗi vertical slice;
- có thể review/test/giải thích toàn bộ hệ thống;
- tự build một branch/lab copy hợp lệ khi team cho phép;
- không tuyên bố tác giả phần teammate làm;
- không đưa code team lên public portfolio khi chưa có quyền.

Nếu team chia task, m hoàn thành ownership trước rồi dùng thời gian học để reproduce/trace các slice khác.

Definition of Done cho mỗi UC

```
catalogue ID/name/actor
-> acceptance + BR
-> screen/navigation
-> route + typed form/query/view
-> Controller
-> business-named Service
-> Repository/entity/migration
-> tests đúng tầng
-> Playwright critical proof khi cần
-> demo seed
-> traceability/document update
-> no-agent teach-back
```

Page render hoặc CRUD row save chưa đủ.

Wave execution card - bắt buộc trước khi chấm gate

Mỗi wave có một card thật trong `lab-kit/labs/swp-waves/`:

Wave	Evidence artifact
W1	W1_EVIDENCE.md
W2	W2_EVIDENCE.md
W3	W3_EVIDENCE.md
W4	W4_EVIDENCE.md
W5	W5_EVIDENCE.md
W6	W6_EVIDENCE.md
W7	W7_EVIDENCE.md
W8	W8_EVIDENCE.md

Card bắt đầu `Status`: `OPEN`. Chỉ đổi gate khi có repo/commit thật, command + test evidence, failure variation và teach-back tương ứng; planning, reference hoặc patch do Agent tạo không tính là learner mastery. Command cụ thể phải lấy từ repo implementation đã inspect, không invent ở coursebook.

W1 - Scaffold và vertical core đầu tiên

Scope

Tuần SWP 1 dựng project một stack và làm vertical slice public catalogue trước, sau đó login skeleton:

- fixed Maven/Spring stack;
- PostgreSQL/Flyway from empty DB;
- feature-package skeleton;
- timezone/Clock configuration;
- security baseline;
- shared layout/error pages;
- UC-05 public active services;
- bắt đầu UC-01 login.

Tại sao UC-05 trước?

Nó nhỏ nhưng đi đủ view -> Controller -> service -> repository -> table -> migration -> test -> Playwright. M học pipeline thật mà chưa gánh transaction phức tạp.

Evidence bắt buộc

- `STACK_PROFILE.txt` là `spring-boot`;
- no alternative frontend/adapter;
- V1 migration build empty PostgreSQL;
- inactive service hidden;
- VND/unit rendered Vietnamese;
- one service/repository/web/Playwright proof phù hợp;
- full reverse trace GET `/services`.

Failure drills

- schema/entity mismatch;
- inactive row leaked;
- enum raw label;
- timezone machine default;
- service query in Controller.

Gate W1

Clean clone builds, DB migrates, `/services` works and m trace được toàn chain. Chưa qua gate thì không scaffold 29 UC rỗng.

W2 - Identity, Staff, customer và catalogue management

UC coverage

- UC-01 Login;
- UC-02 Logout;
- UC-03 Change own password;
- UC-04 Manage Staff accounts;
- UC-06 Manage services/prices;
- UC-07 Manage customers;
- UC-08 Customer order history.

High-value rules

- only STAFF/MANAGER accounts; Manager inherits Staff;
- session actor server-owned;
- active login and generic failure;
- temporary password + forced route + rotation;
- bootstrap Manager protected;
- phone normalized for search but not unique;
- customers have no login;
- catalogue history not rewritten after edit/deactivation.

Build order

1. UserAccount/Flyway/security user details.
2. Login/logout and route matrix.
3. Own password/forced password.
4. Staff management Manager-only.
5. Customer normalization/search/history.
6. Manager catalogue commands.

Tests

- service password/phone/catalogue rules;
- MockMvc session/CSRF/roles/forms;
- repository customer search/history stable order;
- Playwright login/role + customer/catalogue critical flow.

Random change

“Phone input chấp nhận khoảng trắng/dấu chấm/dấu gạch, nhưng stored/public value phải đúng normalization; partial search không được persist.”

Gate W2

STAFF/MANAGER matrix, temporary-password flow, customer duplicate phone và catalogue deactivation đều được chứng minh. M không dùng navigation visibility làm security proof.

W3 - Order intake, search, detail và receipt

UC coverage

- UC-09 Create laundry order;
- UC-10 View/search order list;
- UC-11 View order detail;
- UC-14 Print/reprint order receipt.

Đây là vertical core quan trọng nhất

UC-09 phải:

- select existing customer;
- accept one or more KG/ITEM lines;
- lock selected active services ordered by ID;
- validate quantity and active state;
- snapshot contact/service/unit/price;
- round each line HALF_UP scale 0 then sum;
- generate global-sequence business code with shop-local date;
- insert order/items/initial status history atomically;
- **không ghi payment**;
- redirect detail, rồi UC-20 mới là payment action riêng.

UI/HTTP boundary

Browser không submit trusted unit price, line amount, total, actor, time, status hay code. New Order customer-first; create/edit contact returns through allowlisted context, not arbitrary URL.

Query/detail

Search combines filters and stable sort `received_at DESC, order_code DESC`, fixed page 20. Detail uses focused projections/child queries để tránh items x transactions x history x issues cartesian product.

Receipt dùng business code/snapshots/derived balance/store setting, không raw IDs.

Tests

- pricing/quantity/rounding/atomic snapshot;
- catalogue lock/concurrent edit;
- form tampering ignored/rejected;
- order search filters/range/order;
- detail no N+1/repeated SQL contract;
- Playwright mixed KG + ITEM create and detail/receipt.

Failure drills

- payment fields accidentally in new-order form;
- client total trusted;
- line rounds zero;
- duplicate service lines;

- one write succeeds before later failure;
- receipt uses current catalogue price instead of snapshot.

Gate W3

From empty seeded environment, Staff creates mixed order, detail/receipt show exact snapshots/total and no payment row. M trace UC-09 fully and sửa một validation without AI.

W4 - Revision, cancellation và processing lifecycle

UC coverage

- UC-12 Update received order;
- UC-13 Cancel received order;
- UC-15 View processing queue;
- UC-16 Start processing;
- UC-17 Mark ready.

Revision semantics

Submitted lines are complete desired list:

- existing line carries `orderId`; must belong to route order;
- service ID for existing line cannot change;
- retain old snapshots even if catalogue now inactive;
- new line needs active service/current snapshot;
- omitted existing line removed;
- changing service = remove old + add new;
- no duplicate service;
- recalculated total cannot fall below net paid;
- RECEIVED only.

Lifecycle/queue

Queue contains RECEIVED/PROCESSING, promised time then code. Open issue blocks ready. Every transition locks order and writes immutable status history with actor/time.

Tests

- exact complete-list revision matrix;
- retained inactive snapshot/new inactive rejected;
- foreign/tampered IDs;
- cancellation refund-due behavior/history reason;
- legal/illegal transition table;
- stable queue ordering/overdue display;
- Playwright revise/cancel/processing smoke.

Random change

“Cho phép sửa care note của RECEIVED order nhưng không đổi service/quantity trong một quick-edit form.” Phải quyết định đây là route/form mới hay reuse command; giữ one business command, no generic action dispatcher.

Gate W4

Lifecycle không backward, revision không reprice history, cancellation không delete. Concurrency/lock path được giải thích và tested.

W5 - Issues, immutable money và pickup

UC coverage

- UC-18 Record issue;
- UC-19 Resolve issue;
- UC-20 Record collection;
- UC-21 View payment history;
- UC-22 Print transaction receipt;
- UC-23 Record refund;
- UC-24 Complete pickup.

Invariants

- issue only on RECEIVED/PROCESSING/READY;
- OPEN -> RESOLVED only; no rewash status;
- open issue blocks ready/pickup;
- ledger entries immutable positive collection/refund;
- CASH or BANK_TRANSFER_MANUAL only;
- collection on active order, amount <= remaining;
- refund Manager-only, cumulative <= collected, reason required;
- no money after COMPLETED;
- pickup READY + zero remaining + no open issue;
- same-order commands serialize.

Tests

Service decision tables and concurrent collection are primary. MockMvc covers role/CSRF/status/PRG.

PostgreSQL covers ledger aggregation/history order. Playwright covers one deposit -> issue -> resolve -> ready -> remaining payment -> pickup happy path plus selected negative.

Failure drills

- mutable transaction row;
- refund as negative collection;
- money in UC-09 transaction;
- two collection submits both succeed;
- ready with open issue;
- completion leaves balance;
- Manager-only enforced only in UI.

Gate W5

Full money/issue/pickup flow and each guard pass. Derived values match independent SQL/service oracle; no duplicate balance columns.

W6 - Public tracking, dashboard, reports, settings và schedule

UC coverage

- UC-25 Public tracking;
- UC-26 Operational dashboard;
- UC-27 Revenue report;
- UC-28 Order performance report;
- UC-29 Store settings;
- UC-30 Daily pickup schedule.

Privacy/time/query risks

Public tracking uses exact order code + normalized snapshot phone, same generic mismatch and limited view. No IDs, Staff, internal issue note or other customer.

Revenue filters transaction timestamp in half-open local-date range:

```
gross = COLLECTION sum
refund = REFUND sum
net = gross - refund
```

Order performance cohort uses `received_at`; completion denominator excludes cancelled. Service demand groups distinct service ID + snapshot name + unit; never combine KG and ITEM.

Pickup schedule uses selected local date non-cancelled rows plus carry-over only for today, stable promise/code order.

Tests

- exact/generic public mismatch and privacy;
- fixed Clock dashboard six metrics/attention order;
- timezone boundary near midnight;
- refund on order from another receipt date still counted by transaction date;
- zero/empty denominator;
- distinct services same snapshot name not merged;
- selected-date/carry-over pagination;
- settings display-only.

Failure drills

- revenue by order date/status;
- inclusive `23:59:59` end instead of next-day exclusive;
- OS timezone;
- cancelled order tells customer to pay mathematical remainder;
- public leak issue details;
- opening hours used as hidden promise validation.

Gate W6

All report numbers reproducible by fixed seed/SQL; public view privacy proven; time tests pass in non-Vietnam machine timezone.

W7 - Full regression, documentation và demo integrity

Freeze before document generation

Freeze behavior/schema/routes before final backlog, RDS, SDS, screenshots and diagrams. Source-of-truth order:

```
scope + UC + BR
-> screen/route/form/data contracts
-> code + Flyway + tests
-> verified traceability
-> backlog/RDS/SDS
-> diagrams/final package
```

Do not invent class/sequence diagrams before code exists. Use real symbols/flows.

Regression package

- service transition/money/rule matrix;
- MockMvc auth/binding/CSRF/error/redirect;
- PostgreSQL query/lock/time/report tests;
- one full Spring integration path;
- 20-30 high-value Playwright scenarios, not 30 UCs blindly;
- clean migration seed/demo;
- CI artifacts.

Demo route

```
login -> customer -> mixed order -> deposit -> processing
-> issue -> resolve -> ready -> remaining collection -> pickup
-> public track -> manager reports
```

Use alternate seeded order for cancellation/refund so main flow remains coherent.

Documentation evidence

Every UC name/actor/BR/route/service/table/test agrees. UI screenshots Vietnamese, formal identifiers English. No raw IDs, stale flash, empty report or debug panel.

Gate W7

Plan/code/docs audit green is minimum. M chọn ngẫu nhiên five UCs and trace across every artifact. Clean environment runs without hand-edit DB.

W8 - Defense và graduation

150-minute no-agent simulation

Part	Time	Evidence
closed-book canonical map	15	actors, states, tables, critical rules
unfamiliar UC trace	25	browser-to-DB map
random approved change	45	small diff + tests
seeded bug	30	hypothesis/evidence/regression
Playwright flow	20	independent stable test
teach-back	15	defense answer

Questions m phải trả lời được

- Vì sao customer không có account?
- Vì sao Manager inherit Staff nhưng refund vẫn Manager-only?
- Vì sao UC-09 không ghi payment?
- Vì sao snapshot contact/service/price?
- Vì sao line round trước sum?
- Vì sao balance derive từ immutable ledger?
- Vì sao lock trước reload guards?
- Vì sao no backward status?
- Vì sao report dùng hai cohort timestamp khác nhau?
- Vì sao DB UTC nhưng business zone Việt Nam?
- Vì sao Playwright không thay service/repository tests?
- Vì sao không thêm REST/React/JWT/Docker feature chỉ để “đủ stack”?

Final canonical coverage ledger

Wave	Use cases
W1-W2	UC-01, UC-02, UC-03, UC-04, UC-05, UC-06, UC-07, UC-08
W3	UC-09, UC-10, UC-11, UC-14
W4	UC-12, UC-13, UC-15, UC-16, UC-17
W5	UC-18, UC-19, UC-20, UC-21, UC-22, UC-23, UC-24
W6	UC-25, UC-26, UC-27, UC-28, UC-29, UC-30

Graduation verdict

Pass SWP course và pass v4 không giống nhau.

- **Course pass:** team deliverables meet school rubric.
- **V4 learned:** m explain, modify, test and debug unfamiliar slice without AI patch.

Nếu course đã pass nhưng random-change/seeded-bug gate rớt, giữ project làm phòng gym thêm. Nếu gate đạt nhưng hồ sơ chưa có job, kỹ năng vẫn có giá trị; job timing không được dùng để phủ nhận learning evidence.